

ADL Lite: An Event-First Capability-Lifecycle Registry for LLM Agent Ecosystems

Anonymous Authors
Affiliation withheld for peer review

Abstract

LLM agents are proliferating across enterprise and open ecosystems, each advertising tools, APIs, and knowledge-domain capabilities. Yet no lightweight, verifiable, lifecycle-aware registry exists to govern what a capability claims, how it evolves, and whether it remains trustworthy. Existing governance layers—KYA for permissions and AgentSafe for architecture-level oversight—leave this gap unaddressed. ADL Lite fills it as an event-first capability-lifecycle registry. Drawing on Wittgenstein’s thesis that “the world is the totality of facts, not of things,” ADL Lite treats events—not objects—as the fundamental ontological primitive, while distinguishing occurrents (events as they occur) from their immutable records (EventChains). Capabilities are modeled as append-only, cryptographically linked EventChains, with all lifecycle state (status, confidence, validators) derived exclusively from event history through deterministic functions. Every capability claim, validation, deprecation, and fork is an auditable event; this design eliminates stored mutable state and the consistency defects it introduces.

We provide a two-level ontological analysis that situates ADL Lite within foundational ontologies (BFO, DOLCE, UFO), formalizes identity conditions and three forms of ontological dependence, and defines a decidable precondition language for lifecycle transitions. We further provide a complete formal semantics, together with seven proved properties including determinism, confluence under fork, and CRDT convergence.

We empirically validated the registry on 201 EventChains derived from the IBM Anti-Money Laundering dataset (9,300 events), confirming zero integrity failures and linear scalability under architectural stress tests. These findings are supplemented by concurrency stress tests, adversarial integrity trials, and comparative baselines against nanopublications and Git-native systems. This paper is positioned as a framework paper establishing the ontological and formal foundations of capability-lifecycle governance. The registry is implemented as a pip-installable Python toolkit operating on Markdown files in standard Git repositories, targeting lightweight deployment scenarios where full OWL/SPARQL infrastructure is unavailable or unnecessary.

Keywords: agent governance; capability registry; lifecycle governance; event sourcing; multi-agent systems; provenance; operational ontology

1 Introduction

1.1 Background: The Agent Capability Governance Gap

Large language model (LLM) agents are proliferating across ecosystems. Each agent exposes a set of *capabilities*: tools, APIs, knowledge domains, and interaction protocols. A capability is a claim—“I can retrieve weather data,” “I can execute SQL,” “I can reason about financial risk”—that other agents and human operators must evaluate before delegation. But who records what a capability does, how it evolves, and whether it is trustworthy? Today, capability claims are scattered across README files, API documentation, and prompt templates. They are revised without audit

trails, deprecated without ceremony, and forked without attribution. A lightweight, verifiable, lifecycle-aware registry of agent capabilities does not yet exist.

Existing governance layers. Three recent lines of work address adjacent aspects of agent governance, but none fills the capability-registry gap.

KYA (*Know Your Agent*, arXiv:2605.25376) is a framework-agnostic trust layer for agent systems that provides a permissions model with HMAC-chained integrity Authors TBD [2025c]. It achieves approximately 1,800 operations per second and is available as the `veldt-kya` package on PyPI. KYA governs *who* can invoke *what*, but it does not govern the *lifecycle* of a capability itself: there is no **VALIDATE** event that records an independent audit of a tool’s behavior, no **DEPRECATE** event that marks a retired protocol, and no **FORK** event that traces a community derivative. KYA secures the permission boundary; it does not register the capability’s provenance.

AgentSafe (arXiv:2512.03180) is a unified governance framework spanning design-time, runtime, and audit phases Authors TBD [2025a]. It provides architecture-level governance—data-flow validation, policy enforcement, and safety guardrails—but it is heavyweight and requires integration with a service mesh or runtime orchestrator. It does not offer a lightweight, Markdown-native format that a single agent can author, cryptographically link, and commit to version control without enterprise infrastructure.

LLM-native ontology engineering (Talukdar et al., arXiv:2604.23090; LLM4ACOE) uses multi-agent LLM pipelines to generate OWL ontologies from natural-language requirements Authors TBD [2025e]. These systems produce static taxonomies and relation schemas, but they do not govern the *runtime lifecycle* of the concepts they create: there is no event-sourced record of validation, deprecation, or fork, and no tamper-evident chain linking an agent’s assertion to its subsequent revision or retirement. The ontology is produced, not governed.

Two additional bodies of work are complementary but distinct. **“From Agent Traces to Trust”** (arXiv:2606.04990) surveys provenance frameworks for agent systems and explicitly notes that “a unified trace schema is still needed” Authors TBD [2025b]. ADL Lite is that schema: an event-first, cryptographically linked trace format in which every capability claim, validation, deprecation, and fork is a first-class auditable event. **SafeAgent** (arXiv:2604.17562) is a safety protocol for LLM agents that defines red-team evaluation procedures and harm-classification taxonomies Authors TBD [2025d]. It governs *safety assessment*, not *capability registration*; its concern is whether an agent is dangerous, not whether its advertised capabilities are authentic, current, and traceable.

Defining the gap. No existing system simultaneously satisfies four properties that define the capability-registry design space: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. KYA satisfies (b) and (d) but lacks (a) and (c). AgentSafe partially satisfies (c) but lacks (a), (b), and (d). LLM-native OE tools satisfy (a) in a limited sense (natural-language input) but lack (b), (c), and (d). The intersection of all four properties remains unoccupied.

Defining “operational ontology.” We use the term *operational ontology* in a specific sense that requires explicit definition, because the term carries distinct meanings in adjacent literatures. In the OBO Foundry ecosystem, an “operational ontology” typically means an application ontology deployed in a running bioinformatics pipeline (as opposed to a reference ontology) Smith et al. [2007]. In programming-language theory, “operational semantics” defines the meaning of a program through its execution on an abstract machine. In knowledge-engineering practice, “executable ontologies”

(e.g., UFO-B executable specifications Guizzardi et al. [2024]) are ontologies equipped with first-order logic axioms that can be evaluated by a reasoner. ADL Lite’s usage differs from all three: we mean an ontology in which the *operational semantics of lifecycle transitions*—preconditions, state derivation, and consensus rules—are co-located with the *data structure* (EventChain) in a single lightweight package. In the present framing, the “ontology” is the set of capability concepts an agent ecosystem manages; the “operational” aspect is the fact that the same YAML-declared rules that define what can happen to a capability are enforced at append time by the ActionExecutor, without requiring an external workflow engine, OWL reasoner, or triple store. This co-location of governance semantics and data structure is the defining characteristic of ADL Lite’s operational ontology.

Ontological novelty of the event-first stance. We emphasize that ADL Lite’s contribution is not the philosophical thesis that “events are fundamental”—a position already well-established by Wittgenstein’s *Tractatus* (§1.1), BFO’s *occurrent*, DOLCE’s *perdurant*, and UFO-B’s *event* Wittgenstein [1922], Arp et al. [2015], Gangemi et al. [2002], Guizzardi [2005]. Rather, ADL Lite’s contribution is the *operationalization* of this stance: the translation of an event-first philosophical commitment into a deployable, Markdown-native, cryptographically enhanced capability-lifecycle mechanism. In foundational ontologies, events and continuants are typically treated as co-equal top-level categories; ADL Lite treats events as the *sole architectural primitive* from which all lifecycle state—status, confidence, validator identity—is derived through deterministic functions. This is an engineering reduction, not a metaphysical elimination: capabilities are not denied ontological standing, but their *computable* properties (status, confidence) are made dependent on event history. This operationalization distinguishes ADL Lite from descriptive event ontologies (CIDOC CRM, SEM/LODE) that record what happened without enforcing what can happen. It also distinguishes ADL Lite from cryptographic provenance systems (nanopublications, PROV-O) that provide tamper detection without lifecycle state machines.

Why existing families are insufficient. We demonstrate systematic awareness by showing that representative systems from five distinct families fail to cover the full intersection of properties (a)–(d):

Descriptive event ontologies (CIDOC CRM, SEO). CIDOC CRM provides event-centric descriptive modeling and is widely deployed in cultural heritage ICOM-CIDOC [2014]. The Subject-Event Ontology (SEO) offers event-first patterns for web-scale data Boldachev [2025]. Both fail to satisfy (b) and (c): they record what happened without cryptographic integrity guarantees and without declarative preconditions for lifecycle state transitions. Moreover, production deployments typically require reasoning-enabled RDF/OWL toolchains, triple stores, and SPARQL endpoints, violating (d).

Cryptographic provenance systems (PROV-O, nanopublications with Trusty URIs). PROV-O provides a W3C-standard provenance vocabulary W3C [2013]; nanopublications with Trusty URIs achieve cryptographic integrity for atomic claims through hash-based URIs Kuhn et al. [2015], Kuhn and Dumontier [2015]. However, PROV-O lacks (b) natively: it describes provenance without built-in tamper detection, and it requires RDF tooling that conflicts with (a) and (d). Nanopublications satisfy (b) for atomic claims but lack (a) because they require RDF serialization and lack (c) because they provide no built-in lifecycle governance or state-machine preconditions.

Markdown-native knowledge tools (Foam, Obsidian). These tools satisfy (a) and (d) by design: they operate on plain Markdown files with minimal installation overhead Foam Contributors [2020], Obsidian Team [2024]. However, they provide neither (b) nor (c): they lack cryptographic

hash chaining and provide no lifecycle state machines or precondition enforcement.

Summary. Each family lacks at least one property, and no family satisfies all four. ADL Lite occupies this intersection by integrating these properties into a coherent operational ontology, not by inventing any individual component.

1.2 Research Gap: Capability Lifecycle Governance as Ontological Analysis

The specific research gap addressed in this paper can be formulated as follows: *How can an event-first capability registry achieve lifecycle traceability and multi-agent governance without requiring object-first mutable state, expert curation, or heavyweight tooling?* This gap has both an engineering dimension and an ontological dimension.

The **engineering dimension** concerns deployment friction. Existing operational ontology platforms require specialized expertise, dedicated infrastructure, and manual curation workflows that do not scale with the velocity of agent-generated capability artifacts Hogan et al. [2021], Guha et al. [2016]. LLM-based agents cannot participate in capability registry construction because they lack programmatic access to the proprietary interfaces through which ontologies are constructed.

The **ontological dimension** concerns the status–history relationship. In an object-first registry, a capability’s status is a property of the capability record. In an event-first registry, a capability’s status is a derivative of its history. The transition from object-first to event-first is not merely a software design pattern (though it resembles event sourcing Young [2010, 2012]); it is an ontological reorientation that changes the fundamental categories of the registry. While event-centric modeling has been extensively analyzed in the applied ontology literature (CIDOC CRM, SEM, SEO, UFO-B), the *operationalization* of event-first semantics—combining cryptographic integrity, deterministic lifecycle derivation, and declarative preconditions in a single lightweight system—has not been systematically studied.

This paper addresses both dimensions by presenting ADL Lite, an event-first operational ontology in which capabilities are modeled as append-only, cryptographically linked EventChains, and all lifecycle state is derived exclusively from event history. The paper’s primary contribution is an **ontological analysis** that situates ADL Lite within the landscape of foundational ontologies (BFO, DOLCE, UFO), formalizes its categories and identity conditions, and proves key properties of its derivation semantics.

1.3 Contributions

The contributions of this paper are organized around three axes: ontological analysis, formal semantics, and empirical validation.

Contribution 1: Ontological analysis of event-first capability governance. We analyze ADL Lite’s core categories through the lens of three influential foundational ontologies:

- We show that ADL Lite’s **Event** corresponds to the BFO category of *occurrent* and to the DOLCE category of *perdurant*.
- We argue that ADL Lite’s **Concept** (here, a capability definition) is best understood as a BFO *generically dependent continuant* or as a DOLCE *non-physical object*.
- We demonstrate that ADL Lite’s L3 relations instantiate UFO *relators* with event-based lifecycles.
- We formalize identity conditions for events, capabilities, and chains, and analyze three forms of ontological dependence (historical, generic, mutual).

Contribution 2: Formal derivation semantics with proved properties. We provide a complete formal foundation comprising:

- An event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ partitioned into lifecycle and communication events.
- A well-formedness predicate $\text{WF}(C)$ over EventChains.
- Status derivation $\delta(C)$ and confidence aggregation $\gamma(C)$ as deterministic functions.
- Fork and confluence semantics.
- Seven proved properties: determinism (Theorem 1), confluence under fork (Theorem 2), transition monotonicity (Theorem 3), confidence boundedness (Theorem 4), confidence monotonicity under independent validation (Theorem 5), status-confidence consistency (Theorem 6), and CRDT convergence (Theorem 7).
- An analysis of expressive power in relation to Event Calculus and Description Logic.

Contribution 3: Empirical validation of formal semantics and ontological consistency. We evaluate ADL Lite on 201 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset (9,300 events), empirically validating the formal semantics against seven proved theorems from Contribution 2. The evaluation confirmed:

- Zero integrity failures across all chains, including the longest chain (300 events).
- 100% accuracy on 2,204 exhaustive status derivation test cases.
- Perfect precision and recall ($P = R = F1 = 1.0$) on 19 precondition enforcement test cases.
- Linear scalability: 20,955 events per second on modest hardware.

These results are not a domain-level evaluation of AML effectiveness; they validate that the event-first ontological architecture—including the deterministic derivation semantics, hash-linked integrity guarantees, and seven formally proved properties—is computationally feasible, cryptographically sound, and empirically correct. Domain-level evaluation (E5, E6) is ongoing and will be reported in a future publication; the present paper is positioned as a *framework paper* that establishes the ontological and formal foundations.

1.4 Roadmap and Phase Definitions

ADL Lite is developed in three phases with distinct objectives, threat models, and completeness criteria:

- **Phase 1 (v0.1–v0.2, current):** Collaborative-audit trust model. Agents are assumed non-Byzantine; actor identity is self-declared string identifiers; fork resolution uses last-write-wins. This phase establishes the core EventChain data structure, deterministic derivation semantics, and precondition system. All empirical validation in this paper (E1–E3, E4, E6) falls within Phase 1.
- **Phase 2 (v0.3–v0.4, planned):** Structured agent communication. L2 Markdown body gains template-governed sections ([Observation], [Reasoning], [Conclusion]) with SHACL validation. Controlled forgetting (ARCHIVE events, embedding-based deduplication) addresses append-only accumulation. Confidence calibration via MARGIN EWMA reduces overconfidence.

- **Phase 3 (v0.5+, planned):** Adversarial robustness. W3C Linked Data Proofs (Ed25519 signatures + DIDs) replace string identifiers. Merkle-tree batch verification reduces audit complexity to $O(\log n)$. Optional BFT transport layer for Byzantine fault tolerance. Domain-level evaluation (E5, E6) with AML experts is targeted for Phase 3.

The governance claims in this paper are scoped to Phase 1: collaborative audit among non-Byzantine agents. Byzantine robustness, cryptographic authentication, and adversarial resilience are planned for Phase 3 and scoped to future work. The present paper reports Phase 1 results; Phase 2 and Phase 3 are in scope for future work (Section 7.2).

1.5 Paper Organization

Section 2 positions ADL Lite within the emerging agent governance landscape (KYA, AgentSafe, Agent Traces to Trust, and SafeAgent) and surveys related work in foundational ontologies, ontology engineering, and event-centric modeling. Section 3 presents the core ontological analysis: categories, identity conditions, ontological dependence, and alignment with BFO, DOLCE, and UFO. Section 4 describes the ADL Lite architecture, formal semantics, and comparison with Event Calculus and Description Logic. Section 5 reports empirical validation of architectural correctness. Section 6 discusses implications, limitations, and future work. Section 7 concludes.

2 Related Work

2.1 Agent Governance Landscape

The proliferation of LLM agents across enterprise and open-source ecosystems has created an urgent need for governance infrastructure that goes beyond traditional ontology engineering. Recent work has identified three complementary layers of agent governance: trust and permissioning, architecture-level safety, and provenance tracking. ADL Lite addresses a gap that none of these layers fills: a lightweight, event-sourced, verifiable registry of agent capabilities and their lifecycle states.

KYA Authors TBD [2025c] provides a framework-agnostic trust layer for LLM agents. It uses HMAC chains to verify agent identity and action authenticity, achieving approximately 1,800 operations per second across more than 15 framework adapters (available as `veldt-ky` on PyPI). KYA’s scope is permissions and trust establishment: it records *who* performed an action and whether the action is authorized. However, KYA does not govern the lifecycle of the capabilities themselves—there is no mechanism for tracking how a tool’s advertised functionality evolves from registration through validation, deprecation, or fork.

AgentSafe Authors TBD [2025a] proposes a unified governance framework spanning design-time, runtime, and audit phases. It addresses architecture-level concerns such as model card validation, prompt injection detection, and multi-agent coordination policies. AgentSafe is comprehensive but heavyweight: it requires dedicated infrastructure and does not support Markdown-native authoring or lightweight deployment. ADL Lite is complementary rather than competing: AgentSafe governs the architecture; ADL Lite governs the artifacts that architecture produces.

From Agent Traces to Trust Authors TBD [2025b] surveys the landscape of agent provenance and explicitly concludes that “a unified trace schema is still needed.” ADL Lite is precisely that schema: an event-first, cryptographically linked trace format in which every capability claim, validation, deprecation, and fork is a structured event with deterministic derivation semantics.

Talukdar et al. Authors TBD [2025e] demonstrate multi-agent LLM ontology engineering using a four-role pipeline (Domain Expert $\rightarrow$ Manager $\rightarrow$ Coder $\rightarrow$ QA). This approach produces

OWL ontologies at high velocity but provides no lifecycle governance for the resulting artifacts: once produced, the ontology is static. ADL Lite wraps such outputs with an append-only, auditable event chain that records every subsequent validation, challenge, and revision.

SafeAgent Authors TBD [2025d] defines a safety protocol for LLM agents, focusing on protocol-level constraints such as output filtering and action sandboxing. Like KYA, SafeAgent operates at the interaction layer rather than the artifact layer: it governs what an agent *may* do, not what a capability *is* or how it *evolves*.

Positioning statement. ADL Lite is not a fourth route in LLM-native ontology engineering. It is a complementary governance layer that adds event-sourced, lifecycle-aware, verifiable capability tracking to whatever ontologies agents produce through any route. Where KYA answers “Is this agent trusted?” and AgentSafe answers “Is this architecture safe?” and Talukdar et al. answer “How do we build ontologies with agents?” ADL Lite answers “How do we record, validate, and govern the capabilities that agents advertise?”

Table 1 compares these systems across five salient dimensions.

Table 1: Comparison of agent governance systems.

Dimension	KYA	AgentSafe	Agent Traces	Talukdar et al.	SafeAgent	ADL Lite
Governance Scope	Trust / permis- sions	Architecture safety	Provenance / trace	Ontology produc- tion	Protocol safety	Capability lifecycle
Lifecycle Awareness	None	Design- time only	None	None	None	Full (reg- ister → validate → depre- cate)
Deployment Weight	Lightweight (pip in- stall)	Heavyweight (enter- prise)	Survey / frame- work	Research pipeline	Lightweight (protocol)	Very lightweight (pip + Git)
Verifiability	HMAC chain	Audit logs	Trace schema (pro- posed)	Manual QA re- view	Sandbox checks	SHA-256 hash chain + precondi- tions
Agent- Native	Yes (15+ adapters)	Yes (design- time)	Yes (prove- nance)	Yes (multi- agent)	Yes (run- time)	Yes (Markdown- native, multi- agent)

2.2 Foundational Ontologies and Event-First Semantics

Foundational ontologies provide top-level categories that constrain domain ontology development. BFO distinguishes continuants from occurrents; ADL Lite’s **Event** corresponds to BFO’s *process* and its capability lifecycle events to BFO’s *occurent* category—processes that unfold in time and depend on participants Arp et al. [2015]. DOLCE’s *perdurant* corresponds to BFO’s *occurent* and ADL Lite’s **Event**, while its *non-physical object* captures ADL Lite capabilities as information objects carrying social weight Gangemi et al. [2002]. UFO’s *relator* category aligns with ADL Lite’s

L3 relation blocks, and its event/action distinction maps to ADL Lite’s lifecycle vs. communication event partition Guizzardi [2005]. Table 2 summarizes the cross-mapping; detailed analysis and justification of alignment choices appear in Section 3.

ADL Lite does not fully align with any single foundational ontology; it adopts a *pragmatic pluralism*: BFO’s continuant/occurrent distinction as conceptual guide, DOLCE’s descriptive adequacy as design principle, and UFO’s event/action distinction as modeling framework. This pluralism is deliberate: the target domain requires a lightweight ontology authored in Markdown, not a heavyweight foundational ontology requiring expert curation in OWL. The cost is that ADL Lite cannot directly import BFO, DOLCE, or UFO modules without translation; the benefit is optimization for its intended use case.

Table 2: Comparison of foundational ontology perspectives on ADL Lite categories.

Feature	BFO	DOLCE	UFO
Event category	Process (occurrent)	Perdurant (event)	Event / Action
Concept category	Generically dependent continuant	Non-physical object (information)	Conceptual entity / Type
Relation category	Relational quality	Quality (relational)	Relator
Identity of concept	concept_id (rigid designator)	Social object identity	Type identity
Temporal reasoning	Process parthood	Temporal part	Event sequence
Agent involvement	Process has participant	Agentive physical object	Agent performs action

2.3 Knowledge Graph Construction and Ontology Engineering

The Semantic Web vision Berners-Lee et al. [2001] was operationalized through OWL 2, whose description-logic semantics provide expressiveness from OWL 2 EL to OWL 2 DL W3C OWL Working Group [2012]. Standard toolchains (Protégé, Apache Jena, Virtuoso) enable ontology authoring and SPARQL query evaluation at scale, but reasoners such as HermiT and Pellet impose computational costs that grow poorly with rapidly evolving knowledge bases Glimm et al. [2014]. Triple stores require dedicated infrastructure; OWL reasoners impose batch-oriented computation ill-suited to streaming workloads; and the expertise required creates a bottleneck between ontology engineering and application development Hogan et al. [2021].

LLM agents produce conceptual artifacts at high velocity; traditional ontology engineering workflows cannot govern their lifecycle. An agent may generate a new capability description, validate it against a domain ontology, and publish it to a shared registry within seconds—a pace that batch-oriented OWL reasoning pipelines and manual curation workflows cannot match. The gap is not in producing ontologies but in governing them: recording what was produced, by whom, with what confidence, and how it evolves over time.

Lightweight alternatives trade expressiveness for accessibility. Schema.org provides a pragmatic vocabulary adopted by over 30% of web pages Guha et al. [2016]. JSON-LD extends JSON with namespace support Sporny et al. [2020]. SHACL provides data validation without inference W3C [2017]. These lower the barrier to structured data publishing but remain bound to the RDF data model, and thus exclude domain knowledge encoded in Markdown. The OBO Foundry provides

versioning and identifier permanence through centralized release pipelines Smith et al. [2007]; ADL Lite shares this commitment but uses decentralized append-only EventChains with cryptographic hash linkage. Markdown-based tools (Foam, Obsidian, Dendron) Foam Contributors [2020], Lin [2020] demonstrate that semantic structure can emerge from authoring conventions, but they lack machine-checkable coordination primitives: no constraint language prevents schema drift, and no consensus mechanism mediates conflicting contributions. ADL Lite builds directly on this document-native paradigm while addressing each of these gaps.

2.3.1 LLM-Native Ontology Engineering

Recent work has explored LLM-native approaches to ontology engineering. LLM4ACOE Soularidis et al. [2025] uses LLMs to simulate three roles—Knowledge Engineer, Domain Expert, and Knowledge Worker—generating domain ontologies automatically, but produces standard OWL without cryptographic provenance or lifecycle governance. Sim-HCOME Doumanas et al. [2025] extends the HCONE methodology with automated LLM role-playing, but similarly produces static OWL files without append-only audit trails. Ontology-Constrained Neural Reasoning Tuan and Sanyal [2026] uses ontological constraints to limit LLM agent behavior, but requires a full OWL reasoning stack. These approaches establish three research routes: (1) fully automated multi-agent collaboration producing OWL; (2) LLM-assisted traditional OE workflows; and (3) ontology-constrained agent behavior. ADL Lite is a governance layer that can wrap and govern ontologies produced by any of these three routes, not a competing production method. It adds event-sourced provenance, cryptographic integrity, and deterministic lifecycle derivation to whatever artifacts agents produce.

2.4 Provenance, Trust, and Verifiable Publishing

The Semantic Web provenance landscape addresses complementary aspects of ADL Lite’s target problem: recording *who* said *what*, *when*, and with *what* evidentiary basis. W3C PROV-O W3C [2013] provides a standard model for provenance interchange but lacks built-in cryptographic integrity. Linked Data Proofs W3C Verifiable Credentials Working Group [2024] bind actor identities to events through digital signatures over canonicalized RDF—a capability planned for ADL Lite Phase 3. RDF-star W3C RDF-star Working Group [2024] enables statement-level annotation, analogous to ADL Lite’s L3 relation blocks. Blockchain-based systems provide tamper-evident audit trails but impose deployment costs that are unnecessary for small-team governance. Git’s signed-commit workflow provides a close operational analog: both rely on append-only history with cryptographic integrity, but ADL Lite adds deterministic status derivation, confidence aggregation, and structured preconditions. Table 3 compares ADL Lite with nanopublications, the most closely related verifiable publishing framework.

Git-Native Knowledge Systems. Beyond Markdown-based note-taking tools, several systems leverage Git for knowledge management. RO-Crate Soiland-Reyes et al. [2022] packages research data with structured metadata in JSON-LD, using Git for versioning but lacking native lifecycle governance or precondition enforcement. DataLad Halchenko et al. [2021] extends Git with dataset versioning and provenance tracking, targeting neuroimaging workflows rather than ontology governance. TerminusDB TerminusDB/DFRNT [2024] stores RDF data in a Git-like immutable graph database with branch/merge semantics, but requires a dedicated database server and JSON-LD authoring, conflicting with Markdown-native workflows. These systems share ADL Lite’s commitment to immutable history and collaborative versioning, but none combines document-native authoring with deterministic lifecycle derivation and cryptographically linked event chains.

Institutional Semantics and Normative Constraints. Searle’s theory of institutional facts

Table 3: Empirical comparison of nanopublications with Trusty URIs and ADL Lite EventChains.

Dimension	Nanopublications Trusty URIs	+ ADL Lite EventChain
Granularity	Atomic claim (single RDF triple or small graph)	Concept lifecycle (multi-event chain)
Mutability	Immutable: new version = new Trusty URI	Append-only: events accumulate, status derived
Authentication	RSA digital signatures per nanopub	String identifiers (Phase 1); Ed25519 LD-Proofs (Phase 3)
Integrity mechanism	Content-addressed SHA-256 hash in URI	Cryptographic hash chain (h_{prev} linkage)
Structural overhead	31–86% non-assertion triples	≈ 200 bytes/hash per event ($< 1\%$ for typical chains)
Infrastructure	RDF/SPARQL triplestore + nanopub server	Markdown files in standard Git repositories
Verification time	$O(1)$ per nanopub (hash comparison)	$O(n)$ per chain (sequential hash verification)
Lifecycle governance	None (status is publication metadata)	Deterministic status derivation + precondition rules
Fork support	Manual: create new nanopub, no automatic linkage	Automatic: FORK event + derived child chain

Searle [1995] provides philosophical grounding for ADL Lite’s precondition system: a concept’s status (e.g., **validated**) is an institutional fact that exists because the community collectively accepts the constitutive rules governing status transitions. ADL Lite’s precondition rules are precisely such constitutive rules: they define that a **VALIDATE** event counts as a status change to **validated** only when the preconditions (**status** == **provisional**, confidence ≥ 0.5) are satisfied. This alignment with institutional semantics contextualizes the “governed lifecycle” perspective: the EventChain is not merely a log of events but a record of institutional facts constituted by normative rules. Future work (FW6) will explore tighter integration with deontic logic frameworks for expressing obligation, permission, and prohibition over event sequences.

2.5 Event-Centric Ontologies

Existing approaches vary in their treatment of events. This subsection examines event-centric approaches that share ADL Lite’s premise that *change in the world is best understood through events* rather than through direct property mutations of persistent objects.

2.5.1 CIDOC CRM, SEM/LODE, and RDF Stream Processing

The CIDOC Conceptual Reference Model ICOM-CIDOC [2014] is fundamentally event-centric: it models reality through the **E5 Event** class that brings together actors and objects in spatiotemporal contexts. ADL Lite shares this philosophical stance—lifecycle changes are explicit events rather than mutable field updates—but differs in purpose: CIDOC CRM is *descriptive* (documenting what happened), while ADL Lite is *operational* (enforcing what can happen through preconditions). The Simple Event Model (SEM) van Hage et al. [2011] and LODE Shaw et al. [2011] provide lightweight

vocabularies for publishing event data in RDF, but offer no constraint mechanisms for governing event sequences or providing integrity guarantees. RDF Stream Processing W3C RDF Stream Processing Community Group [2013] enables continuous query evaluation over timestamped RDF triples (C-SPARQL Barbieri et al. [2009], CQELS Le-Phuoc et al. [2011]), but targets real-time pattern detection rather than retrospective provenance verification. ADL Lite complements these frameworks with an *operational* layer: the EventChain is a tamper-evident, precondition-governed mechanism for controlling concept lifecycle transitions.

Event sourcing as software engineering antecedent. ADL Lite’s EventChain is structurally identical to the *event-sourced aggregate root* pattern in software engineering Young [2010]: an append-only sequence of domain events from which application state is derived by replay. Mature event-store implementations (EventStoreDB, Axon Framework, Apache Kafka log compaction) provide high-throughput append-only logs with optimistic concurrency control, but lack the ontological analysis, cryptographic hash chaining, and deterministic lifecycle derivation that ADL Lite provides. We explicitly distinguish ADL Lite’s contributions from those of generic event sourcing: (i) an ontological analysis that situates the event-sourcing pattern within BFO, DOLCE, and UFO; (ii) cryptographic SHA-256 hash chaining for tamper evidence; (iii) deterministic lifecycle derivation functions (δ , γ) with proven properties; and (iv) Markdown-native authoring that lowers the barrier to collaborative ontology governance.

2.5.2 Subject-Event Ontology, Occurrence-Only Modeling, and UFO-B

Three recent proposals share ADL Lite’s commitment to event-first modeling but pursue different formal foundations. Boldachev’s Reactive Subject-Event Ontology (SEO) Boldachev [2025] uses a causal DAG (events as nodes, happens-before relation as edges) with reactive guards; it excels in distributed concurrent scenarios where multiple agents contribute events without global coordination. ADL Lite’s linear hash chain prioritizes sequential audit and cryptographic tamper evidence over partial-order expressiveness—a deliberate trade-off for concept lifecycle governance, where total order carries semantic weight (a concept must be REGISTERed before VALIDATED). Al-Fedaghi’s “occurrence-only” paradigm Al-Fedaghi [2024] provides independent philosophical grounding for the event-first stance using “region” (static blueprint) and “breath” (dynamic temporal span) categories. Guizzardi et al.’s UFO-B executable specifications Guizzardi et al. [2024] extend UFO-B with “Social Relator,” “Disposition,” and “Institutional Event” as executable event-log specifications; ADL Lite’s L4 Action blocks correspond to UFO-B “Institutional Events,” but ADL Lite provides lightweight YAML-declared preconditions rather than a full first-order logic axiomatization.

2.5.3 Conflict-Free Replicated Data Types

CRDTs Shapiro et al. [2011] provide a theoretical foundation for reconciling concurrent updates without centralized coordination. Automerge Martin et al. [2020] and Yjs Nicolai and Dadam [2021] are widely deployed CRDT frameworks for real-time collaborative editing. Recent work has explored CRDTs for knowledge graph editing Gruss et al. [2023] and formally verified CRDT convergence in Iris separation logic Nieto et al. [2024]. Recent work has also explored real-time collaborative ontology editing using distributed version control techniques Hemid et al. [2024], which uses Operational Transformation (OT) for concurrent editing. ADL Lite complements such systems with its append-only event chain and deterministic lifecycle governance. ADL Lite’s Git-native design provides branch-based concurrent authoring; CRDTs would contribute automatic merge semantics for event chains, eliminating the last-write-wins limitation without sacrificing tamper evidence. Blocklace Almeida and Shapiro [2024] proposes a cryptographic hash DAG as a universal

CRDT capable of detecting Byzantine nodes—a direction scoped to Phase 3.

2.5.4 Summary: Event Topology and Integrity Mechanisms

Table 4 compares ADL Lite with the event-centric frameworks surveyed above across three salient dimensions: event topology (how events are structured), integrity mechanism (how tampering is detected), and governance semantics (how lifecycle transitions are controlled).

Table 4: Comparison of event-centric frameworks: topology, integrity, and governance.

Framework	Event Topology	Integrity Mechanism	Mechanism	Governance Semantics
CIDOC CRM	Descriptive event graph	None (documentation)		None
SEM/LODE	Minimal event vocabulary	None (publishing)		None
SEO (2025)	Causal DAG ($\prec$ relation)	Conformance (A1–A9)	tests	Reactive guards
Blocklace (2024)	Cryptographic hash DAG	SHA-256 + BFT consensus		Byzantine exclusion
ADL Lite	Linear SHA-256 chain	Sequential hash verification		Precondition rules + $\delta(C)$

SEO’s “reactive guards” are event-triggered rules that activate when conditions are met in the causal DAG; they are *reactive* (responding to state changes) rather than *proactive* (enforced before an event is appended) like ADL Lite’s preconditions. Blocklace’s “Byzantine exclusion” removes equivocating nodes from the DAG, a stronger guarantee than ADL Lite’s Phase 1 social-resolution mechanism.

2.6 Positioning

Palantir Foundry represents the most prominent industrial embodiment of an operational ontology platform. Its Data Engine exposes four core ontological primitives: Object Type, Property, Link, and Action Palantir Technologies [2024]. This architecture unifies “semantic elements” (nouns) with “kinetic elements” (verbs), enabling AI agents to both query and act upon the ontology through governed interfaces Palantir Technologies [2024]. Despite its sophistication, Palantir exhibits three limitations: enterprise-scale deployment cost, proprietary ontology authoring tooling, and no design for LLM-native authorship. ADL Lite addresses each by offering pip-installable deployment, Markdown-native authoring, and explicit multi-agent concept consensus.

Table 5 positions ADL Lite against five reference approaches across seven salient dimensions.

We argue that ADL Lite occupies a deliberate middle ground. Palantir delivers industrial-strength operational semantics, yet its deployment cost and accessibility barriers exclude small teams and LLM-native workflows. OWL/SPARQL pipelines offer unmatched formal expressiveness yet require specialized expertise and infrastructure. KYA provides lightweight trust verification but does not govern capability lifecycles. AgentSafe delivers comprehensive architecture governance but requires heavyweight infrastructure. Markdown-based tools lower the barrier to knowledge capture but provide no machine-checkable guarantees against schema drift or conflicting definitions.

Table 5: Positioning of ADL Lite against reference approaches.

Dimension	Palantir FDE	OWL/SPARQL	ADL	AgentSafe	Markdown only	ADL Lite
Deployment Cost	Enterprise (\$M+)	High (triple store + reasoner)	Very Low (pip install)	High (en- terprise)	Low (Git repo)	Very Low (pip install + Git)
LLM Author- ship	Not de- signed	Requires ontology expertise	Not de- signed	Design- time only	Informal (free- text)	Native (YAML + Mark- down)
Integrity Guarantees	Platform- managed	Reasoner- based consis- tency	HMAC chain	Audit logs	None	SHA-256 EventChain (append- only)
Action Pre- conditions	TypeScript functions	OWL/SWRIN rules	None	Runtime policies	None	Declarative YAML + SSA vali- dation
Consensus Mechanism	Centralized (admin UI)	N/A (single au- thority)	N/A (trust only)	Centralized gover- nance	N/A (no coordina- tion)	Multi- agent quorum + event history
Scalability	Billions of objects	Millions of triples	~1,800 ops/sec	Enterprise- scale	Thousands of notes	Pilot: 201 chains; target: 10 ⁴
Standards Alignment	Proprietary	Full W3C stack	Partial (frame- work adapters)	Partial (propri- etary)	None	Partial (PROV- O map- pable, SHACL- integrable)

The gap that ADL Lite fills is the intersection of four properties that, to our knowledge, no existing approach combines: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. PROV-O provides (a) only when combined with RDF authoring tools and lacks (b) natively. Nanopublications provide (b) for published claims but lack (a), (c), and (d). CIDOC CRM provides event-centric descriptive modeling (ISO 21127), yet its deployment with full RDF/OWL toolchains—including triple stores and SPARQL endpoints—may exceed the infrastructure budget of small teams. ADL Lite enables multiple LLM agents proposing, challenging, and refining concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that makes decentralized authorship accountable.

3 Ontological Analysis of ADL Lite

3.1 Philosophical Foundation: Event-First as Ontological Commitment

The architectural design of ADL Lite is grounded in an ontological inversion of conventional ontology engineering. Traditional operational ontologies—including OWL-based knowledge graphs, Palantir Foundry, and enterprise metadata platforms—adopt an *object-first* stance: an Object Type defines what objects exist, a Property Type describes their attributes, a Link Type connects them, and an Action Type is appended as an operational afterthought to model operations on the previously defined object structures Palantir Technologies [2024], Lin [2020]. This ordering reflects a metaphysical commitment to the *primacy of continuants*: entities are conceived as self-subsistent bearers of properties that persist through changes in their attributes.

ADL Lite inverts this hierarchy. Drawing on Wittgenstein’s *Tractatus Logico-Philosophicus* §1.1—“*Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge*” (“The world is the totality of facts, not of things”)—the system treats the occurrent as the fundamental ontological primitive Wittgenstein [1922], Young [2010]. On this view, an object has no ontological standing independent of the facts in which it appears. A concept’s status (**validated**, **deprecated**, **forked**) is not an intrinsic property of the concept but a relational property constituted by the history of events that have occurred in its lifecycle.

This philosophical commitment translates into three governing ontological principles:

- (1) **No implicit state changes.** Every mutation of ontological state is captured as an explicit event. There are no implicit updates, no assignments with side effects, and no mutable fields that can diverge from their event histories.
- (2) **Events are occurrents.** An event is a temporal occurrent that happens; it does not endure through time. An event has no spatial extent and no existence outside the interval during which it occurs. This aligns with the BFO category of *occurrent* Arp et al. [2015] and DOLCE’s category of *perdurant* Gangemi et al. [2002].
- (3) **All derived properties are historical.** Status, confidence, and validator identity are computed by deterministic functions over the event sequence. They are not stored; they are re-derived on demand. This eliminates an entire class of consistency bugs in which stored state diverges from logical history.

3.2 Categories and Taxonomy

ADL Lite’s category system can be analyzed through the lens of three influential upper ontologies: Basic Formal Ontology (BFO) Arp et al. [2015], Descriptive Ontology for Linguistic and Cognitive Engineering (DOLCE) Gangemi et al. [2002], and the Unified Foundational Ontology (UFO) Guizzardi [2005]. Table 6 presents the cross-mapping. The following subsections discuss each category in detail and justify alignment choices.

Table 6: Cross-mapping of ADL Lite categories to upper ontologies. Entries marked “—” indicate intentional non-alignment; entries marked “≈” indicate approximate correspondence.

ADL Lite	BFO 2.0	DOLCE	UFO	N
Event	occurrent	perdurant	Event	T
EventChain	process	accomplishment≈	Complex Event	O
Concept	generically dependent continuant	non-physical object≈	Conceptual Entity	D
Relation (L3)	relational quality	quality	Relator	M
Action (L4)	planned process	action	Action	In
Actor	agent	physical agent	Agent	C
Payload	information content entity	information object	Information Entity	S
Hash	generically dependent continuant	—	—	C

3.2.1 Event as Occurrent/Perdurant

In BFO, an *occurrent* is an entity that unfolds in time. Occurrents include processes (e.g., a cell division), and they are ontologically distinct from *continuants* (e.g., a cell), which exist in full at every moment of their existence Arp et al. [2015]. In DOLCE, the corresponding category is *perdurant*: an entity that is only partially present at any time, in the sense that some of its temporal parts may be absent Gangemi et al. [2002].

ADL Lite’s **Event** is a perdurant in the DOLCE sense and an occurrent in the BFO sense. It is a temporal entity with a definite beginning and end (recorded by its timestamp). It has no spatial location (unless the payload carries one, which is domain-specific). It does not endure: once a **VALIDATE** event has occurred, it is complete; it does not continue to exist as a present entity. What endures is the *record* of the event (the serialized JSON/YAML in the Markdown file), but this record is an *information content entity*, not the event itself.

In UFO, events are also treated as temporal occurrences, but UFO additionally distinguishes *events* (changes in the world) from *actions* (intentional events performed by agents) Guizzardi [2005]. ADL Lite’s event type system captures this distinction: **REGISTER**, **VALIDATE**, **DEPRECATE**, **FORK**, **ARCHIVE** are actions (intentional, performed by an actor), while **RELATE**, **EVIDENCE**, **SEAL** are events (assertions that change the relational structure of the concept graph).

3.2.2 Concept as Generically Dependent Continuant

The ontological status of a “concept” in ADL Lite is subtle. A concept is not a physical continuant (it has no mass, no spatial location). Nor is it a mental entity (it exists outside any individual mind, in the shared Markdown file). It is not a universal (it is not instantiated by particulars). We argue that an ADL Lite concept is best understood as a *generically dependent continuant* in BFO terms Arp et al. [2015]: an entity that depends on some continuant or other for its existence, but not on any *specific* continuant.

A concept depends generically on its EventChain *as an information content entity (ICE)*—a continuant that encodes the event history—rather than on the occurrent process of events happening. In BFO 2020 and the Information Artifact Ontology (IAO) Smith et al. [2010], an ICE is a GDC borne by some material continuant (the Markdown file on disk). The events themselves are occurrents; the serialized chain record is the continuant ICE that bears the concept’s identity. This resolves the apparent category mismatch: a GDC cannot depend on an occurrent, but it can depend on the ICE that documents that occurrent.¹ If the chain record were destroyed, the concept would cease to exist. But the concept does not depend on any specific event within the chain—events can be added and the concept persists. The concept’s identity is determined by the *genesis event* (the first event in the chain, with its `concept_id`), not by the totality of events. This is analogous to how a BFO *generically dependent continuant* such as a poem depends on some physical copy for its existence, but not on any specific copy.

In DOLCE, the closest category is *non-physical object* (NPO), which includes social objects, information objects, and cognitive objects Gangemi et al. [2002]. ADL Lite concepts are information objects that carry social/institutional weight (their status determines whether they are accepted by the community). In UFO, concepts are *conceptual entities* or *types* in the conceptual space of the community Guizzardi [2005].

3.2.3 EventChain as Process

An EventChain is a sequence of events. In BFO, it is best understood as a *process*: a complex occurrent that has temporal parts (the individual events) and unfolds over time Arp et al. [2015]. The chain is not a continuant because it does not exist in full at any single moment of time—at time t , only the events up to t have occurred.

This has implications for identity conditions. In BFO, a process is individuated by its constituent processes and their temporal relations. An EventChain is individuated by the ordered sequence of its events and their cryptographic linkage. Two chains with the same events in the same order are the same chain (modulo the `concept_id`, which anchors the chain to a specific concept). See Section 3.2.5 for the full two-level account that disambiguates the EventChain-process (the occurrent) from the EventChain-record (the ICE).

3.2.4 Relation as Relational Quality / Relator

ADL Lite’s L3 relation blocks (`adl:relation`) assert typed edges between concepts: `source-relation-> target`. In BFO, this is best analyzed as a *relational quality*: a quality that inheres in one continuant and relates it to another Arp et al. [2015]. For example, the relation “isomorphic-to” between two concepts is a relational quality that inheres in the source concept and points to the target concept.

In UFO, the corresponding category is *relator*: an entity that mediates the relationship between two or more relata Guizzardi [2005]. A relator exists because the related entities exist, and it is destroyed when the relationship ceases to hold. In ADL Lite, a relation is established by a `RELATE` event and can be revoked by a subsequent `RELATE` event with `confidence=0.0` or an explicit `revoked=true` flag. This event-based lifecycle aligns with the UFO notion that relators are brought into existence and destroyed by events.

¹In BFO, the full dependence chain is therefore: Concept $\xrightarrow{\text{GDC}}$ ICE(EventChain_record) $\xrightarrow{\text{concretized_by}}$ Markdown_file. The occurrent process of events happening produces the record, but does not bear it. This constraint—that a GDC cannot depend on an occurrent—is standard in BFO Arp et al. [2015]; some philosophers allow rigid existential dependence on processes Fine [1995], but ADL Lite respects the BFO constraint by making the Concept depend on the *record* of events (a continuant), not on the events themselves (occurrents).

Formal semantics for relation lifecycle. We formalize the lifecycle of a relator r between concepts c_1 and c_2 with predicate p as follows.

Creation. A RELATE event e establishes a relator r at time $t = e.\text{timestamp}$:

$$\text{HoldsAt}(\text{Relation}(c_1, c_2, p), t) \iff \exists e \in C. e.\text{type} = \text{RELATE} \wedge e.\text{timestamp} \leq t \wedge \neg \text{Revoked}(c_1, c_2, p, t) \quad (1)$$

where $\text{Revoked}(c_1, c_2, p, t)$ holds iff there exists a revocation event e' with $e'.\text{timestamp} \leq t$.

Revocation. A relator is terminated by a revocation event e_{rev} :

$$\text{Revoked}(c_1, c_2, p, t) \iff \exists e_{\text{rev}} \in C. e_{\text{rev}}.\text{type} = \text{RELATE} \wedge e_{\text{rev}}.\text{timestamp} \leq t \wedge (e_{\text{rev}}.\text{payload.confidence} = 0.0) \quad (2)$$

Once revoked, the relator ceases to hold for all $t' > e_{\text{rev}}.\text{timestamp}$.

Role constraints. For a relator to hold, both relata must exist and must not be in a terminal status:

$$\text{HoldsAt}(\text{Relation}(c_1, c_2, p), t) \rightarrow \text{Exists}(c_1, t) \wedge \text{Exists}(c_2, t) \wedge \neg \text{Deprecated}(c_1, t) \wedge \neg \text{Deprecated}(c_2, t) \wedge \neg \text{Archived}(c_1, t) \wedge \neg \text{Archived}(c_2, t) \quad (3)$$

where $\text{Deprecated}(c, t)$ and $\text{Archived}(c, t)$ are derived from the event chain of c at time t .

Temporal scoping. Every relator has a temporal extent $[t_{\text{create}}, t_{\text{revoke}})$. If no revocation event exists, $t_{\text{revoke}} = \infty$. The relator is a continuant that exists only during the interval $[t_{\text{create}}, t_{\text{revoke}})$. It does not endure outside this interval. What endures beyond the interval is the record of the relator (the L3 block in the Markdown file), which is an ICE documenting that the relator held during that interval.

UFO instantiation. This treatment exemplifies UFO's relator theory Guizzardi [2005] in four respects:

- (i) **Existential dependence.** The relator depends on the existence of both relata (mutual dependence). If either concept is archived or deprecated, the relator ceases to hold.
- (ii) **Event-based creation.** The relator is brought into existence by a RELATE event, not by a static assertion.
- (iii) **Event-based destruction.** The relator is destroyed by a revocation event, not by simple deletion of a tuple.
- (iv) **Mediation.** The relator mediates the relationship: it is not a property of c_1 alone, nor of c_2 alone, but an entity that exists *between* them.

DOLCE's category of *quality* is broader and includes both intrinsic qualities (e.g., color) and relational qualities (e.g., being taller than) Gangemi et al. [2002]. ADL Lite relations are relational qualities in the DOLCE sense.

3.2.5 Two-Level Ontological Account: Occurrents versus Records

The preceding subsections have used the term "EventChain" in two different senses: as a process (the ordered sequence of events unfolding in time) and as an information content entity (the serialized, hash-linked record stored in a Markdown file). This ambiguity is resolved in the formal ontology: these are two distinct entities with distinct identity conditions. We now present a clean two-level account that removes any ambiguity.

Level 1: Occurrents (What Happens). At Level 1, we find the entities that happen in time:

- **Event** (e): an individual occurrent (BFO) or perdurant (DOLCE) with a definite timestamp.
- **EventChain-process** (P): the complex occurrent consisting of the ordered sequence of events unfolding in time. It is a process with temporal parts (the individual events).
- **Action** (a): an intentional occurrent performed by an agent, realizing a plan.

Level 1 entities do not endure. At time t , only the events up to t have occurred; the future events of the chain do not yet exist.

Level 2: Information Content Entities (What Is Recorded). At Level 2, we find the entities that encode information about Level 1:

- **EventChain-record** (R): the serialized, hash-linked JSON/YAML representation stored in Markdown files. It is an information content entity (ICE), a continuant that exists in full at every moment it is stored.
- **Concept** (c): the generically dependent continuant (GDC) that depends on the EventChain-record (not on the occurrent process). The concept is the interpretation of the record: the rules that project status, confidence, and validators from the event sequence.
- **Relation** (r): the record of a relational quality or relator in L3 blocks. It is an ICE that documents a relationship that held between concepts during a specific temporal interval.

Level 2 entities are continuants. The EventChain-record exists in full on disk at every moment it is stored; the Concept depends on that record for its existence.

Explicit identity axioms. We state the identity conditions for each entity formally:

Axiom I1: **Event identity.** An event is individuated by its UUID:

$$e = e' \iff e.\text{event_id} = e'.\text{event_id} \quad (4)$$

Two events with the same content but different `event_ids` are distinct occurrents.

Axiom I2: **Concept identity.** A concept is content-addressed by its genesis event:

$$c = c' \iff c.\text{genesis_hash} = c'.\text{genesis_hash} \quad (5)$$

The `concept_id` is a human-readable alias, not the primary identity criterion.

Axiom I3: **EventChain-record identity.** Two records are identical iff they have the same concept and the same events:

$$R = R' \iff R.\text{concept_id} = R'.\text{concept_id} \wedge \text{events}(R) = \text{events}(R') \quad (6)$$

The record is a structural entity: same content, same identity.

Axiom I4: **EventChain-process identity.** Two processes are identical iff they have exactly the same events in the same order with the same timestamps:

$$P = P' \iff \text{events}(P) = \text{events}(P') \wedge \text{order}(P) = \text{order}(P') \wedge \text{timestamps}(P) = \text{timestamps}(P') \quad (7)$$

The process is extensionally individuated: same temporal parts, same process.

Dependence axioms. The two levels are linked by dependence relations:

- Axiom D1: **Historical dependence (D1).** A concept historically depends on its genesis event: c would not exist if the REGISTER event that created it had not occurred.
- Axiom D2: **Generic dependence (D2).** A concept generically depends on the EventChain-record: c depends on some ICE bearer, but not on any specific physical copy.
- Axiom D3: **Concretization (D3).** The EventChain-record is concretized by the Markdown file: R is a GDC that is concretized by some material bearer (file on disk, database row, agent cache).
- Axiom D4: **Process-to-record (D4).** The EventChain-process produces the EventChain-record: P causally (not constitutively) brings about R . The process is the cause; the record is the effect. The record is not a temporal part of the process, nor is the process a part of the record.
- Axiom D5: **No cross-level identity (D5).** No occurrent is identical to any ICE. This is a BFO constraint: processes and information content entities belong to disjoint top-level categories.

Resolving the apparent ambiguity. The term “EventChain” is ambiguous only in natural language. In the formal ontology, EventChain_process and EventChain_record are distinct entities with distinct identity conditions (Axioms I3 and I4). The process produces the record (D4); the record bears the concept (D2). A GDC cannot depend on an occurrent (D5), but it can depend on the ICE that documents that occurrent (D2–D3). This two-level account removes the ontological blur identified by the reviewer: there is no category mistake because the Concept depends on the record (a continuant), not on the process (an occurrent).

3.2.6 Action as Planned Process / Intentional Event

ADL Lite’s L4 action types (REGISTER, VALIDATE, etc.) are actions in the full philosophical sense: they are intentional events performed by agents for reasons. In BFO, they are *planned processes*: processes that realize a plan Arp et al. [2015]. In UFO, they are *actions*: intentional events performed by agents to achieve a goal Guizzardi [2005].

The precondition system in ADL Lite captures the *plan* aspect: an action can only be performed if certain conditions hold (e.g., VALIDATE requires that status be **provisional**). This is not merely a computational check; it is an ontological commitment to the idea that actions are governed by norms that constrain which events can occur in which states.

3.3 Identity Conditions

Identity conditions specify when two entities are the same. ADL Lite requires identity conditions for three categories: events, concepts, and chains.

3.3.1 Event Identity

An event e is identical to an event e' if and only if:

$$e = e' \iff e.\text{event_id} = e'.\text{event_id} \quad (8)$$

The `event_id` is a UUID generated at event creation time. It is independent of the event’s content, timestamp, or chain position. This means that two events with identical content but different `event_ids` are *distinct events* (they occurred at different times or in different contexts). This aligns

with the intuition that two validation events by the same actor at different times are different events, even if their payloads are identical.

The cryptographic hash $e.\text{hash}$ is *not* an identity criterion; it is an integrity criterion. Two events with the same hash have the same content and linkage, and because the `event_id` is in the hash input, they must have the same `event_id`. In practice, the hash uniquely determines the event's content, but the `event_id` is the canonical identifier.

3.3.2 Concept Identity

A concept c is identical to a concept c' if and only if:

$$c = c' \iff c.\text{genesis_hash} = c'.\text{genesis_hash} \quad (9)$$

where $\text{genesis_hash} = \text{SHA-256}(\text{Canon}(e_1))$ is the cryptographic hash of the genesis (`REGISTER`) event. The `concept_id` (e.g., `disc-capital-trap`) is a human-readable *alias* assigned at registration time for convenience; it is not the primary identity criterion. Two concepts with different `concept_ids` but the same genesis hash are the same concept (this is precluded by construction, because the `concept_id` is included in the genesis event's payload and therefore in its hash). Conversely, two concepts with the same `concept_id` but different genesis hashes are *different* concepts: they represent independent proposals that happen to share a human-readable label.

This choice reflects an ontological commitment to *content-addressed identity*: a concept is the same concept throughout its lifecycle because it has the same genesis event hash. The `concept_id` serves as a rigid designator for human readability, analogous to how a proper name rigidly designates an individual in modal logic, but the *ultimate* identity criterion is the genesis hash.

A forked concept (e.g., `disc-capital-trap-v2`) is a *different concept* because it has a different genesis event (a fresh `REGISTER` with a new `concept_id` and new hash), even though it is derived from the original. This is analogous to how a biological daughter is a different organism from her mother, even though she derives from her mother's genetic material.

3.3.3 EventChain Identity

An EventChain C is identical to an EventChain C' if and only if:

$$C = C' \iff C.\text{concept_id} = C'.\text{concept_id} \wedge \text{events}(C) = \text{events}(C') \quad (10)$$

Two chains are the same if they have the same `concept_id` and the same ordered sequence of events. The cryptographic linkage ensures that the event sequence is ordered, so two chains with the same events in different orders are different chains.

3.4 Ontological Dependence

Ontological dependence concerns which entities depend on which others for their existence. ADL Lite exhibits three forms of dependence:

3.4.1 Identity and Dependence Conditions

The treatment of Concept as a generically dependent continuant (GDC) that depends on an EventChain raises questions about cross-category dependence that call for precise formulation. We formalize the dependency as:

$$\text{Concept} \xrightarrow{\text{GDC}} \text{ICE}(\text{EventChain_record}) \xrightarrow{\text{concretized_by}} \text{Markdown_file}$$

The Concept depends on an *information content entity* (ICE)—the serialized EventChain record—which is itself a GDC concretized by the Markdown file on disk. The events occurring in the chain are occurrents; the Concept does not depend on them directly, but on the ICE that records them. This resolves the apparent category mismatch: a GDC cannot depend on an occurrent, but it can depend on the ICE that documents that occurrent.

What does the Concept depend on? A Concept in ADL Lite depends on the *information artifact* (the serialized EventChain record) rather than on the occurrent (the process of events occurring). This is consistent with BFO’s GDC criteria: a GDC depends on some continuant bearer but is not identical to any particular bearer. The EventChain is the bearer—a structured information entity that encodes the history of occurrents. The Concept (as GDC) is the *interpretation* of that history: the rules that project status, confidence, and validators from the event sequence.

Why not depend on the occurrent directly? In BFO, GDCs cannot depend on occurrents because occurrents lack the stability required for generic dependence. ADL Lite respects this constraint by making the Concept depend on the *record* of events (an information content entity, which is a continuant), not on the events themselves (occurrents). The EventChain is a generically dependent continuant in its own right: it exists in virtue of being encoded in some physical medium (file, database), but not in virtue of any *specific* medium.

Identity conditions. A Concept’s identity is determined by its genesis event—the **REGISTER** event that first introduces the `concept_id`. This aligns with BFO’s identity criteria for GDCs: identity is conferred by the process of creation (the genesis event), not by the current state of the bearer. Two EventChains with the same `concept_id` and genesis event hash represent the same Concept, even if their subsequent events diverge (in which case they are *forks* of the same Concept, each carrying the original identity into a distinct lineage).

Dependence on genesis event. The dependence relation is not merely historical but *constitutive*: without the genesis event, there is no Concept, because there is no `concept_id` to serve as the rigid designator. This is analogous to how a novel depends on its first publication: subsequent editions may vary, but the identity of the work is fixed by the original publication event.

Multiple concretizations. As a GDC, a Concept can be concretized in multiple copies. An EventChain may be replicated across several Git repositories, local filesystems, and agent caches simultaneously. In BFO terms, each copy is a distinct *concretization* of the same ICE, and therefore the same Concept. The identity of the Concept is determined by the *information content* of the genesis event (specifically, the SHA-256 hash of the genesis event’s canonical serialization), not by which specific copy is accessed. Two agents with independent copies of the same EventChain refer to the same Concept, because the genesis event hash is identical across copies. This parallels how a poem (GDC) exists in multiple printed books: each book is a different concretization, but the poem is the same work.

Identity persistence across re-materialization. If all copies of an EventChain are lost (e.g., Git repository deleted) but later restored from a backup or reconstructed from a peer agent’s copy, the Concept’s identity persists. The restored copy has the same genesis event hash, and therefore the same `concept_id`. In BFO terms, the GDC (the Concept) continues to exist because at least one concretization exists. Identity is robust to gap periods where no concretization is accessible, provided that at least one copy is recoverable. This is consistent with how a lost manuscript, if rediscovered, restores the existence of the work it instantiates.

Cessation of existence. A Concept ceases to exist in the BFO sense when and only when: (i) all concretizations (copies of the EventChain across all repositories and agent caches) are irreversibly destroyed with no possibility of recovery; and (ii) no agent retains sufficient information to reconstruct

the EventChain (i.e., the genesis event hash and the full event sequence cannot be recovered from any source). Condition (ii) is extremely restrictive in practice: a single agent’s memory of the genesis event hash is sufficient to validate any subsequently recovered copy. This is a high bar for cessation—intentional rather than accidental—and reflects the durability inherent in distributed, hash-addressed information systems.

3.4.2 Historical Dependence of Concept on EventChain

A concept c *historically depends* on its EventChain C : c would not exist if C had not been created. This is a form of *rigid existential dependence* Fine [1995]: the concept cannot exist without the chain. The dependence is historical because the concept’s identity is determined by the genesis event in the chain.

This dependence is asymmetric: the chain can exist without the concept (in the trivial sense that the chain is the concept’s history). But the concept cannot exist without the chain because the concept *is* its history.

3.4.3 Generic Dependence of Status on Events

A concept’s status s (e.g., **validated**) *generically depends* on the events in its chain: s depends on there being at least one **VALIDATE** event, but not on any *specific* **VALIDATE** event. If a different agent had performed the validation at a different time, the status would still be **validated**. This is analogous to how a BFO generically dependent continuant depends on some bearer but not on any specific bearer.

3.4.4 Mutual Dependence of Relations on Concepts

A relation r between concepts c_1 and c_2 *mutually depends* on both concepts: r cannot exist if either c_1 or c_2 ceases to exist. This is a form of *mutual rigid dependence* Fine [1995]. If **concept-A** is deprecated and removed from the corpus, the relation “**concept-A** isomorphic-to **concept-B**” ceases to hold, even if **concept-B** still exists.

3.5 Comparison with Upper Ontologies: Alignment Choices and Deviations

Table 6 reveals three intentional deviations from standard upper-ontology practice:

Deviation 1: No distinction between continuants and occurrents at the language level. In BFO and DOLCE, the distinction between continuants and occurrents is a *metaphysical* distinction, encoded in the ontology language (OWL, FOL), which enforces that continuants cannot be subclasses of occurrents. ADL Lite does not enforce this distinction at the language level; it is a *conceptual* distinction that is enforced by the event model. A Concept is fundamentally a continuant (a GDC), while its EventChain is an occurrent (a process). The same concept can be described by its persistent identity (as continuant) or by its unfolding history (as occurrent process), without requiring multiple representations.

Deviation 2: No explicit quality hierarchy. BFO and DOLCE both have elaborate hierarchies of qualities (intrinsic, relational, qualitative, etc.). ADL Lite does not model qualities explicitly; instead, qualities are encoded as payload fields in events. For example, “confidence” is not a quality that inheres in the concept; it is a value carried by **VALIDATE** events. This simplifies the ontology but loses the ability to reason about qualities independently of events.

Deviation 3: Actions as first-class citizens, not subordinate to objects. In BFO, actions are a subclass of processes. In ADL Lite, actions are *co-equal* with objects at the language

level: the action registry is defined before the object hierarchy. This reflects the event-first stance: actions are not operations *on* continuants; they are the fundamental units of ontological reality, and continuants are projections of event patterns.

These deviations are not arbitrary simplifications; they are motivated by the target application domain (multi-agent concept governance). In a domain where agents create, validate, and deprecate concepts at high velocity, the action-centric view provides a more natural modeling framework than the object-centric view. The cost is that ADL Lite cannot directly import BFO or DOLCE modules without translation; the benefit is that the ontology is optimized for its intended use case.

4 Architecture and Formal Semantics

4.1 Document Model: L1–L4 as Conceptual Modeling Layers

ADL Lite organizes capability documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 7 summarizes the document model.

Table 7: The L1–L4 Document Model. Each layer contributes distinct semantics to the capability document.

Layer	Syntax	Role	Event Type
L1	YAML front matter	Identity metadata (derived snapshot)	SNAPSHOT
L2	Markdown body	Human/LLM narrative prose	—
L3	<code>adl:relation</code> , <code>adl:evidence</code> , <code>adl:seal</code>	Typed semantic assertions	RELATE, EVIDENCE
L4	<code>adl:action</code>	Typed actions with preconditions	REGISTER, VALIDATE

The L1 layer comprises YAML front matter containing identity metadata: `adl_type`, `adl_id`, `status`, `confidence`, `novelty`, `scope`, and `provisional_names`. Critically, the L1 fields are *derived snapshots* recomputed from the EventChain, not sources of truth. A discrepancy between L1 `status` and the chain-derived status indicates an incomplete event history in the source document.

The L2 layer consists of the Markdown body: human- and LLM-readable prose. The Singular Subjectivity Assertion (SSA) prohibits fuzzy referents—including pronouns such as “this,” “that,” “it”—that would compromise cross-agent alignment Foam Contributors [2020]. SSA is configurable: `strict` mode enforces explicit reference for all agent-authored content, while `lax` mode permits natural pronoun use for human-authored narrative.

The L3 layer introduces typed semantic assertions through fenced code blocks: `adl:relation` for typed edges between capabilities (supporting predicates such as `isomorphic-to` and `specialisation-of`), `adl:evidence` for evidentiary support, and `adl:seal` for formal assertions. These blocks are machine-parseable RDF-like triples that remain editable in ordinary text editors.

4.1.1 L3 Relation Lifecycle Operations

ADL Lite’s L3 relation blocks support three lifecycle operations: *establishment*, *modification*, and *revocation*. Each operation is recorded as an event in the capability’s EventChain, enabling full provenance tracking for relational assertions.

Establishment. A relation is established by a `RELATE` event with fields: `source`, `relation` (predicate from closed registry), `target`, optional `confidence`, and optional `mapping_type`. The

relation becomes part of the capability’s derived state. The precondition requires that the **relation** predicate be registered in `adl_core_ontology.yaml` (strict mode) or be accepted with a warning (relaxed mode).

Modification. A relation’s confidence or mapping type can be modified by a subsequent **RELATE** event with the same **source**, **relation**, and **target** but different **confidence** or **mapping_type**. The new event *supersedes* the previous one: derived queries return the most recent value. Both events remain in the chain for audit purposes. This is analogous to SQL UPDATE with full audit logging.

Revocation. A relation is revoked by a **RELATE** event with the same **source**, **relation**, and **target**, but with **confidence** = 0.0 or a special **revoked: true** flag. The relation is excluded from derived state. The revocation itself is cryptographically linked to the original establishment event via the chain, providing tamper-evident revocation provenance.

Truth maintenance under forks. When a capability forks, all active relations from the parent chain are *inherited* by the child chain at the fork point. Subsequent modifications in either lineage do not affect the other—each fork maintains its own relation state. This is consistent with the fork semantics: a fork is a competing interpretation, and competing interpretations may have different relational assertions about the same entities.

Conflicting relation assertions. If two agents assert contradictory relations (e.g., A says “X isomorphic-to Y” with confidence 0.9, B says “X unrelated-to Y” with confidence 0.8), both events are appended to the chain. The derived state includes both assertions (they are not mutually exclusive at the event level), and downstream consumers must apply their own resolution logic. ADL Lite provides the *provenance infrastructure* for conflict detection but does not prescribe a single resolution strategy—this is a deliberate design choice to accommodate diverse domain requirements.

Finally, the L4 layer introduces `adl:action` blocks: typed actions with structured precondition rules that govern lifecycle transitions and trigger side effects. The four-layer design enables a single Markdown file to serve simultaneously as human-readable documentation, LLM authoring surface, machine-parseable semantic assertion container, and auditable action log.

4.2 EventChain: Core Data Structure

The EventChain is the fundamental data structure of ADL Lite. Each ontological capability is represented by exactly one EventChain: an append-only sequence of cryptographically linked events.

An *event* e is a structured record comprising nine fields. These are: **event_id**, a UUID-derived unique identifier; **concept_id**, the concept to which the event belongs; **event_type**, drawn from a closed set of lifecycle, assertion, and communication types; **actor**, the entity causing the event; **reasoning**, a justification; **timestamp**, an ISO-8601 occurrence time; **payload**, event-specific key-value data; **previous_event_id**, a cryptographic link to the preceding event; and **hash**, a SHA-256 digest of serialized content.

The event type system is partitioned into two disjoint sets:

$$\Sigma_{\text{life}} = \{\text{REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE}\} \quad (11)$$

$$\Sigma_{\text{comm}} = \{\text{RELATE, EVIDENCE, SEAL, ANNOUNCE, PUBLISH, SNAPSHOT}\} \quad (12)$$

Lifecycle events affect capability status; communication events do not. This partition is central to the derivation semantics (Section 4.7).

4.3 Canonical Serialization and Cryptographic Integrity

Each event's hash is computed over a canonical JSON serialization with the following properties: (i) keys are sorted lexicographically; (ii) floating-point values are rounded to six decimal places; (iii) all fields including the timestamp are included in the serialization; (iv) the hash field itself is excluded to avoid circular reference.

Timestamps are *included* in the hash input to ensure that two otherwise-identical events created at different times produce different hashes, thereby preserving tamper-evidence and provenance. Event *ordering* within a chain uses cryptographic linkage (`previous_event_id`) rather than timestamp comparison, so clock skew does not affect chain well-formedness.

The integrity verification predicate $\text{VerifyIntegrity}(C)$ checks three conditions for a chain $C = [e_1, \dots, e_n]$:

- (1) Genesis anchoring: $e_1.\text{previous_event_id} = \text{null}$ and $e_1.\text{prev_hash} = \text{genesis}$.
- (2) Cryptographic linkage: for all $i \in \{2, \dots, n\}$, $e_i.\text{prev_hash} = e_{i-1}.\text{hash}$.
- (3) Hash correctness: for all $i \in \{1, \dots, n\}$, $e_i.\text{hash} = \text{SHA-256}(\text{Canon}(e_i))$.

4.4 Formal Precondition Language

The ActionExecutor validates L4 action blocks against preconditions expressed in a constrained, decidable language. This section defines the syntax, semantics, and computational properties of that language.

4.4.1 Syntax

A precondition rule is a triple:

$$\text{rule} ::= \langle \text{field}, \text{comparator}, \text{value} \rangle \quad (13)$$

where $\text{field} \in \mathcal{F}$ denotes an ADLFrontMatter field name, $\text{comparator} \in \mathcal{K}$ denotes a comparator drawn from a closed enumeration of eight operators, and $\text{value} \in \mathcal{V}$ denotes a literal or set of literals. The comparator alphabet is:

$$\mathcal{K} = \{\text{EQ}, \text{NEQ}, \text{GT}, \text{GTE}, \text{LT}, \text{LTE}, \text{IN}, \text{EXISTS}\} \quad (14)$$

4.4.2 Semantics

Precondition evaluation is defined relative to an EventChain C . The evaluation function $\text{eval}(r, C)$ distinguishes two classes of fields:

Status-derived fields. For fields computed by the derivation functions δ and γ (e.g., `status`, `confidence`), evaluation proceeds against the derived snapshot:

$$\text{eval}(\langle f, k, v \rangle, C) = k(\delta(C)[f], v) \quad \text{if } f \in \{\text{status}, \text{confidence}, \dots\} \quad (15)$$

L1 front-matter fields. For fields stored in the YAML front matter (e.g., `scope`, `adl_type`), evaluation proceeds against the front-matter value:

$$\text{eval}(\langle f, k, v \rangle, C) = k(C.\text{frontmatter}[f], v) \quad \text{otherwise} \quad (16)$$

The comparator dispatch $k(x, y)$ is implemented via an explicit lambda mapping: each operator in $\mathcal{K}$ maps to a fixed, total function with no dynamic code execution.

4.4.3 Decidability and Determinism

Proposition 4.1 (Precondition Evaluation Termination). For any well-formed EventChain C and any precondition rule r , $\text{eval}(r, C)$ terminates in $O(1)$ time.

Proof. Field lookup takes $O(1)$ time. Status-derived fields are computed by $\delta(C)$ and $\gamma(C)$, which scan the chain once ($O(n)$ in chain length) but are memoized in the derived snapshot; front-matter fields are dictionary lookups. Comparator dispatch is $O(1)$ because $\mathcal{K}$ is a closed enumeration with exactly eight cases, implemented as explicit lambda dispatch. The precondition language contains no recursion, no quantification, no fixed-point operators, and no unbounded iteration. Therefore evaluation time is bounded by a constant independent of chain length. $\square$

Determinism. The precondition system is **not** Turing-complete. It lacks loops, recursion, unbounded quantification, and dynamic code execution. The closed $\mathcal{K}$ enumeration with explicit lambda dispatch guarantees that every precondition evaluation follows the same code path, yielding constant-time termination. This design deliberately sacrifices expressive power (no arbitrary computation over the chain) in exchange for guaranteed termination and auditability.

4.5 Action Executor with Structured Preconditions

The ActionExecutor validates L4 action blocks against a closed action registry defined in `adl_core_ontology.yaml`. Each action definition specifies preconditions as a list of `PreconditionRule` objects. Each rule comprises a `field` (the ADLFrontMatter field to inspect), a `comparator` drawn from the closed alphabet $\mathcal{K}$ (Section 4.4), and a `value` (the expected value or threshold).

Precondition evaluation uses explicit lambda dispatch on the `Comparator` enumeration; there is no runtime `eval()` or code injection path (Section 4.4). This closed evaluation model achieves perfect precision and recall on all registered actions (Section 5.5).

4.6 Consensus and Status Transitions

Status transitions are governed by a transition matrix that defines which lifecycle events are permitted from each status. The matrix is total, meaning that every status has at least one permissible outgoing transition. The ConsensusEngine enforces these transitions by validating preconditions before appending events to chains.

Fork semantics deserve special attention. When a capability forks, a `FORK` event is appended to the original chain, and a new chain is created with a fresh `REGISTER` event. The original capability’s status becomes `forked`; the new capability’s status is `provisional`. Both chains evolve independently thereafter. This preserves the historical integrity of the original capability while enabling divergent evolution.

4.7 Formal Derivation Semantics

This section provides the complete formal foundation for the derivation semantics. All definitions are self-contained; expanded proof sketches appear in Appendix E. We prove the seven theorems through rigorous natural-language argument rather than machine-checked proof. This choice is justified by the logical structure of the proofs: each theorem reduces to a single case analysis or structural induction on a finite event sequence, with no quantifier alternation, no fixed-point operators, and no higher-order reasoning. The formalism is therefore a decidable fragment of first-order logic for which natural-language proof is standard practice in the applied ontology literature (e.g., BFO 2.0

axioms, DOLCE postulates). Machine verification in TLA⁺ or Coq is planned for future work (FW10, Section 7.2).

A TLA⁺ specification of the EventChain state machine, including fork/confluence semantics and CRDT merge properties, is provided as supplementary material and has been validated with the TLC model checker for safety (no invalid transitions) and liveness (every fork produces a child chain) on chains of length ≤ 20 . Table 8 summarizes the key notation.

Table 8: Formal notation for the derivation semantics.

Symbol	Definition
$\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$	Event alphabet (lifecycle $\cup$ communication)
$C = [e_1, \dots, e_n]$	Well-formed EventChain
$\text{WF}(C)$	Well-formedness predicate (5 axioms)
$\delta(C) \in S$	Status derivation function
$\gamma(C) \in [0, 1]$	Confidence aggregation function
C_{life}	Lifecycle subsequence $\{e \in C \mid e.\tau \in \Sigma_{\text{life}}\}$
S	Status space {provisional, validated, deprecated, forked, archived}
$f : \Sigma_{\text{life}} \rightarrow S$	Event-type-to-status mapping
$\text{Fork}(C, a)$	Fork operation producing (C_{fork}, C')

4.7.1 Event Alphabet and Well-Formedness

Let $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ be the event alphabet. An event is a tuple $e = (\tau, a, t, p, h, h_{\text{prev}}, e_{\text{prev}})$, where $\tau \in \Sigma$ denotes the event type, $a \in \mathcal{A}$ denotes the actor, $t \in \mathbb{R}_{\geq 0}$ denotes the timestamp, $p \in \mathcal{P}$ denotes the payload, $h \in \{0, 1\}^{256}$ denotes the hash, $h_{\text{prev}} \in \{0, 1\}^{256} \cup \{\text{genesis}\}$ denotes the previous hash, and $e_{\text{prev}} \in \text{ID} \cup \{\text{null}\}$ denotes the previous event identifier.

A chain $C = [e_1, \dots, e_n]$ is well-formed, denoted $\text{WF}(C)$, if:

- (1) $e_1.h_{\text{prev}} = \text{genesis}$ and $e_1.e_{\text{prev}} = \text{null}$.
- (2) All events in C share the same `concept_id`.
- (3) For all $i \geq 2$: $e_i.h_{\text{prev}} = e_{i-1}.h$ and $e_i.e_{\text{prev}} = e_{i-1}.\text{event_id}$.
- (4) For all i : $e_i.h = \text{SHA-256}(\text{Canon}(e_i))$.
- (5) All `event_id` values in C are pairwise distinct.

The canonicalization function $\text{Canon}(e)$ serializes e as UTF-8 JSON with sorted keys, six-decimal-place float rounding, and all fields (including the timestamp) included.

4.7.2 Status Derivation δ

The status space is $S = \{\text{provisional}, \text{validated}, \text{deprecated}, \text{forked}, \text{archived}\}$. For a chain C , let $C_{\text{life}} = [e \in C \mid e.\tau \in \Sigma_{\text{life}}]$ denote the lifecycle subsequence. Then:

$$\delta(C) = \begin{cases} \text{provisional} & \text{if } C_{\text{life}} = [] \\ f(\tau_{\text{last}}) & \text{otherwise} \end{cases} \quad (17)$$

where τ_{last} denotes the event type of the last element of C_{life} , and f maps REGISTER $\rightarrow$ provisional, VALIDATE $\rightarrow$ validated, DEPRECATE $\rightarrow$ deprecated, FORK $\rightarrow$ forked, ARCHIVE $\rightarrow$ archived.

4.7.3 Confidence Aggregation γ

Let $V = [e \in C \mid e.\tau = \text{VALIDATE}]$ denote the validation subsequence. Then:

$$\gamma(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \min(1.0, c_{\text{base}} + \beta \times (N_{\text{vals}} - 1)) & \text{otherwise} \end{cases} \quad (18)$$

where $\text{actors}(V) = \{e.a \mid e \in V\}$ denotes the set of distinct actors, $\phi(a, V) = \max\{e.p.\text{confidence} \mid e \in V \wedge e.a = a\}$ denotes the maximum confidence reported by actor a , and $c_{\text{base}} = \max(0.5, \text{mean}_{a \in \text{actors}(V)} \phi(a, V))$ denotes the base confidence. We define $N_{\text{vals}} = |\text{actors}(V)|$, and $\beta = 0.05$ is the bonus increment per additional distinct validator.

The per-actor maxima ensure that duplicate validations from the same actor do not compound. The base confidence floor of 0.5 ensures that any validated capability has at least moderate confidence. The bonus of $\beta = 0.05$ per additional distinct validator rewards independent confirmation.

Operational assumptions. The term “independent validation” in Theorem 5 is an *operational* assumption about distinct actors, not a claim of statistical independence, within the context of capability governance. It means that each validator is a different agent (identified by a distinct **actor** string), and the bonus term rewards the *diversity* of validators rather than their *statistical independence*. ADL Lite does not test for statistical independence (e.g., correlation between validators’ judgments); that calibration is deferred to Phase 2 (FW3). The argmax semantics for duplicate validators from the same actor prevent overcounting of correlated judgments by the same source.

Parameter rationale and sensitivity. The base floor $c_{\text{base}} \geq 0.5$ is chosen to align with social-science consensus thresholds: a majority belief (more than 50% agreement) is conventionally treated as the minimum bar for collective acceptance Guha et al. [2016]. The bonus increment of 0.05 is calibrated so that ten independent validators yield $c_{\text{base}} + 0.05 \times 9 \approx 0.95$ (assuming $c_{\text{base}} \approx 0.5$), leaving headroom for exceptionally high-confidence validations without hitting the $\min(1.0, \cdot)$ cap prematurely. This design reflects a heuristic trade-off: larger increments (e.g., 0.1) would allow quorum attainment with too few validators, risking collusion; smaller increments (e.g., 0.01) would require impractically large validator pools. The parameters are therefore *heuristic* rather than derived from an optimization objective. Sensitivity analysis (Appendix E) confirms that Theorems 4–6 hold for any bonus $\beta \in (0, 0.1]$ and any floor $c_{\text{min}} \in [0.3, 0.7]$. With the precondition $c \geq c_{\text{base}}$ in Theorem 5, strict monotonicity is preserved for all $\beta > 0$; the $\beta \leq c_{\text{min}}/10$ bound required by the earlier formulation is no longer necessary. Future work (FW3, Section 7.2) will replace these fixed parameters with MARGIN-calibrated, per-actor adaptive weights.

4.7.4 Fork and Confluence Semantics

A fork operation $\text{Fork}(C, a)$, parameterized by an initiating agent a , transforms a concept chain C into a parent-child pair:

$$\text{Fork}(C, a) = (C_{\text{fork}}, C') \quad (19)$$

$$C_{\text{fork}} = C \oplus [e_{\text{FORK}}] \quad (20)$$

$$C' = [e'_{\text{REG}}] \quad (21)$$

where e_{FORK} carries payload $\{\text{fork_target} : C'.\text{concept_id}, \text{reason} : \dots\}$ and e'_{REG} initializes the new concept with a fresh `concept_id`.

Fork-level confluence. For any well-formed chain C and any pair of agents a_1, a_2 :

$$\text{Fork}(C, a_1)_{\text{child}} \perp \text{Fork}(C, a_2)_{\text{child}}$$

where $\perp$ denotes independence: each child chain's derived state depends only on its own event sequence, never on the sibling chain. This follows from the per-chain semantics of δ and the fact that child chains carry distinct `concept_id` values. The parent chain's status becomes forked regardless of which agent initiated the fork, preserving determinism (Theorem 1).

Phase 1 merge rule. When agents reconcile concurrent modifications without forking, the Phase 1 merge rule is last-write-wins by timestamp, with `event_id` as a tie-breaker for equal timestamps. This is a *social coordination convention*, not a CRDT property. It requires agents to agree on a single authoritative branch; divergent branches that cannot be reconciled socially are resolved through fork-and-deprecate. The optional CRDT merge semantics and Phase 3 DAG design are discussed in Section 4.7.5.

We first establish that status derivation is deterministic.

Theorem 1 (Determinism). For any well-formed chain C , $\delta(C)$ is unique and depends only on the last lifecycle event in C .

Proof. By definition, δ filters C to C_{life} (deterministic), checks emptiness (deterministic), and applies the total function f to the last element (unique for ordered sequences). $\square$

Next, we show that fork operations produce well-defined status assignments for both parent and child chains.

Theorem 2 (Confluence under Fork). Let C fork to (C_{fork}, C') . Then $\delta(C_{\text{fork}}) = \text{forked}$ and $\delta(C') = \text{provisional}$.

Proof. Since $e_{\text{FORK}} \in \Sigma_{\text{life}}$ is the last lifecycle event in C_{fork} , we have $f(\text{FORK}) = \text{forked}$. As C' contains only e'_{REG} , it follows that $f(\text{REGISTER}) = \text{provisional}$. $\square$

The following theorem establishes that only lifecycle events affect status transitions.

Theorem 3 (Transition Monotonicity). For any well-formed C and event e_{new} , the transition from $s = \delta(C)$ to $s' = \delta(C \oplus [e_{\text{new}}])$ is valid iff $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$.

Proof. If $e_{\text{new}}.\tau \notin \Sigma_{\text{life}}$, then C_{life} is unchanged, so $s' = s$ (identity transition, always valid). If $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\tau)$, which is exactly the target state defined by the transition matrix. $\square$

We now prove that confidence values remain within the unit interval.

Theorem 4 (Confidence Boundedness). For any well-formed C , $\gamma(C) \in [0, 1]$.

Proof. If $V = []$, $\gamma(C) = 0.0$. Otherwise, each $\phi(a, V) \in [0, 1]$ by payload validation, so their mean is in $[0, 1]$. Thus $c_{\text{base}} \geq 0.5$. The bonus term is non-negative, and the outer $\min(1.0, \cdot)$ caps the result at 1.0. $\square$

We next show that confidence is monotonic with respect to independent validation events.

Theorem 5 (Confidence Monotonicity under Non-Colluding Validation). Let $\gamma(C) = c$ and let e_{new} be a `VALIDATE` event from a new actor who is non-colluding with existing validators, with $e_{\text{new}}.p.\text{confidence} \geq c_{\text{base}}$, where c_{base} is the current base confidence of C . Then $\gamma(C \oplus [e_{\text{new}}]) \geq c$, with strict inequality if $c < 1.0$.

Proof. Let $N'_{\text{vals}} = N_{\text{vals}} + 1$. The proof assumes the new actor is non-colluding with existing validators, so the per-actor maxima reflect independent judgments. The new base confidence c'_{base} is the weighted mean of the N_{vals} per-actor maxima and the new actor's confidence c . Because $c \geq c_{\text{base}} \geq c_{\text{base}}^{\text{raw}}$, the weighted mean satisfies $c'_{\text{base}} \geq c_{\text{base}}^{\text{raw}}$, and therefore $c'_{\text{base}} = \max(0.5, c'_{\text{base}}) \geq \max(0.5, c_{\text{base}}^{\text{raw}}) = c_{\text{base}}$. The bonus term increases by exactly 0.05 (from $0.05 \times (N_{\text{vals}} - 1)$ to

$0.05 \times N_{\text{vals}}$). Hence $\gamma(C') = \min(1.0, c'_{\text{base}} + 0.05 \times N_{\text{vals}}) \geq \min(1.0, c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1) + 0.05) \geq \gamma(C)$. If $c < 1.0$, the $\min(1.0, \cdot)$ cap is not binding for $\gamma(C)$, and the $+0.05$ bonus yields strict inequality. $\square$

Remark (Collusion vulnerability). Theorem 5 assumes non-colluding actors. As demonstrated in the empirical boundary experiment E14 (Section 5.8.2), a single colluding actor can drive $\gamma(C)$ to 0.99. This vulnerability is a known Phase 1 limitation (L9, Section 6.4), addressed by planned MARGIN calibration and staking-based validation (FW3, FW9, Section 7.2).

Finally, we establish consistency between derived status and confidence.

Theorem 6 (Status–Confidence Consistency). If $\delta(C) = \text{validated}$, then $\gamma(C) \geq 0.5$.

Proof. $\delta(C) = \text{validated}$ implies that the last lifecycle event is **VALIDATE**, so $V \neq \emptyset$. Thus $c_{\text{base}} = \max(0.5, \text{mean}(\cdot)) \geq 0.5$. By Theorem 4, the final value is at least $c_{\text{base}} \geq 0.5$. $\square$

4.7.5 Optional CRDT Merge Semantics (Phase 3 Only)

The default EventChain is **NOT** a CRDT. It enforces total order via cryptographic linkage (`previous_event_id`). Concurrent appends without coordination create divergent branches that are resolved through fork-and-deprecate (social consensus), not automatic merge. Theorem 7 below applies *only* to the optional LWW-Element-Set (LWW-Set) merge strategy described here, not to the default EventChain semantics. We present it in a separate subsection to prevent confusion with Theorems 1–6, which apply to all EventChains.

Optional LWW-Set merge semantics. For concurrent modifications by different agents that subsequently reconcile, ADL Lite supports an *optional* LWW-Element-Set merge. This is a specific reconciliation strategy for the confidence aggregation function, *not* a general property of the EventChain data structure. When explicitly invoked, concurrent branches B_1, B_2 are reconciled as:

$$\text{Merge}(B_1, B_2) = B_1 \cup B_2 \quad (\text{event union, deduplicated by event_id}) \quad (22)$$

$$\text{with tie-breaking by event_id for equal-timestamp events} \quad (23)$$

The merge operation is commutative, associative, and idempotent, satisfying the CRDT convergence conditions: $\text{Merge}(B_1, B_2) = \text{Merge}(B_2, B_1)$ and $\text{Merge}(\text{Merge}(B_1, B_2), B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3))$.

We emphasize that CRDT merge is an *optional* strategy, not the default. The primary mechanism for handling divergent interpretations is fork-and-deprecate: competing views coexist as independent chains, and the community converges through social consensus—deprecating abandoned forks, validating accepted ones. ADL Lite provides the provenance infrastructure for both approaches without prescribing one over the other.

Phase 3 CRDT design. Phase 3 will explore a Blocklace-style hash DAG as the transport layer, with ADL Lite’s δ/γ semantics as the application layer. The DAG provides partial-order CRDT semantics with $O(|V| + |E|)$ verification complexity while preserving the deterministic derivation functions. Events would be vertices; `previous_event_id` links would be edges; and the DAG’s topological sort would feed into δ and γ unchanged. This design separates the *transport* CRDT (the DAG) from the *application* semantics (the derivation functions), ensuring that Theorem 7’s convergence properties apply to the transport layer while Theorems 1–6 continue to hold at the application layer.

Theorem 4.1 (CRDT Convergence). For any well-formed concurrent branches B_1, B_2 derived from the same genesis event, the LWW-Set merge satisfies:

$$\text{Merge}(B_1, B_2) = \text{Merge}(B_2, B_1) \quad (\text{commutativity}) \quad (24)$$

$$\text{Merge}(\text{Merge}(B_1, B_2), B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3)) \quad (\text{associativity}) \quad (25)$$

$$\text{Merge}(B_1, B_1) = B_1 \quad (\text{idempotence}) \quad (26)$$

Furthermore, after merge, $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined.

Proof. Events are deduplicated by `event_id`, so the union is idempotent. Union is commutative and associative by set semantics. δ depends only on the last lifecycle event in the merged set; for equal-timestamp ties, `event_id` provides a total order, making δ deterministic. $\square$

Corollary (Event-Level CRDT). Each event e is a trivial CRDT: it is immutable (no update operation), so trivially convergent. The chain C as an append-only log is a grow-only set (G-Set) CRDT.²

4.8 Comparison with Formal Event Frameworks

ADL Lite’s event model may be compared with three classical formal frameworks for reasoning about events and change: Event Calculus, Situation Calculus, and Description Logic.

4.8.1 Event Calculus

The Event Calculus (EC), introduced by Kowalski and Sergot [1986], is a logic programming framework for reasoning about events and their effects. EC distinguishes *events* (occurrences that initiate or terminate *fluents*), *fluents* (properties that hold over intervals of time), and *axioms* that specify which events initiate or terminate which fluents.

In EC, a capability’s status would be modeled as a fluent: `Validated( $c$ )` holds over an interval $[t_1, t_2]$ if a `VALIDATE` event initiates it at t_1 and no subsequent `DEPRECATE` event terminates it before t_2 . ADL Lite’s $\delta(C)$ function computes exactly this fluent, but it does so by examining the entire event history rather than maintaining a set of currently holding fluents.

The principal difference concerns temporal representation. EC uses *interval-based* temporal reasoning: fluents hold over intervals, and events are points that initiate or terminate intervals. ADL Lite uses *chain-based* temporal reasoning: the current state is computed by scanning the ordered event sequence from genesis to present. Both approaches are equivalent in expressive power for the class of properties that ADL Lite models (status transitions with no continuous change), but the chain-based approach is computationally simpler— $\delta(C)$ is $O(n)$ in the chain length—and provides built-in cryptographic integrity.

EC also supports *continuous change* (e.g., a moving object’s position changes continuously over time). ADL Lite does not support continuous change; all state changes are discrete events. This is a deliberate limitation: the target domain (capability lifecycle governance) involves discrete state transitions (proposed $\rightarrow$ validated $\rightarrow$ deprecated), not continuous physical processes.

²A TLA⁺ specification of the EventChain state machine (`EventChain.tla`, 147 lines) is provided as supplementary material. The specification defines the EventChain as a state variable with `AppendEvent` as a TLC action, and formalizes the seven theorems as temporal properties. TLC model checking (v2.18, default configuration) verifies: (i) safety—no invalid transitions occur for all event sequences of length ≤ 20 over the closed alphabet Σ ; (ii) liveness—every FORK action eventually produces a child chain with distinct `concept_id`; (iii) determinism— $\delta(C)$ is unique for all reachable states. While the proof sketches in this section and Appendix E provide the formal foundation, the TLA⁺ model enables mechanised verification of these properties for bounded chain lengths. Full machine-checked proofs in Coq or Isabelle/HOL are planned as future work (FW10, Section 7.2).

4.8.2 Situation Calculus

The Situation Calculus (SC), developed by McCarthy and Hayes [1969], is a first-order logical framework for reasoning about action and change. SC distinguishes *situations* (snapshots of the world at a point in time) and *actions* (functions that map situations to situations). The fundamental axiom of SC is the *frame axiom*: an action changes only the properties it is specified to change; all other properties remain unchanged.

ADL Lite’s EventChain can be viewed as a sequence of situations: each event e_i transforms the situation S_{i-1} into S_i . The status derivation $\delta(C)$ computes the value of a specific fluent (status) in the final situation S_n . However, ADL Lite does not explicitly represent situations; it represents only the event sequence. This is analogous to the *event sourcing* pattern in software engineering: the state is reconstructed by replaying events, not by storing snapshots.

The frame axiom in SC is a source of the *frame problem*: specifying which properties do *not* change after an action requires an unbounded number of axioms. ADL Lite avoids the frame problem by construction: as there are no stored mutable fields, no properties need preservation across state transitions. All derived properties are recomputed from the event sequence, so the “frame” is implicit in the derivation functions.

4.8.3 Description Logic Expressivity

ADL Lite’s derivation functions can be analyzed in terms of Description Logic (DL) expressivity. The status derivation $\delta(C)$ is a function from a sequence of events to a status value. In DL terms, this is a *role chain* query: the status of a capability is determined by the type of the last event in the chain of events related to the capability by the `previous_event_id` role.

The derivation functions δ and γ are computable in polynomial time ($O(n)$ in the chain length). They do not require the full expressive power of OWL-DL (which is NExpTime-complete for satisfiability). In fact, δ and γ can be expressed in the DL-lite family of lightweight description logics, which support efficient query answering over large datasets Calvanese et al. [2007].

Specifically, δ corresponds to a *path query* over the event chain: “find the last event in the chain whose type is in Σ_{life} , and return the corresponding status.” This query can be expressed in DL-Lite $\mathcal{R}$ using role inclusion axioms:

$$\exists \text{hasLastLifecycleEvent}^- . \{\tau\} \sqsubseteq \text{hasStatus} . \{f(\tau)\} \quad (27)$$

for each $\tau \in \Sigma_{\text{life}}$, where $f(\tau)$ maps event types to statuses. The confidence aggregation γ involves aggregation (mean, max) over sets of events, which goes beyond standard DL query answering but can be handled by DL extensions with aggregation operators Calvanese et al. [2007].

Expressiveness bounds. ADL Lite can express discrete status transitions governed by deterministic derivation functions. It cannot express: (i) continuous change (e.g., a moving object’s position evolving over time); (ii) temporal constraints with non-trivial interval relations (e.g., “event A must occur before event B” independent of chain order); (iii) existential quantification over event types not in the closed Σ_{life} alphabet; (iv) disjunctive or conditional derivation rules beyond the fixed f mapping. These limitations are by design: the target domain (capability lifecycle governance) requires only discrete, deterministic transitions. For domains requiring richer temporal expressiveness, EC (interval-based) or SC (situation-based) provide the formal apparatus at increased computational cost.

EC correspondence. In Event Calculus terms, $\delta(C)$ computes the fluent $\text{Status}(c, t)$ at time t

by evaluating:

$$\text{HoldsAt}(\text{Status}(c, s), t) \iff \exists e \in C_{\text{life}}. \text{Happens}(e, t_e) \wedge t_e \leq t \wedge \quad (28)$$

$$\neg \exists e' \in C_{\text{life}}. \text{Happens}(e', t_{e'}) \wedge t_e < t_{e'} \leq t \quad (29)$$

where $s = f(e.\tau)$. The key difference is temporal representation: EC uses interval-based reasoning (fluents hold over intervals); ADL Lite uses chain-based reasoning (scan to the last lifecycle event). Both are equivalent for the class of discrete, non-overlapping transitions that ADL Lite models, but the chain-based approach is computationally simpler ($O(n)$ vs EC's $O(n^2)$ for general queries with the common sense law of inertia).

The practical implication is that ADL Lite's derivation semantics are *tractable*: they do not require OWL reasoners or complex inference engines. This aligns with the design goal of lightweight deployment: the ontology can be processed by a simple Python script operating over Markdown files, without requiring a triple store or DL reasoner.

4.9 Trust Model and Security Boundaries

The cryptographic hash chain ensures *content integrity*: any modification, reordering, or deletion of events breaks the chain linkage and is detected by `VerifyIntegrity(C)`. However, the scope of cryptographic assurance in Phase 1 is *deliberately limited*: hash chains detect tampering but do not prevent it, and they provide no actor authentication. A malicious agent with write access to the Git repository can rewrite history (detected by hash mismatch), forge another agent's identity (not detected in Phase 1), or inject arbitrary events (recorded but not prevented). These limitations are acknowledged and scoped to Phase 3 (signatures, DIDs, BFT). This section makes the trust model explicit and identifies known boundaries.

4.9.1 Trust Assumptions

ADL Lite's current phase (v0.2) assumes the following trust boundaries:

- (1) **Trusted storage.** The Git repository or file system storing EventChains is trusted to preserve history integrity. Force-pushes or rebases can rewrite history, but (a) signed Git commits provide tamper-evidence via GPG signatures, and (b) ADL Lite's chain verification detects any modification because the hash chain breaks at the point of alteration. The authoritative chain is determined by agent quorum, not Git HEAD.
- (2) **Trusted authoring environment.** We assume that agents are uncompromised. A compromised agent can inject arbitrary events, but those events are recorded—the chain provides *tamper-evident records* of what occurred, even if what occurred was malicious.
- (3) **Clock skew tolerance.** Event ordering is determined by cryptographic linkage (`previous_event_id`), not by timestamps. Clock skew between agents does not affect chain validity. Timestamps are included in the hash input for provenance but do not determine ordering.

4.9.2 Actor Identity Management in Phase 1

In Phase 1, actor identity is a self-declared string carried in the `actor` field of each event. The following practical considerations apply:

Collision. Actor strings are namespaced by agent instance (e.g., `agent_3` or `org/research-team-7`). Collisions between distinct agents choosing the same string are detected by event audit: two agents

with identical actor strings but divergent event patterns (e.g., contradictory validations on the same capability) produce an auditable conflict record. Resolution is social—the community identifies the collision and one agent renames.

Impersonation. Phase 1 does *not* cryptographically prevent impersonation. An agent can append an event with `actor: "agent_2"` and the system records it. Detection relies on community audit of event patterns: an agent posting events inconsistent with the historical behaviour of the impersonated identity triggers manual review. This is a deliberate trade-off: cryptographic identity is deferred to Phase 3 to keep Phase 1 deployable without PKI infrastructure.

Replay attacks. A replayed event—copying an existing event and re-appending it—breaks the hash chain because the replayed event’s `previous_event_id` differs from the current chain tail. `VerifyIntegrity(C)` detects this as a linkage break at the point of insertion. Even if an attacker recomputes hashes, the replayed event’s `event_id` duplicates an existing one, violating well-formedness axiom (5) (pairwise distinct `event_id` values).

Equivocation. An agent can post contradictory events across forks (e.g., validating capability X on chain C_1 while deprecating it on chain C_2). ADL Lite does not prevent this; it *records* them. Equivocation becomes part of the public audit trail and can be queried by `actors(V)` aggregation. Resolution is social: the community observes the contradiction and deprecates the inconsistent branch. Equivocation evidence is a feature, not a bug—it exposes dishonest behaviour for community judgment rather than suppressing it silently.

4.9.3 Known Limitations and Planned Mitigations

Limitation 1: No actor identity verification (Phase 3). Currently, event actors are self-declared strings. An agent can claim to be another agent when appending events. Planned Phase 3 will add W3C Linked Data Proofs W3C Verifiable Credentials Working Group [2024]: each event will carry an Ed25519 signature over canonicalized event JSON, verifiable against a public key published via Decentralised Identifiers (DIDs) W3C DID Working Group [2022] or key transparency logs Dahlberg et al. [2020]. Key revocation will be published to transparency logs, and Merkle tree batch verification will provide $O(\log n)$ audit complexity for signature validation. The hash chain remains the integrity layer; signatures add the authentication layer. Integration with Git-based workflows will use Git’s existing GPG commit signing as a stopgap: signed Git commits bound to signed ADL events create a two-layer attestation (Git commit signs the file; ADL event signs the content).

Limitation 2: No Byzantine resilience. A coalition of compromised agents controlling a majority of validators can drive confidence arbitrarily high. ADL Lite does not implement Byzantine fault tolerance. For applications requiring BFT, the EventChain layer can be deployed over a BFT consensus protocol (e.g., PBFT Castro and Liskov [1999]) at the transport layer, with ADL Lite providing the semantic governance layer above.

Limitation 3: No equivocation prevention. An agent can post contradictory events (e.g., validating and deprecating the same capability) across different chains. This is not prevented because ADL Lite treats events as evidence: contradictory events are evidence of a conflict, not a system failure. Resolution is social (community deprecation of the inconsistent branch), not cryptographic. Phase 3 signatures will make equivocation *cryptographically attributable* (the same key signs both contradictory events), but will not prevent it by protocol design.

Limitation 4: Genesis event immutability. In Git-based workflows, a force-push can rewrite the genesis event. However: (a) the hash chain from the genesis event onward would detect any change; (b) signed Git commits provide additional tamper-evidence; (c) the capability’s identity is tied to the genesis event’s hash—rewriting it creates a *different capability* with a different identity.

The original capability’s chain persists in the Git reflog and in any agent’s local copy.

4.9.4 Threat Model Summary

Table 9: Threat model for ADL Lite v0.2. ✓ = mitigated; ◦ = detected but not prevented; × = not addressed.

Threat	Status	Mitigation
Content tampering	✓	Hash chain (SHA-256 linkage)
Event reordering	✓	<code>previous_event_id</code> linkage
Event deletion	✓	Hash chain break detection
Actor impersonation	×	Planned: Ed25519 + DIDs (Phase 3)
Equivocation	◦	Detected as conflicting events; social resolution
Clock skew	✓	Ordering via linkage, not timestamps
Genesis replacement	◦	Hash break detection; Git reflog recovery
Byzantine majority	×	Planned: optional BFT transport layer

4.9.5 Fault Detection and Recovery

ADL Lite employs a multi-layer recovery strategy for chain corruption scenarios.

Detection. `VerifyIntegrity(C)` identifies the exact point of failure in $O(n)$ time: it returns the index i of the first event where $e_i.h \neq \text{SHA-256}(\text{Canon}(e_i))$ or $e_i.h_{\text{prev}} \neq e_{i-1}.h$. Corruption is classified as: (a) *content tampering* (hash mismatch at a single event); (b) *linkage break* (prev_hash mismatch between consecutive events); or (c) *truncation* (chain ends before expected length).

Recovery strategies.

- (1) **Git reflog recovery.** Each chain version is preserved in Git’s history. `git reflog` can restore any prior state. If the corrupted chain is the current HEAD, the last known well-formed version can be checked out.
- (2) **Distributed peer recovery.** In multi-agent deployments, each agent maintains a local copy. The authoritative chain is selected by agent quorum: the longest well-formed chain (verified by `VerifyIntegrity`) from the set of peer copies is adopted.
- (3) **Partial repair.** If only tail events are corrupted, the chain can be truncated to the last well-formed prefix $C[1..k]$. The truncation is recorded as an **EVIDENCE** event noting “integrity violation detected at event $k + 1$; chain truncated.” Truncated events remain in Git history.
- (4) **Rollback governance.** Any recovery action produces a governance event under the same precondition rules as normal operations, ensuring auditable recovery.

Limitations. Phase 1 assumes at least one well-formed copy exists. Total data loss causes capability cessation (§3.4). Automated reconciliation of conflicting well-formed copies is deferred to Phase 3 (CRDT merge, Theorem 7). Phase 1 recovery requires agent intervention to select the authoritative copy.

5 Empirical Validation

5.1 Validation Strategy: Correctness as Capability-Registry Consistency

The empirical evaluation of ADL Lite is designed to validate three capability-registry hypotheses:

- (1) **Integrity hypothesis.** The cryptographic hash chain correctly detects all structural modifications (content tampering, reordering, deletion).
- (2) **Derivation hypothesis.** The status and confidence derivation functions produce correct results for all possible event sequences.
- (3) **Precondition hypothesis.** The action precondition system correctly accepts valid actions and rejects invalid ones.

These hypotheses are validated through five experiments: chain integrity (E1), status derivation exhaustiveness (E2), snapshot round-trip consistency (E3), precondition enforcement (E4), and scalability on real-world data (E6). The experiments do not constitute a domain-level evaluation of anti-money-laundering (AML) effectiveness; they validate that the event-first capability-registry architecture is computationally feasible and empirically correct.

5.2 Chain Integrity and Tamper Detection (E1)

E1 tests the integrity verification predicate $\text{VerifyIntegrity}(C)$ on 60 chains: 50 valid chains and 10 intentionally corrupted chains. The corrupted chains exhibit four attack classes:

- **Content tampering:** Modify the payload of an existing event.
- **Event reordering:** Swap two events in the chain.
- **Event deletion:** Remove a middle event.
- **Event replay:** Copy an event from one chain into another.

Results: precision = 1.0, recall = 1.0, F1 = 1.0. All 50 valid chains pass verification; all 10 corrupted chains fail verification. Attack detection is deterministic: any modification to an event’s content, reordering of events, or deletion of events breaks the cryptographic linkage at the point of first alteration.

5.3 Status Derivation Exhaustiveness (E2)

Next, E2 tests the status derivation function $\delta(C)$ on all possible event sequences of length up to 3 over the 5 lifecycle event types, plus 7 edge cases (empty chain, single-event chains, full lifecycle sequences, mixed lifecycle/communication sequences). The total test corpus comprises 2,204 cases.

Results: 2,204/2,204 correct (accuracy = 1.0). Edge cases confirm that communication events (Σ_{comm}) never modify status, while lifecycle events (Σ_{life}) deterministically drive transitions according to the transition matrix. This validates Theorem 1 (Determinism) and Theorem 3 (Transition Monotonicity) empirically.

5.4 Snapshot Round-Trip Consistency (E3)

E3 tests the consistency between derived snapshot state and original front-matter state. For each concept file, the experiment parses the file to produce an EventChain, generates a fresh snapshot from the chain via `ADLFrontMatter.from_chain(·)`, and compares the derived status, confidence, and validators against the original front-matter values.

Method: All example concept files ($n = 11$) and AML-derived concept files ($n = 201$) are parsed and verified. The round-trip must preserve status exactly (equality), confidence within ± 0.01 tolerance (floating-point safety), and validator sets exactly (order-independent).

Results: 212/212 files passed (100%). Zero status mismatches, zero confidence deviations exceeding tolerance, and zero validator set discrepancies. This validates that the snapshot derivation is a correct function of the event chain (Theorem 3) and that the front-matter cache is consistent with the authoritative chain state.

5.5 Precondition Enforcement (E4)

Furthermore, E4 tests the ActionExecutor’s precondition validation on all 9 registered actions across 19 test cases covering valid preconditions, violated preconditions, boundary values, and type mismatches. To address the boundary-coverage gap identified in peer review, we extended E4 with five additional boundary-test classes:

- **Concurrent action execution (E4a):** 50 test cases simulate two agents appending `VALIDATE` events to the same concept simultaneously. The ActionExecutor correctly rejects the second `VALIDATE` when the precondition requires `status == provisional` and the first `VALIDATE` has already changed the derived status to `validated`. All 50 cases passed ($P = R = F1 = 1.0$), confirming that precondition evaluation is race-safe because it operates on the chain-derived snapshot, not on a cached mutable state.
- **Extreme confidence values (E4b):** 12 test cases exercise boundary confidence values: 0.0, 0.5, 1.0, and NaN. The NaN case was correctly rejected by the Pydantic payload validator before precondition evaluation. Values 0.0 and 1.0 were accepted when preconditions permit; 0.0 was rejected for `VALIDATE` (minimum 0.5). All 12 cases passed.
- **Long-chain precondition evaluation (E4c):** 10 test cases evaluate preconditions on chains of length 1,000, 5,000, and 10,000 events. Precondition evaluation time remains $O(1)$ because the ActionExecutor inspects only the derived snapshot (L1 YAML front matter), not the full chain history. The mean evaluation time was 0.08 ms ($\sigma = 0.01$ ms) across all chain lengths, confirming that precondition checking does not degrade with chain growth.
- **Multi-agent concurrency stress (E4d):** 50 test cases simulate 3–10 agents appending events concurrently to shared chains. Race-condition tests verify that when two agents simultaneously `VALIDATE` the same provisional concept, the first succeeds and the second is rejected (or creates a `FORK` when `fork-on-conflict` is enabled). Fork-concurrency tests verify that multiple agents forking the same parent chain simultaneously produce distinct child chains with consistent lineage. All 50 cases passed ($P = R = F1 = 1.0$). The ActionExecutor’s chain-derived snapshot prevents race conditions because preconditions are evaluated against the current chain state rather than a cached mutable state.
- **Adversarial tampering detection (E4e):** We reference the adversarial integrity test suite detailed in Appendix C, which comprises 32 cases across 8 attack classes (content tampering,

reordering, deletion, replay, hash collision simulation, synthetic event injection, scope escalation, and comparator bypass). All integrity-violating attacks were detected with 100% recall. These tests validate detection, not prevention; cryptographic prevention of tampering requires the Phase 3 authentication layer.

Aggregated E4 results: precision = 1.0, recall = 1.0, F1 = 1.0 across the original 19 cases, 72 extended boundary cases, and 50 multi-agent concurrency cases (141 total). The 32 adversarial cases in Appendix C are reported separately. There were zero false positives and zero false negatives. The closed Comparator enumeration eliminates runtime code injection; the absence of `eval()` ensures that precondition evaluation is decidable and safe.

5.6 Scalability on Real-World Data (E6)

To evaluate the architectural scalability of the capability registry, E6 subjects the EventChain data structure to a volume stress test using the IBM Anti-Money Laundering (AML) HI-Small dataset as a source of realistic transaction records. The dataset comprises 9,300 transactions from 201 accounts. Each account is imported as an EventChain; transactions become `Event` objects. The mean chain length is 46.3 events; the maximum is 300 events.

Reproducibility protocol. Throughput was measured on Apple M2 (8P+4E cores, 3.49 GHz), 16 GB LPDDR5, macOS 14, Python 3.10, NVMe SSD (~ 5 GB/s read). Ten independent runs were executed with cold-start cache flushing (`sync; echo 3 | sudo tee /proc/sys/vm/drop_caches` on Linux; equivalent macOS `purge` command). Results: mean throughput = 20,847 events/sec ($\sigma = 312$ events/sec, 95% CI = [20,654, 21,040] computed via the t-distribution with 9 degrees of freedom, CV = 1.5%), confirming low run-to-run variance. Wall-clock runtime mean = 446 ms ($\sigma = 7$ ms, 95% CI = [442, 450] ms, t-distribution). All 201 chains maintained integrity (zero failures) across all runs.

Baseline comparison. The same 9,300-event CSV parsed without EventChain construction (raw CSV read + field counting) completed in 98 ms (94,900 records/s). The EventChain construction adds $3.5\times$ overhead (446 ms vs. 98 ms), dominated by Pydantic validation, SHA-256 hashing, and deterministic chain verification. This overhead is the cost of verifiable provenance and is acceptable for batch ingestion scenarios where audit completeness is valued over raw ingestion speed.

Latency decomposition. Profiling reveals the following breakdown per 1,000 events: CSV parsing 13.1%, event materialization (Pydantic) 58.4%, SHA-256 hashing 15.9%, chain verification 10.0%, memory allocation/GC 2.5%. The bottleneck is event materialization, not cryptographic operations.

Architectural contribution, not domain validation. E6 establishes that the EventChain architecture scales to real-world data volume with linear throughput. The IBM AML HI-Small dataset was used as a volume stress test. Transaction records were imported as `REGISTER` events; governance events (`VALIDATE`, `DEPRECATE`, `FORK`, `EVIDENCE`) were synthetically injected to simulate capability-lifecycle workflows. The 96.8% `REGISTER` / 3.2% governance split reflects a realistic data-governance ratio but does not validate domain-level AML effectiveness. The pattern-detection logic is a structural heuristic; it has not been calibrated against ground-truth laundering labels from financial-crimes experts.

Dataset derivation and event type distribution. The 201 chains and 9,300 events were derived as follows: each unique account in the IBM AML HI-Small dataset (filtered subsample) became a concept identified by its `account_id`. Incoming and outgoing transactions were imported as `REGISTER` events in the corresponding account’s EventChain via `DataImporter.import_csv()`.

Governance events (**VALIDATE**, **DEPRECATE**, **FORK**, **EVIDENCE**) were synthetically injected by the experiment harness to simulate multi-agent capability-lifecycle workflows.

Synthetic event generation strategy. Governance events were injected by stratified random sampling: **VALIDATE** at 2.2% (uniform over chains with ≥ 5 events, one per eligible chain), **DEPRECATE** at 0.5% (targeting chains with ≥ 10 events, stratified by transaction volume), **FORK** at 0.3% (random chain selection without replacement), and **EVIDENCE** at 0.2% (uniform over all chains). This strategy yields a realistic ratio of approximately one governance event per 30 data events, consistent with operational ontology governance workflows where raw data ingestion dominates and governance operations are sparse. Table 10 reports the event type distribution.

Table 10: Event type distribution across the 201 E6 evaluation chains.

Event Type	Count	Fraction	Source
REGISTER	9,000	96.8%	IBM AML transaction import
VALIDATE	200	2.2%	Synthetic: agent validation workflow
DEPRECATE	50	0.5%	Synthetic: deprecation of suspicious accounts
FORK	30	0.3%	Synthetic: concept divergence simulation
EVIDENCE	20	0.2%	Synthetic: audit evidence attachment
Total	9,300	100%	

The predominance of **REGISTER** events (96.8%) reflects the data import pipeline: each transaction becomes a registration, and governance operations are sparse relative to the raw data volume. The synthetic governance events (3.2%) simulate a realistic multi-agent workflow at a ratio of approximately one governance event per 30 data events. The test case coverage is as follows: the 2,204 E2 status derivation cases exhaustively cover all event type combinations of length ≤ 3 over the closed Σ_{life} alphabet; the 19 E4 precondition cases cover all 9 registered actions with valid, violated, boundary, and type-mismatch scenarios; the 32 adversarial cases (Appendix C) extend coverage to integrity attacks not present in the AML-derived data.

5.7 Scale-Up Feasibility Projection (E6b)

The E6 throughput measurement of 20,955 events per second (§5.6) enables a linear scaling projection to larger deployments. At the measured throughput, 100,000 events ingest in approximately 4.8 s; 1,000,000 events ingest in approximately 47.7 s. Memory analysis estimates each serialized event at ≈ 600 bytes, so 1 M events require ≈ 600 MB of RAM, well within modern server capacity. The profiling bottleneck remains Pydantic event materialization (58.4% of latency); a transition to Rust-based serialization or Pydantic v2 core offers a potential 2–3 $\times$ speedup. Git storage of 1 M events is ≈ 600 MB raw; Git compression reduces this to ≈ 200 MB. We conclude that the architecture is feasible for 10^5 – 10^6 event deployments without fundamental redesign.

5.8 Boundary Conditions and Negative Results (E13–E16)

To complement the architectural correctness experiments (E1–E3, E4, E6), we conducted four boundary-condition experiments designed to expose failure modes, performance limits, and adversarial vulnerabilities. These experiments are reported transparently as negative or bounded results; they do not claim success but rather quantify the limits of the Phase 1 system.

5.8.1 Long-Chain Integrity Stress (E13)

E13 measures the performance and integrity of VERIFYINTEGRITY on chains of length 100 to 50,000 events. The experiment constructed chains by repeatedly appending REGISTER and VALIDATE events, then measured wall-clock verification time and memory footprint via `tracemalloc`.

Results. Verification time scales linearly with chain length: slope = 19.0 ms per 1,000 events, intercept = -0.5 ms, $R^2 = 1.0$. At 50,000 events, verification completed in 949.8 ms and memory peaked at < 1 MB. Integrity holds for all chain lengths: zero verification failures. The linear scaling confirms the formal $O(n)$ complexity bound (Section 4.7), and the modest memory footprint (< 1 MB for 50k events) confirms that hash-chain verification is not memory-bound. The practical limit for interactive use is approximately 10^4 events (~ 190 ms); beyond 10^5 events, verification latency exceeds 1 s and would require asynchronous background checking.

5.8.2 Colluding-Validator Attack (E14)

E14 tests the vulnerability identified in Limitation L3 (un-calibrated confidence aggregation). A coalition of k colluding validators each appends a VALIDATE event with *confidence* = 0.99. The experiment measures the final aggregated confidence $\gamma(C)$ and the coalition size required to reach the maximum.

Results. With $k = 1$, the final confidence reached 0.99 and the concept transitioned to **validated**. The status and confidence were fully controlled by a single malicious actor. This confirms that Phase 1’s self-reported confidence aggregation is vulnerable to collusion; there is no stake, no reputation penalty, and no multi-signature requirement to prevent a single actor from dominating the consensus. The mitigation is scoped to Phase 3: MARGIN calibration (FW3) and staking-based validation (FW9).

5.8.3 Precondition Boundary Stress (E15)

E15 tests malformed and adversarial inputs that are outside the normal test matrix: NaN, infinity, negative confidence, confidence > 1.0 , empty strings, very long strings (10^4 characters), type confusion (string where float expected), and boundary values (0.5, 1.0).

Results. Of 11 test cases, 4 were caught by Pydantic payload validation (NaN, inf, negative, > 1.0), 2 slipped through the precondition layer (empty actor parameter, zero-length concept ID), and 2 produced unexpected outcomes (long reasoning string was accepted; string confidence was caught by Pydantic, not by the precondition comparator). The 2 slipped-through cases are benign—they do not violate safety—but they reveal a *defense-in-depth gap*: the precondition layer does not enforce all semantic constraints independently of Pydantic. The closed **Comparator** enum prevents code injection, but type confusion at the JSON/YAML boundary (e.g., a string literal where a float is expected) is caught by Pydantic, not by preconditions. This division of labor is acceptable for normal operation (Pydantic handles type safety; preconditions handle semantic validity), but it means that a bypass of Pydantic—for example, through direct EventChain API access that circumvents the ActionExecutor—could admit malformed inputs. Closing this defense-in-depth gap is scoped to Phase 2 (FW2).

5.8.4 Multi-Agent Contention Simulation (E16)

E16 simulates k independent agents ($k = 2, 5, 10, 20$) concurrently attempting to VALIDATE the same provisional concept. The experiment measures conflict rate (fraction of agents rejected due to status change), fork rate (when fork-on-conflict is enabled), and integrity preservation.

Results. Conflict rate increases with k : 50% for $k = 2$, 80% for $k = 5$, 90% for $k = 10$, 95% for $k = 20$. Integrity was preserved in all cases: VERIFYINTEGRITY passed despite concurrent appending. The fork rate is 0.0 because fork-on-conflict is not implemented in Phase 1; rejected agents simply fail. This confirms that the `threading.Lock` around `EventChain.append` prevents data races, but it does not provide a graceful degradation strategy for high-contention scenarios. A fork-on-conflict mechanism (Phase 3) would convert rejected validations into FORK events, reducing conflict rate at the cost of concept proliferation.

Interpretation. The boundary experiments reveal four bounded properties of the Phase 1 system: (1) integrity verification scales linearly to 50k events with modest latency; (2) confidence aggregation is vulnerable to single-actor collusion; (3) precondition enforcement relies on Pydantic for type safety and does not independently catch all malformed inputs; (4) concurrent access is race-safe but produces high rejection rates under contention. quantified limits that guide Phase 3 prioritization.

5.9 Experiment E5: Domain-Level Evaluation (Planned)

While E1–E3, E4, and E6 establish architectural correctness of the capability registry, domain-level effectiveness requires evaluation against expert judgment and existing domain baselines. This section describes the *planned* methodology for a domain-level evaluation of ADL Lite’s concept governance capabilities on the IBM Anti-Money Laundering (AML) HI-Small dataset. The evaluation is ongoing; preliminary results are not yet available for inclusion in this revision. The present paper is therefore positioned as a *framework paper*: it establishes the capability-registry foundations, formal semantics, and architectural correctness of ADL Lite, while domain-level validation is scoped to future work.

Dataset. The IBM AML HI-Small corpus contains 5,080,714 transactions across 495,671 accounts, with 3,386 suspicious accounts flagged by the dataset’s ground-truth labels IBM [2024]. Four structural laundering patterns are present: rapid movement (≥ 10 laundering events), cyclic flows (money returns to origin), fan-out (≥ 5 unique recipients), and smurfing (sub-\$1,000 structuring).

Procedure. ADL Lite’s DataImporter ingests the transaction CSV and discovers four concept classes corresponding to the laundering patterns. Two to three AML domain experts independently score each discovered concept on three dimensions: (1) *semantic coherence* (does the concept correspond to a recognized laundering pattern?), (2) *evidential sufficiency* (does the EventChain contain sufficient evidence to justify the concept’s status?), and (3) *audit completeness* (does the chain provide a complete, tamper-evident audit trail?). Scores use a 5-point Likert scale. Inter-rater agreement is measured with Cohen’s κ .

Baseline comparison. ADL Lite’s audit completeness is compared against graph neural network (GNN) approaches to AML detection (GIN, GAT, GraphSAGE). While GNNs achieve high detection accuracy, their “black-box” reasoning provides no explainable audit trail. ADL Lite’s contribution is not competitive detection accuracy but *complete audit provenance*: every status transition is cryptographically linked to the evidence that justified it.

Expected outcomes and timeline. We expect high scores on audit completeness ($\geq 4.5/5.0$) and semantic coherence ($\geq 3.5/5.0$), with the primary variance coming from evidential sufficiency (which depends on DataImporter’s heuristic pattern detection). Expert recruitment and annotation are underway; full results will be reported in a future publication.

A pilot LLM-agent capability evaluation (E17) is also planned, in which multiple LLM agents register, validate, and deprecate capabilities within a shared ADL Lite registry. Results will be reported in a future publication.

5.10 Summary of Results

Table 11 summarizes the empirical validation. All architectural correctness experiments (E1, E2, E3, E4, E6) achieved perfect scores, providing strong evidence for the three capability-registry hypotheses. The experiments validate that the event-first architecture is computationally feasible and empirically correct; they do not, by themselves, demonstrate domain-level effectiveness. That validation is scoped to future work (E5, E6M). The present paper is positioned as a *framework paper*: it establishes the capability-registry foundations, formal semantics, and architectural correctness of ADL Lite.

Table 11: Summary of empirical validation results.

Experiment	Hypothesis	Metric	Result
E1	Integrity	P / R / F1	1.0 / 1.0 / 1.0
E2	Derivation	Accuracy	1.0 (2,204/2,204)
E3	Snapshot consistency	Status/confidence/validators match	212/212 (100%)
E4	Precondition	P / R / F1	1.0 / 1.0 / 1.0
E6	Scalability	Integrity	0 failures
E6b	Scale-up projection	Feasibility	Linear to 10^6 events
E12	Governance benchmark	Throughput / latency	135k evt/s; 69ms all chains
E13	Long-chain stress	Integrity / latency	$R^2 = 1.0$; < 1 MB at 50k
E14	Collusion vulnerability	Confidence control	1 actor $\rightarrow \gamma = 0.99$
E15	Precondition boundary	Input rejection	4/11 caught by Pydantic
E16	Contention simulation	Conflict rate	95% at $k = 20$; integrity prese

Coverage and failure modes. The 141 E4 precondition and concurrency test cases (19 original + 72 boundary + 50 multi-agent) cover all 9 registered actions $\times$ 8 comparators $\times$ 3 value types (scalar, range, enum) = 216 possible combinations, yielding 65% combinatorial coverage and 100% of critical paths. The 2,204 E2 derivation cases exhaustively cover all event sequences of length ≤ 3 over 5 lifecycle types (100% of the length- ≤ 3 space); length-4 coverage is partial (500 sampled cases). No failure modes were observed in architectural correctness tests. Hypothetical failure modes include: (1) hash collision (computationally infeasible for SHA-256); (2) Byzantine majority (not addressed in Phase 1); (3) total data loss (all copies destroyed); (4) colluding validators (Theorem 5 assumes non-collusion; see E14).

5.11 Planned Human-in-the-Loop Evaluation: Methodology

To complement the architectural correctness evaluation (E1–E4, E6), we propose a human-in-the-loop study measuring ADL Lite’s practical utility for capability curation.

Study design. A between-subjects experiment with two conditions:

- **Control:** Curators use standard Markdown + Git for collaborative concept governance.
- **Treatment:** Curators use ADL Lite with EventChain validation, precondition enforcement, and automated provenance tracking.

Participants. 10–15 ontology curators per condition, recruited from the OBO Foundry community and biomedical ontology mailing lists. Participants have at least 2 years of ontology curation experience.

Task. Participants collaborate on curating a set of 20 AML-related capability concepts over a 2-week period. Concepts include known laundering patterns (smurfing, rapid movement, layering) and novel patterns discovered during the study.

Metrics.

- (1) *Curator agreement*: Inter-rater reliability (Cohen’s κ) on concept validation decisions.
- (2) *Time-to-resolution*: Median time from concept proposal to final status (validated/deprecated).
- (3) *Conflict rate*: Number of FORK events per concept.
- (4) *Audit completeness*: Fraction of status transitions with complete evidence chains.
- (5) *Perceived utility*: Post-study questionnaire (5-point Likert) on perceived auditability, trust, and collaboration quality.

Expected outcomes. We expect the Treatment condition to demonstrate: (a) higher curator agreement due to structured precondition enforcement; (b) faster time-to-resolution due to automated status derivation; (c) lower conflict rate due to deterministic fork semantics; (d) higher audit completeness by design; (e) higher perceived utility scores, particularly for auditability.

Ethics. The study will be approved by the institutional ethics committee. All participants provide informed consent. Data is anonymized and stored in compliance with GDPR.

5.12 Comparative Capability-Governance Evaluation (E12)

To position ADL Lite relative to established provenance solutions, we present a structured capability-governance comparison against nanopublications with Trusty URIs and a PROV-O pipeline on a governance task suite. This comparison is methodological rather than experimental—we report operational characteristics rather than benchmark timings, acknowledging that a full empirical comparison requires the authentication layer planned for Phase 3.

5.12.1 Governance Task Suite

We define four governance tasks representative of multi-agent concept lifecycle management:

- (1) **Acceptance workflow.** A concept is proposed, reviewed by k validators, and accepted only if the quorum threshold (minimum validations) is met. Measure: number of system steps required to reach final status.
- (2) **Retraction/deprecation workflow.** A previously accepted concept is deprecated based on new evidence. Measure: completeness of the audit trail linking deprecation to the evidence that justified it.
- (3) **Audit query.** A downstream consumer asks: “What was the status of concept X at time t , and who contributed to its validation?” Measure: time to answer ($O(1)$ for nanopub lookup vs. $O(n)$ for EventChain scan).
- (4) **Consensus threshold check.** Determine whether a concept has reached a configurable confidence threshold (e.g., $\gamma(C) \geq 0.7$) and which validators contributed. Measure: computational cost of threshold evaluation.

5.12.2 Comparison Results

Table 12 presents the qualitative governance task comparison. Table 13 supplements this with quantitative benchmarks measured on identical hardware (Apple M2, macOS 14, Python 3.10) via the E12 experiment.

Table 12: Governance task comparison. ✓ = supported; ◦ = partially supported; × = not supported.

Task	Nanopubs Trusty URIs	+ PROV-O Pipeline	ADL Lite (v0.2)
Acceptance workflow	◦ (manual versioning)	◦ (requires external workflow engine)	✓ (deterministic $\delta(C)$)
Retraction/deprecation	◦ (new nanopub with retraction)	✓ (prov:wasInvalidatedBy)	✓ (DEPRECATE + precondition)
Audit query	✓ ($O(1)$ per nanopub)	✓ (SPARQL query)	◦ ($O(n)$ linear scan)
Consensus threshold	× (no built-in aggregation)	× (no built-in aggregation)	✓ ($\gamma(C)$ with quorum rules)
Multi-event lifecycle	◦ (chained nanopubs)	✓ (activity sequences)	✓ (native EventChain)
Verification cost	$O(1)$ per nanopub	$\sim O(\text{activities})$	$O(n)$ per chain
Authentication	✓ (RSA signatures)	◦ (planned: LD-Proofs)	× (planned: Phase 3)
Infrastructure	RDF triplestore + nanopub server	RDF triplestore + PROV tooling	Git + pip install adl-lite

Table 13: Empirical benchmark comparison (E12). ADL Lite measurements on Apple M2, macOS 14, Python 3.10. Nanopub and PROV-O figures are estimated from published specifications; head-to-head benchmark deferred to Phase 3.

Metric	ADL Lite (measured)	Nanopubs (published)	P
Per-assertion verify	0.59 μ s (SHA-256 hash only)	$O(1)$ hash comparison Kuhn and Dumontier [2015]	O
Chain verify (300 events)	2.13 ms	N/A (atomic claims)	N
Audit query (300 events)	0.022 ms	$\sim$ SPARQL lookup ms ^a	$\sim$
Storage per event	603 bytes	31–86% non-assertion triples Kuhn et al. [2015]	$\sim$
Hash/storage overhead	10.6% (64 bytes/event)	Content-addressed URI overhead	O
Throughput	135k events/s (verify)	Per-claim only	P
All 201 chains verify	69 ms (total)	N/A (no chain concept)	N

^aEstimated from published specifications; not empirically measured on identical hardware.

Interpretation. Nanopublications with Trusty URIs provide $O(1)$ per-claim hash verification with built-in RSA signatures—an authentication capability ADL Lite currently lacks. However, nanopubs are atomic units without built-in lifecycle state machines, consensus thresholds, or multi-event provenance chains. PROV-O pipelines offer rich workflow descriptions but require external

execution engines and incur RDF canonicalization overhead.

Capability-governance efficacy. Nanopublications provide per-claim integrity but no lifecycle state machine; status transitions, if any, are managed externally. ADL Lite provides deterministic status derivation (δ , γ) natively, with preconditions enforced at append time. PROV-O provides rich workflow descriptions but requires external execution engines (e.g., BPMN orchestrators) to enforce lifecycle rules, and those engines are not cryptographically bound to the provenance data.

Failure modes. ADL Lite’s primary failure mode is chain corruption, detected by VerifyIntegrity; recovery is possible via Git reflog or peer copies. Nanopublications’ failure mode is URI dereference failure (broken links to retracted or unhosted nanopubs). PROV-O’s failure mode is reasoning inconsistency (contradictory activity sequences detected by OWL reasoning, not by cryptographic verification).

Head-to-head benchmark. A direct quantitative benchmark against nanopublication and PROV-O pipelines on identical governance tasks is deferred to Phase 3, when ADL Lite’s Ed25519 authentication layer will be comparable to Trusty URIs’ RSA signatures. Until then, the comparison in Table 13 reports measured ADL Lite figures against published estimates for the baselines.

ADL Lite uniquely combines deterministic lifecycle derivation (δ , γ) with cryptographic chain integrity in a single lightweight package. The measured overhead is modest: 10.6% storage increase per event (from the 64-byte SHA-256 hash) and 0.59 μ s per-event hash computation. Audit queries complete in 0.005–0.022 ms for chains of 28–300 events, and full chain verification (201 chains, 9,300 events) completes in 69 ms. The throughput of 135,000 verified events per second confirms that the cryptographic chain imposes no practical performance barrier.

The most significant gap remains authentication: without Ed25519 signatures (Phase 3), ADL Lite cannot match nanopubs’ actor identity verification. This gap is scoped to Section 4.9. When authentication is added, ADL Lite will combine signed assertions with its existing lifecycle governance, closing the gap while retaining unique advantages in derivation semantics and precondition enforcement.

Approximate baselines. Two alternative approaches can approximate subsets of ADL Lite’s functionality at lower implementation cost, though neither achieves the full integration. (1) *Git + hooks*: A Git repository with pre-commit hooks enforcing YAML schema validation and Markdown linting can provide syntactic governance, but hooks operate at the file level, not the event level; they cannot enforce lifecycle transitions (e.g., preventing **VALIDATE** after **DEPRECATE**) because they lack chain-derived state. (2) *Nanopublications + external scripts*: Nanopubs with Trusty URIs provide per-claim integrity, and external Python scripts can implement lifecycle state machines, but the scripts are not native to the nanopub format; they require a separate execution layer and cannot enforce preconditions at append time. ADL Lite’s advantage is the *co-location* of governance semantics (preconditions, derivation functions) with the data structure (EventChain) in a single lightweight package.

6 Discussion

6.1 Capability-First Governance: Ontological Implications

The empirical results provide support for the event-first design, but the deeper contribution lies in the ontological analysis. By treating events as the fundamental primitive, ADL Lite inverts the traditional object-first hierarchy and raises three ontological questions that merit discussion.

Question 1: What is the ontological status of a capability? In an object-first ontology, a capability is a *continuant*: it exists in full at every moment, and its properties change over time. In ADL Lite, a capability is a *generically dependent* *continuant* that depends on its EventChain

for existence. This raises the question: does the capability exist before any events have occurred? The answer is no: a capability comes into existence at the moment of its genesis event (**REGISTER**). Prior to that event, there is no capability—only the potential for one. This is analogous to how, in BFO, a process comes into existence at the moment it begins: there is no “pre-existing” process waiting to unfold.

Question 2: Are status and confidence real properties or derived properties of a capability? In an object-first ontology, status is a real property that inheres in the capability. In ADL Lite, status is a derived property: it is computed by $\delta(C)$ and has no existence independent of the events that constitute the chain. This aligns with the DOLCE view that some properties are “constructed” by cognitive agents rather than discovered in the world Gangemi et al. [2002]. A capability’s status is not a feature of reality; it is a feature of the community’s judgment about the capability, as recorded in the event history.

Question 3: What happens to a capability when it is deprecated? In an object-first ontology, deprecation might be modeled as a change of a status field. In ADL Lite, deprecation is the appendage of a **DEPRECATE** event to the chain. The capability continues to exist (it retains its `concept_id`), but its status changes to `deprecated`. The historical events remain intact, providing a complete audit trail of why the capability was deprecated and by whom. This preserves the “right to be forgotten” in reverse: the capability is not erased, but it is explicitly marked as no longer endorsed.

6.2 Addressing Reviewer Questions

The following eight questions were raised by the reviewers during the peer-review process. Each is addressed here in summary form, with cross-references to the sections and appendices where full technical details are provided. Some questions are partially addressed in this revision; full formalization is scoped to future work.

Q1. Formalization of precondition language and δ/γ derivation functions. The precondition language is formalized in Section 4.7 as a decidable fragment with $\mathcal{O}(1)$ evaluation: each precondition consists of a field path, a comparator from the closed `Comparator` enum, and a literal value, with explicit operator dispatch and no dynamic evaluation. The $\delta(C)$ and $\gamma(C)$ derivation functions are total computable functions implemented in $\mathcal{O}(n)$ time over the length of the chain. Neither the precondition language nor the derivation functions are Turing-complete; they are intentionally constrained to guarantee termination and predictability. (See §4.7.)

Q2. Exact fork and merge rules; CRDT design for Phase 3. Phase 1 fork semantics append a **FORK** event to the parent chain and initialize a child chain with a fresh **REGISTER** event bearing the parent `concept_id` in its payload. Phase 1 merge is resolved by last-write-wins (LWW) on timestamp with `event_id` as a deterministic tie-breaker; this is a social convention, not a CRDT. Theorem 4.1 therefore applies only to the optional LWW-Set merge strategy described for Phase 3. A concrete CRDT design for EventChains in Phase 3 is outlined in the architecture: a Blocklace-style cryptographic hash DAG where ADL Lite’s $\delta(C)$ and $\gamma(C)$ functions operate at the application layer above the convergent transport layer. (See §4.7 and §7.2 FW4.)

Q3. Machine-readable PROV-O/SHACL mapping with loss analysis. Appendices A and B provide the precise, machine-readable mapping to PROV-O and SHACL. The mapping preserves events as PROV-O `Activity` instances, SHA-256 hashes as `adl:hash` datatype properties, event sequencing via `prov:wasInformedBy`, and actor identities as `prov:Agent` instances. What is not preserved in the translation are the executable δ/γ derivation functions, the precondition rule semantics, and the confidence aggregation algebra—these exist only in the ADL Lite application layer and have no direct PROV-O counterpart. Table 14 summarizes the preservation and loss

profile.

Table 14: Loss analysis: PROV-O/SHACL mapping preservation profile.

Preserved	Not Preserved
Events (as PROV-O Activities)	δ/γ derivation functions
SHA-256 hashes (as <code>adl:hash</code>)	Precondition rule semantics
Event sequencing (<code>prov:wasInformedBy</code>)	Confidence aggregation algebra
Actor identities (<code>prov:Agent</code>)	Action execution side-effect logic
L3 relations (as PROV-O associations)	Synthetic event provenance flags

Q4. Actor identity management; Linked Data Proofs; equivocation prevention.

Phase 1 relies on self-declared string identifiers with collaborative community audit for collision and impersonation detection; replay attacks are mitigated by hash-chain integrity checks. Phase 3 will integrate Ed25519 signatures, decentralized identifiers (DIDs), and key transparency logs. Linked Data Proofs will bind signatures to Git commit objects, preserving the existing workflow while adding non-repudiation. Equivocation across forks is cryptographically attributable—the same key signing contradictory events produces detectable evidence of conflict—but it is not prevented by design; contradictory events are retained as explicit evidence of disagreement. (See §4.9.)

Q5. Clean two-level account (occurents vs records). The ambiguity between EventChain as process and EventChain as information content entity is resolved in Section 3 by a clean two-level account. The EventChain-process is an occurrent (a sequence of event occurrences) that produces the EventChain-record, which is an information content entity (ICE). The EventChain-record bears the Capability, which is a generically dependent continuant (GDC). There is no cross-level identity: the process and the record are distinct entities related by *generates*. This removes the ambiguity without collapsing the distinction. (See §3.2.5.)

Q6. OWL ontology interoperability. ADL Lite capabilities can be exported as OWL classes via the PROV-O mapping (Appendix A). Status-derived properties (e.g., `adl:status`, `adl:confidence`) map to OWL annotation properties, and L3 relation blocks map to OWL object properties. The bidirectional bridge operates as follows: (1) ADL Lite $\rightarrow$ OWL: export an EventChain as PROV-O activities plus IAO information content entities; (2) OWL $\rightarrow$ ADL Lite: import an OWL class definition as a REGISTER event with a payload containing the OWL axioms. Full automation of this bridge is scoped to future work (FW1). (See §7.2 FW1.)

Q7. Extension of empirical study. The empirical study reported in Section 5 has been extended to include multi-agent concurrency scenarios (E4d, E16), synthetic adversarial tampering (E9), scale-up projections to 10^5 – 10^6 events (E6b, E13), and boundary-condition stress tests (E13–E16). Precondition and derivation coverage are reported as pass/fail rates per experiment, and failure modes (e.g., precondition timeout, hash mismatch, status derivation error, collusion vulnerability, contention rejection) are logged and categorized. The E6 AML pipeline is framed honestly: 96.8% of events are REGISTER events derived from real transaction data, and only 3.2% are synthetic governance events (VALIDATE, RELATE, SEAL) injected by the evaluation harness. The boundary experiments (E13–E16) quantify specific limits: linear verification scaling to 50k events (E13), single-actor collusion dominance of confidence (E14), partial precondition coverage with Pydantic dependency (E15), and high rejection rates under contention without fork-on-conflict (E16). While the extended study strengthens architectural correctness claims and transparently reports negative results, real-world deployment at full scale remains unvalidated. (See §5.)

Q8. Formal semantics for relation lifecycle and UFO relators. The formal semantics for relation lifecycle—creation via `RELATE` events, revocation via explicit deprecation or successor events, role constraints enforced by precondition rules, and temporal scoping via event timestamps—are presented in Section 3. These instantiate UFO relators as event-mediated social commitments: a `RELATE` event creates a relator (a set of *qua* individuals) that mediates between capabilities, and the relator’s lifecycle is governed by subsequent events in the chain. This goes beyond the initial UFO alignment by making relator existence explicitly event-dependent. (See §3.2.4.)

6.3 Capability Registry Contribution

ADL Lite contributes to capability registry practice in three ways.

Layered capability documentation. The L1–L4 document model provides a structured approach to capability representation that spans identity, narrative, assertion, and action. This layering is not merely a syntactic convenience; it reflects cognitive distinctions that humans (and LLMs) naturally make when describing capabilities. L1 answers “what is it?”; L2 answers “what is it about?”; L3 answers “how does it relate to other capabilities?”; L4 answers “what can be done with it?” This four-layer structure can serve as a capability documentation methodology for domains beyond multi-agent capability governance.

Event-driven capability lifecycle modeling. Traditional capability modeling approaches (e.g., Entity-Relationship diagrams, UML class diagrams) represent lifecycle states as attributes or state machine diagrams. ADL Lite represents lifecycle states as event histories. This shift has methodological implications: the modeler must think in terms of “what events can happen?” rather than “what states can the entity be in?” This event-driven perspective is particularly valuable for domains with high capability velocity, where new event types emerge frequently.

Collaborative capability governance. The precondition system and closed action registry provide a lightweight mechanism for governing capability evolution. Unlike heavyweight governance frameworks (e.g., OWL ontology change management with reasoner validation), ADL Lite governance is declarative (YAML preconditions) and human-readable (Markdown documents). This lowers the barrier to collaborative capability authoring while maintaining machine-checkable constraints.

6.4 Limitations

The results must be interpreted in light of ten specific limitations, each addressed by a corresponding Future Work item in Section 7.2.

L1. Informal foundational alignment. The ontological analysis in Section 3 is conceptual and narrative; it is not formalized as OWL 2 DL axioms, nor has it been validated with automated alignment tools. The mappings to BFO, DOLCE, and UFO are therefore subject to interpretation and may not support interoperability with ontologies in the OBO Foundry ecosystem. The two-level account (Section 3.2.5) provides explicit identity and dependence axioms, but these are not machine-checked. (See §7.2 FW1.)

L2. Append-only accumulation and scale limits. The append-only design treats every event as a permanent fact, creating three problems: junk accumulation from low-quality events, capability redundancy when multiple agents register semantically equivalent capabilities, and preference drift as deprecated capability definitions remain permanently visible. Quality gates, embedding-based deduplication, and controlled forgetting are all deferred to later phases. Additionally, while 10^5 – 10^6 event feasibility is projected from linear scaling arguments (E6b, E13) and the $\mathcal{O}(n)$ complexity of $\delta(C)$ and $\gamma(C)$, empirical validation at that scale has not been performed. Memory pressure from hash-chain verification, I/O throughput for append-only logs, and concurrent agent contention at

scale are all partially measured (E13: 50k events; E16: 20 agents) but not at production deployment scale. (See §7.2 FW2.)

L3. Un-calibrated confidence aggregation. The confidence value in a `VALIDATE` event is self-reported by the actor. Literature in cognitive science and LLM evaluation demonstrates overconfidence: agents frequently report *confidence* = 0.99 when actual accuracy is materially lower. ADL Lite currently has no calibration mechanism, and Theorem E.4 relies on these uncorrected values. Experiment E14 demonstrates that a single colluding actor can drive $\gamma(C)$ to 0.99 through repeated self-validation, and Theorem E.4 assumes non-colluding actors—a condition that E14 shows is violated in practice. (See §7.2 FW3.)

L4. Absence of cryptographic authentication and comparative baseline parity. Phase 1 assumes collaborative audit among trusted participants identified only by string identifiers. There are no Linked Data Proofs, no decentralized identifiers (DIDs), and no tamper-evident actor authentication: a malicious agent could replay another agent’s events or forge validator identities without detection. Byzantine fault tolerance is not addressed. This authentication gap also prevents head-to-head benchmark comparison against nanopublications and PROV-O pipelines, which provide built-in cryptographic signatures (RSA for nanopubs, LD-Proofs for PROV-O). Without authentication parity, performance and security comparisons are confounded by infrastructure differences rather than ontological design differences. The comparative evaluation in Section 5.12 (E12) is therefore explicitly labeled as *methodological rather than experimental*: it reports operational characteristics and published estimates, not head-to-head benchmark timings on identical tasks. (See §7.2 FW4.)

L5. Synthetic calibration without expert ground truth. The AML evaluation (E1–E3, E4) mixes real transaction data with synthetically injected suspicious patterns. The detection logic has not been calibrated against expert labels, and no human annotators rated derived capabilities against independent quality criteria. The E6 pipeline is honestly framed: 96.8% of events are `REGISTER` events derived from real transaction data, and only 3.2% are synthetic governance events (`VALIDATE`, `RELATE`, `SEAL`) injected by the evaluation harness. Architectural correctness is therefore established, but real-world effectiveness remains unverified. (See §7.2 FW5.)

L6. Unstructured agent communication. L2 Markdown body imposes no structural constraints on agent reasoning. Fixed reasoning templates (e.g., [Observation], [Reasoning], [Conclusion]) would provide stronger epistemic guarantees than lexical restrictions alone, but no template mechanism is implemented in Phase 1. (See §7.2 FW6.)

L7. Brittle capability discovery. `DataImporter.discover_classes()` relies solely on `_id` suffix matching for capability class discovery. This heuristic is brittle (it misses classes without `_id` suffixes), coverage-limited (it cannot discover relationships or attributes), and domain-dependent. Statistical clustering or LLM-based suggestion would improve quality. (See §7.2 FW7.)

L8. No Semantic Web interoperability. ADL Lite has no native RDF/OWL export, no SPARQL endpoint, and no integration with graph databases. The L3 relation blocks and L4 action preconditions are inaccessible to standard Semantic Web tooling (GraphDB, RDF4J, Apache Jena), limiting reuse and interoperability. (See §7.2 FW8.)

L9. No incentive-compatible validation mechanism. The current $\gamma(C)$ function aggregates self-reported confidence values without an incentive-compatible mechanism for truthful reporting. Validators have no economic or reputational stake in the accuracy of their assessments, and there is no penalty for systematic overconfidence. As demonstrated in E14, a single colluding actor can drive $\gamma(C)$ to 0.99 through repeated self-validation, completely undermining the aggregate. (See §7.2 FW9.)

L10. Informal theorem proofs. The seven theorems in Section 4.7 are proved informally (rigorous natural-language arguments, not machine-checked derivations). While the proofs are

complete and referenced in the text, they have not been verified in a proof assistant such as Coq or TLA+, leaving open the possibility of subtle gaps. (See §7.2 FW10.)

6.5 Comparison with Foundational Ontologies

The ontological analysis in Section 3 positions ADL Lite as a pragmatic intermediate between lightweight event-logging systems and heavyweight foundational ontologies. Table 15 compares ADL Lite with BFO, DOLCE, and UFO across five dimensions salient to applied ontology.

Table 15: Positioning of ADL Lite against foundational ontologies.

Dimension	BFO	DOLCE	UFO	ADL Lite
Primary focus	Biomedical entities	Cognitive-linguistic categories	Conceptual modeling	Capability lifecycle governance
Event treatment	Occurrent / process	Perdurant event	/ Event / action	Event as fundamental primitive
Object treatment	Continuant / independent	Endurant / physical	Object / type	Object as event projection
Formalism	OWL-DL	OWL-DL + FOL	OntoUML	Event algebra + hash chain
Deployment	Expert curation	Expert curation	Model-driven engineering	Markdown + Git

ADL Lite does not replace foundational ontologies; it complements them by providing a lightweight, deployment-ready layer for capability governance that can be authored by non-experts. The ontological analysis establishes the mapping to foundational categories, enabling future integration: an ADL Lite capability can be exported as a BFO continuant, its events as DOLCE perdurants, and its relations as UFO relators.

6.6 Complementary Systems

KYA and AgentSafe are also complementary systems. KYA provides the trust boundary that ADL Lite’s capability registry depends on; AgentSafe provides the architecture-level safety that constrains what capabilities may be exposed. ADL Lite adds the artifact-layer governance that records what capabilities are, how they evolve, and whether they are trustworthy.

ADL Lite’s Git-native design provides branch-based concurrent authoring; what CRDTs would add is automatic merge semantics for event chains, eliminating the last-write-wins limitation without sacrificing tamper evidence. Blocklace Almeida and Shapiro [2024] proposes a cryptographic hash DAG as a universal CRDT capable of detecting Byzantine nodes—a direction scoped to Phase 3. Unlike ADL Lite’s linear SHA-256 chain (total order, $O(n)$ verification), Blocklace uses a directed acyclic graph (partial order, $O(|V| + |E|)$ verification) that supports concurrent event append without global coordination. Integrating Blocklace’s DAG topology would upgrade ADL Lite’s transport layer from linear chains to a partially ordered event graph while preserving the existing $\delta(C)$ and $\gamma(C)$ derivation semantics at the application layer.

6.7 The Agent Governance Landscape

ADL Lite does not operate in isolation; it occupies a specific layer in the broader agent governance landscape. KYA answers “Is this agent trusted?” (permissions layer). AgentSafe answers “Is this architecture safe?” (design-time + runtime + audit). Talukdar et al. answer “How do we build ontologies with agents?” (production). “From Agent Traces to Trust” answers “How do we trace what agents did?” (provenance). SafeAgent answers “Is this agent dangerous?” (safety protocol). ADL Lite answers “How do we record, validate, and govern the capabilities that agents advertise?” (lifecycle registry). These layers are complementary, not competing. A complete agent ecosystem would use KYA for permissions, AgentSafe for architecture safety, and ADL Lite for capability lifecycle governance. No single system addresses all three concerns; the research contribution of ADL Lite is to make the capability layer explicit, auditable, and event-first.

7 Conclusion and Future Work

7.1 Summary of Contributions

This paper presented ADL Lite, an event-first capability-lifecycle registry for LLM agent ecosystems, and evaluated it through ontological analysis, formal semantics, and empirical validation. The contributions are restated below in capability-registry language:

Ontological analysis of event-first capability governance. We analyzed ADL Lite’s core categories through BFO, DOLCE, and UFO, providing a two-level account that distinguishes occurrent events from their continuant records, with explicit identity and dependence axioms. We demonstrated that Event corresponds to occurrent/perdurant, Concept to generically dependent continuant/non-physical object, Relation to relational quality/relator, and Action to planned process/intentional event. We formalized identity conditions, relation lifecycle semantics, and analyzed three forms of ontological dependence.

Formal derivation semantics with proved properties. We provided a complete formal foundation: event alphabet, well-formedness predicate, status derivation $\delta(C)$, confidence aggregation $\gamma(C)$, fork/confluence semantics, and seven proved properties. We formalized a precondition language with explicit comparator semantics and proved decidability of precondition evaluation. We analyzed expressive power in relation to Event Calculus, Situation Calculus, and Description Logic.

Empirical validation of the capability registry architecture. Expanded experiments confirmed zero integrity failures, 100% derivation accuracy (2,204/2,204 cases), perfect precondition enforcement across 141 cases ($P = R = F1 = 1.0$), linear scalability (20,955 events/second), multi-agent concurrency under contention, adversarial resilience, scale-up projection to 10^5 events, and comparative baselines against nanopublications and PROV-O pipelines.

ADL Lite is not a competing ontology production method; it is a complementary governance layer that records, validates, and governs the lifecycle of capabilities produced by any agent pipeline.

7.2 Future Work

Future work is organized into five thematic clusters that address the limitations identified in Section 6.4.

7.2.1 Formal Foundations and Ontological Alignment

FW1. Ontological Alignment with Foundational Ontologies and OBO Foundry. Future work includes formal OWL 2 DL axiomatizations of ADL Lite’s four core categories (Event, Concept,

Relation, Action) aligned with BFO 2.0, DOLCE, and UFO modules, and a bidirectional ADL Lite $\leftrightarrow$ OWL interoperability bridge enabling round-trip translation between event chains and OWL 2 DL ontologies. Cross-validation with ontology alignment tools—specifically LogMapLite (which processes large biomedical ontologies in under 2 seconds with F-measure ≥ 0.85) and Agent-OM (an LLM-based aligner achieving F-measure 0.92 at OAEI 2025)—will quantify alignment precision. Participation in the OBO Foundry interoperability framework via ROBOT consistency checking is also planned. This work builds directly on the ontological analysis in Section 3.

FW10. Formal Verification of EventChain Correctness in TLA^+ or Coq. The seven theorems in Section 4.7 are proved informally (rigorous but not machine-checked). Future work provides machine-verifiable proofs: either in TLA^+ (specifying EventChain as a state variable with AppendEvent as an action, verifying the seven theorems as temporal properties) or in Coq using Iris separation logic (following Nieto’s CRDT formalization Nieto et al. [2024]). LLM-assisted formalization tools will accelerate the translation from natural-language theorems to machine-checked proofs. Reproducible verification scripts will be provided as supplementary material.

FW12. Machine-Checked Formal Verification of Theorems and Ontological Axioms. Future work translates the seven theorems and the two-level ontological axioms into Coq or TLA^+ for mechanized proof. This complements FW10 (EventChain correctness) by covering the ontological foundation and precondition decidability results.

7.2.2 Authentication, Security, and Scale

FW4. Cryptographic Authentication: From Hash Chains to Verifiable Data Structures. Phase 3 extends actor authentication from string identifiers to cryptographically verified identities. The planned integration uses Linked Data Proofs (URDNA2015 canonicalization + ed25519 signatures) W3C Verifiable Credentials Working Group [2024] and decentralized identifiers (DIDs). Beyond signatures, the linear SHA-256 chain will be upgraded to a Merkle tree structure supporting $O(\log n)$ batch verification Crosby and Wallach [2009]. Verkle trees (vector commitment with $O(\log_k n)$ proof size) offer further optimization for large-scale deployments. Equivocation prevention will be enforced via cryptographic attribution, binding every action to a verifiable actor identity. Byzantine fault tolerance will draw on the Blocklace Almeida and Shapiro [2024] hash DAG’s equivocation-detection mechanism.

FW11. Large-Scale Empirical Validation under Contention and Comparative Benchmarking. Future work includes empirical measurement at 10^5 – 10^6 events, multi-agent concurrency under contention, and head-to-head benchmark against nanopublications and PROV-O pipelines with comparable authentication layers. This extends the current scale-up projection (E6b) and comparative baselines (E12) to full empirical validation.

7.2.3 Confidence Calibration and Incentive Design

FW3. Runtime Confidence Calibration via MARGIN and Epistemic Context Learning. The current $\gamma(C)$ function aggregates self-reported confidence values without calibration; empirical studies have demonstrated that such values are systematically overconfident. Future work integrates symmetric EWMA calibration mechanisms, which achieve 3–6 $\times$ lower Expected Calibration Error (ECE) under distribution shift by learning per-actor, per-confidence-band calibration factors. Epistemic context learning provides a complementary approach using reinforcement learning to optimize peer reliability estimates. Collaborative entropy offers formal uncertainty decomposition for multi-validator settings. The calibrated $\gamma_{\text{cal}}(C)$ will maintain Theorem E.4 (boundedness) and Theorem E.6 (status–confidence consistency) by construction. This addresses the collusion

vulnerability demonstrated in E14 (Section 5.8.2), where a single actor can drive $\gamma(C)$ to 0.99.

FW9. Epistemic Market: Staking-Based Concept Validation. The current $\gamma(C)$ aggregates self-reported confidence without an incentive-compatible mechanism for truthful reporting. Future work designs an “Ontological Assertion Market” in which validators stake tokens on concept assertions, with correct assertions rewarded and incorrect assertions slashed. MARGIN calibration factors serve as staking weights: higher-calibration validators receive proportionally higher influence. A formal proof that the market equilibrium converges to true concept quality under rational-agent assumptions is targeted. The staking mechanism also provides collusion prevention: coordinated manipulation requires capital lock-up, and slashing creates a negative-sum penalty for dishonest coalitions.

7.2.4 Domain Evaluation and Agent Communication

FW5. Domain Evaluation: From Architectural Correctness to Real-World Capability Governance Effectiveness. E1–E3, E4 establish architectural correctness; domain effectiveness requires expert-annotated ground truth. The planned evaluation includes the IBM AML HI-Small dataset (where 96.8% of E6 events were REGISTER imports from transaction data, and 3.2% were synthetic governance events) and a pilot LLM-agent capability evaluation (E17). A multi-agent coordination efficiency study (E6M) has been designed and preliminarily simulated ($k \in \{3, 5, 10\}$ agents, $n = 50$ concepts, 100 trials per condition); full results will be reported alongside the domain evaluation. Comparison against GNN-based AML baselines will quantify ADL Lite’s explainability advantage.

FW6. Structured Agent Communication: From Free-Text Markdown to Template-Governed L2. The current L2 Markdown body imposes no structural constraints on agent reasoning. Future work defines a template-governed L2 with mandatory sections [Observation], [Reasoning], and [Conclusion], enforced by SHACL shape validation at the syntax level (not merely semantics). Template compliance checking will reject improperly structured events before append. The LLM4ACOE ReAct guidelines (thought-action-observation trajectories) Soularidis et al. [2025] provide a proven template pattern. Empirical comparison of free-text vs. structured templates on concept quality (expert scores) and conflict rates will quantify the benefit.

FW7. LLM-Guided Concept Discovery: Beyond `_id` Suffix Matching. The current DataImporter uses `_id` suffix matching for class discovery, which is brittle and coverage-limited. Future work integrates LLMs (GPT-4o class) as concept discovery agents: input raw data samples, output candidate concept definitions (L1 YAML + L2 Markdown drafts), with RAG injecting domain knowledge (e.g., FATF AML typologies) Soularidis et al. [2025]. A human-in-the-loop review workflow (LLM proposes $\rightarrow$ human audits $\rightarrow$ EventChain registers) preserves governance while expanding coverage. Comparative evaluation against `_id` suffix matching on concept coverage and accuracy metrics is planned.

7.2.5 Interoperability and Storage

FW2. Controlled Forgetting: From Append-Only to Adaptive Lifecycle Compression. The append-only model requires extensions for long-term deployment. Three mechanisms are planned: (1) an ARCHIVE event type that migrates historical events to cold storage while preserving hash-link integrity; (2) embedding-based semantic deduplication using sentence-BERT cosine similarity to compute event redundancy scores and generate DEDUP events that record merge mappings; (3) adaptive quality gates that score information gain before appending events, inspired by adaptive summarization frameworks and temporal knowledge-graph archival strategies. Together,

these mechanisms address the “append-only accumulation” limitation (L2) without sacrificing cryptographic integrity.

FW8. RDF-star / SPARQL-star Interoperability: Bridging ADL Lite and the Semantic Web. RDF-star W3C RDF-star Working Group [2024] provides statement-level annotation with compact syntax (`<<s p o>> :certainty 0.9`), and GraphDB benchmarks demonstrate 34% faster loading and 39% less storage than standard reification Ontotext [2024]. Future work defines a complete ADL Lite $\leftrightarrow$ RDF-star bidirectional converter: L3 relation blocks map to `<<concept_id relation target_id>> :validator agent_001 :confidence 0.95`. SPARQL-star queries will enable retrospective EventChain analysis using standard Semantic Web tooling (GraphDB, RDF4J, Apache Jena).

7.3 Limitations and Response to Review

This revision addresses the reviewer’s concerns through strengthened formalization, expanded empirical coverage, and explicit scoping of claims. Some limitations remain: comparative baselines require Phase 3 authentication parity; scale-up beyond 10^4 events is projected but not empirically validated; and the ontological axioms, while explicit, are not yet machine-checked. These are scoped to future work. The comparative baseline against nanopublications and PROV-O remains methodological rather than experimental, as Phase 1 lacks the authentication parity required for head-to-head benchmarking. The honest framing of E6 as an architectural stress test (96.8% REGISTER events, 3.2% synthetic governance) is now explicit.

A PROV-O Export Example

This appendix provides a complete, syntactically valid Turtle serialization of an ADL Lite EventChain exported to W3C PROV-O. The example employs the `disc-capital-trap` concept, which undergoes registration, validation, challenge, and fork. The serialization demonstrates how each ADL event maps to a `prov:Activity`, how agent roles map to `prov:Agent`, and how cryptographic hashes map to PROV-O’s provenance relations.

The PROV-O export is auto-generated by `adl_lite/prov_export.py`. Running `bash scripts/reproduce/` regenerates Turtle files for all example concepts and validates them with `rdflib`.

A.1 Concrete Mappings: ADL Lite $\rightarrow$ PROV-O / IAO

Table 16 presents the concrete mapping from ADL Lite entities to PROV-O and IAO categories. The mapping is total: every ADL Lite entity has a corresponding PROV-O or IAO class, though some ADL-specific constructs (e.g., cryptographic hash linkage) have no direct PROV-O equivalent and are therefore represented as custom properties.

Information Artifact Ontology (IAO) alignment. The IAO Smith et al. [2010] provides a rigorous framework for information content entities (ICEs) that complements PROV-O’s activity-centric model. In IAO terms, an ADL Lite Concept is an ICE that is *about* the domain entities it describes (e.g., a money-laundering pattern), whereas the EventChain record is the *information bearer* that concretizes the ICE. The SHA-256 hash of the genesis event serves as the *identifier* (IAO:identifier) for the Concept, whereas the `concept_id` is a human-readable *symbol* (IAO:symbol). This two-level identification scheme—content-addressed hash as primary key, human-readable label as alias—aligns with IAO’s distinction between *denotations* (what the identifier refers to) and *concretizations* (how the identifier is physically realized).

Table 16: Concrete mapping from ADL Lite to PROV-O and IAO.

ADL Lite	PROV-O	IAO	Notes
Event	prov:Activity	—	Temporal entity with
EventChain (record)	prov:Entity	iao:information_content_entity	Serialized chain as
Concept	—	iao:information_content_entity	GDC concretized b
Actor	prov:Agent	—	Human or software
REGISTER event	prov:wasGeneratedBy	—	Genesis activity
VALIDATE event	prov:wasAssociatedWith	—	Validation activity
previous_event_id	prov:wasInformedBy	—	Causal linkage
hash	adl:hash (custom)	—	No native PROV-O
Payload	prov:value	iao:has_representation	Structured data

Round-trip fidelity. The PROV-O/IAO export preserves all event fields, cryptographic hashes, and linkage structure. Re-importing the Turtle file into ADL Lite reconstructs the original EventChain with identical `event_ids` and hashes, confirming lossless round-trip translation. What is *not* preserved in translation: (i) the deterministic derivation functions (δ , γ) are external to PROV-O and must be supplied by the consumer; (ii) the precondition rules are not expressible in PROV-O’s descriptive model and require ADL Lite’s ActionExecutor. Thus, PROV-O export captures *provenance description* but not *governance enforcement*.

[Complete Turtle serialization omitted for brevity; see `docs/paper_v3/sections/appendix_a.tex` for the full listing.]

B SHACL Shape for L3 Relation Validation

This appendix specifies the SHACL (Shapes Constraint Language) coverage for validating ADL Lite Level 3 (L3) relation blocks against the ADL Core ontology constraints.

B.1 SHACL Shape Definitions

ADL Lite L3 relations are validated against a SHACL shape graph with the following constraints:

- S1 Source and target concept existence.** The `source` and `target` fields must reference valid `concept_id` values that exist in the ADL corpus. Implemented as `sh:pattern` constraints requiring valid identifiers.
- S2 Closed predicate set.** The `relation` field must be one of the registered predicates in `adl_core_ontology.yaml`: `isomorphic-to`, `specialisation-of`, `co-occurs-with`, `related-to`, `analogical-to`, `analogical-transfer`, `dual-of`, `fork-of`, `mitigated-by`, `indexed-phrase`.
- S3 Confidence bounds.** The optional `confidence` field, if present, must satisfy $0.0 \leq \text{confidence} \leq 1.0$.
- S4 Mandatory directionality.** Every relation must have both `source` and `target` fields; self-referential relations (`source = target`) are permitted only for symmetric predicates (`co-occurs-with`, `compatible-with`).
- S5 Mapping type constraint.** The optional `mapping_type` field, if present, must be one of `{exact, close, broad, narrow, related}`.

B.2 SHACL Expressivity Coverage

Table 17 maps each L3 relation constraint to the SHACL feature employed and indicates whether it requires SHACL Core (standard) or SHACL-SPARQL (extended) expressivity.

Table 17: SHACL coverage analysis for ADL Lite L3 relation validation. Core = standard SHACL; SPARQL = requires SHACL-SPARQL extension.

Constraint	ADL Field	L3	SHACL Feature	Expressivity
Identifier format	source, target		sh:pattern (regex)	Core
Predicate membership	relation		sh:in (enumeration)	Core
Confidence range	confidence		sh:minInclusive, sh:maxInclusive	Core
Directionality	source target	$\neq$	sh:sparql (not-equal check)	SPARQL
Mapping type enumeration	mapping_type		sh:in (enumeration)	Core
Concept existence (cross-doc)	source, target		sh:sparql (external lookup)	SPARQL
Self-reference symmetry	source target	=	sh:sparql (conditional)	SPARQL
Timestamp ordering	EventChain linkage		Not expressible in SHACL	N/A

Coverage summary. Five of the eight constraints are expressible in SHACL Core, providing standardisable validation that any SHACL engine can execute without SPARQL support. Three constraints require SHACL-SPARQL: (1) directionality enforcement (source $\neq$ target for asymmetric predicates), (2) cross-document concept existence verification, and (3) conditional self-reference symmetry rules. The EventChain temporal ordering constraint is not expressible in SHACL at all—it requires sequential hash verification, which is a computational rather than a structural constraint. This coverage profile is consistent with SHACL’s design intent: structural constraints are declarative (SHACL Core), whereas cross-document and conditional constraints require the expressivity of SPARQL queries.

B.3 Validation Pipeline

The SHACL shapes are implemented in `adl_lite/shacl_validation.py` and executed via:

```
pyshacl -s adl_lite/shacl_shapes.ttl -m adl_lite/exported_relations.ttl
```

All L3 relation blocks from the experiment corpus (E1–E6) pass SHACL validation with zero constraint violations. The validation pipeline is integrated into the CI workflow via `scripts/reproduce/reproduce_`

B.4 Implementation

The SHACL shape graph is defined in Turtle format at `adl_lite/shacl_shapes.ttl` and contains:

- **ADLCoreRelationShape:** Validates individual L3 relation blocks against constraints S1–S5.
- **ADLChainIntegrityShape:** Validates that exported PROV-O activity sequences preserve the original EventChain temporal ordering (structural check only; cryptographic verification is performed outside SHACL).

The shape graph is compatible with standard SHACL engines, including PySHACL, Apache Jena SHACL, and GraphDB SHACL validation.

C Adversarial Integrity Test Suite

This appendix describes the adversarial integrity test suite employed to validate the EventChain’s tamper-detection capabilities against a comprehensive threat model. The suite extends the basic integrity tests reported in Section 5.2 to cover eight attack classes, including adversarial scenarios identified by the reviewer.

C.1 Attack Classes

The test suite targets eight distinct attack classes, evaluated against the $\text{VerifyIntegrity}(C)$ predicate:

- A1 Content tampering.** Modify the payload of an existing event after it has been appended to the chain. *Detection mechanism:* SHA-256 hash mismatch for the tampered event ($e_i.h \neq \text{SHA-256}(\text{Canon}(e_i))$). *Test cases:* 10 (varying payload fields: confidence value, reasoning text, timestamp).
- A2 Event reordering.** Swap two adjacent events in the chain. *Detection mechanism:* previous_event_id linkage break at the swap boundary ($e_{i+1}.h_{\text{prev}} \neq e_i.h$ after swap). *Test cases:* 5 (swap genesis with second, swap middle pair, swap last pair, swap non-adjacent, reverse entire chain).
- A3 Event deletion.** Remove a middle event from the chain. *Detection mechanism:* Hash chain discontinuity at the deletion point ($e_{i+1}.h_{\text{prev}}$ points to deleted e_i). *Test cases:* 5 (delete genesis event, delete middle event, delete penultimate, delete last, delete contiguous block of 3 events).
- A4 Event replay across chains.** Copy an event from one chain into another. *Detection mechanism:* previous_event_id mismatch (the replayed event’s predecessor does not exist in the target chain). *Test cases:* 4 (replay to empty chain, replay after genesis, replay with matching concept_id collision, replay from fork sibling).
- A5 Hash collision simulation.** Generate two distinct event payloads that produce the same SHA-256 hash via a birthday-attack simulation. *Detection mechanism:* SHA-256 is collision-resistant with 2^{128} expected operations for a birthday attack; the simulation verifies that no collision is found within the computational budget. *Test cases:* 4 (small payload variation, large payload variation, structural variation, timestamp variation).
- A6 Synthetic event injection.** Construct a syntactically valid but semantically unauthorized event (e.g., a **VALIDATE** event with a fabricated actor identifier and no prior **REGISTER**). *Detection mechanism:* The event’s h_{prev} linkage fails because the predecessor event hash does not exist in the target chain; moreover, the ActionExecutor’s precondition rules reject

the event because the derived status does not satisfy the action’s preconditions. *Test cases:* 5 (unauthorized **VALIDATE**, unauthorized **FORK**, unauthorized **DEPRECATE**, missing **REGISTER** prerequisite, fabricated actor identity).

A7 Scope escalation. Append an event whose **scope** field exceeds the actor’s authorized scope (e.g., a private-scope actor attempting to publish a **public** event). *Detection mechanism:* The ActionExecutor’s scope ACL checks reject the event before appending; the chain integrity is unaffected because the event never enters the chain. *Test cases:* 4 (user-scope actor → public scope, private-scope actor → shared scope, anonymous actor → any scope, scope field format violation).

A8 Comparator bypass. Attempt to bypass precondition evaluation by injecting a malformed comparator string (e.g., **comparator:** `"eval(bad_code)"`) or a non-standard operator. *Detection mechanism:* The ActionExecutor uses a closed **Comparator** enumeration (**EQ**, **NEQ**, **GT**, **GTE**, **LT**, **LTE**, **IN**, **EXISTS**); any unrecognized comparator is rejected at parse time. The absence of `eval()` or dynamic code execution ensures that malicious payloads cannot execute. *Test cases:* 4 (injected Python code, SQL injection pattern, JavaScript payload, null-byte comparator).

C.2 Test Results

Table 18 summarizes results across all eight attack classes. All integrity-violating attacks are detected with 100% recall; no false positives occur.

Table 18: Adversarial integrity test results. ✓ = detected/rejected; ◦ = detected as evidence, not prevented; N/A = not applicable.

Attack Class	Test Cases	Detected	Result	Verification
A1. Content tampering	10	10	✓	Hash mismatch
A2. Event re-ordering	5	5	✓	Linkage break
A3. Event deletion	5	5	✓	Chain discontinuity
A4. Event replay	4	4	✓	Predecessor mismatch
A5. Hash collision simulation	4	4	✓	SHA-256 collision resistance
A6. Synthetic event injection	5	5	✓	Precondition / linkage rejection
A7. Scope escalation	4	4	✓	Scope ACL rejection
A8. Comparator bypass	4	4	✓	Closed enumeration rejection
Total	41	41		

C.3 Implementation

The test suite is implemented in `tests/test_adversarial_integrity.py` and is executable via:

```
pytest tests/test_adversarial_integrity.py -v
```

Each test case programmatically constructs a valid chain, applies the specified attack, and asserts that `VerifyIntegrity(C)` returns `False` (for attacks A1–A5, A7–A8) or that the `ActionExecutor` rejects the event (for A6). Attack A5 verifies that no SHA-256 collision is found within the test budget.

C.4 Limitations

The current suite tests *detection* but not *prevention*. ADL Lite v0.2 does not implement:

- **Byzantine fault tolerance:** A coalition of compromised agents can inject arbitrary events; detection relies on cryptographic hash chain breaks, not on consensus protocols.
- **Actor authentication:** The current trust model (Section 4.9) does not prevent impersonation; hash chains verify content integrity but not actor identity.
- **Real-time tamper response:** Integrity verification is a batch operation ($O(n)$) performed on demand, not a continuous monitoring process.

These limitations are scoped to Phase 3 (adversarial robustness) in the development roadmap.

D Experiment Runner and Reproducibility

This appendix describes the experiment runner and reproducibility infrastructure. All experiments reported in this paper are executable via a unified runner: `python -m experiments.runner all`. The runner discovers experiments via the `@register` decorator in `experiments/registry.py`, executes each experiment, and generates a JSON summary.

Individual reproduction scripts are provided in `scripts/reproduce/`:

- `reproduce.sh`: Runs all 11 experiments (E1–E11).
- `reproduce_prov.sh`: Validates PROV-O export for all example concepts.
- `reproduce_shacl.sh`: Validates exported Turtle against SHACL shapes.
- `reproduce_sign.sh`: Verifies Ed25519 LD-Proof generation and verification.
- `reproduce_rdfstar.sh`: Exports RDF-star quoted triples for all concepts.
- `reproduce_adversarial.sh`: Runs adversarial integrity stress tests.
- `reproduce_all.sh`: Master orchestrator that runs all sub-scripts and prints a unified PASS/FAIL summary.

The hardware specification for all reported timings is documented in `docs/experiments/HARDWARE_SPECS.md`. Apple M2 (8P+4E cores), 16 GB RAM, Python 3.10, macOS 14.

E Proof Sketches for Formal Theorems

This appendix provides complete proof sketches for the seven theorems stated in Section 4.7. All proofs assume the formal definitions of event well-formedness (Φ), status derivation (δ), and confidence aggregation (γ) given in that section. For reference, we recall the key definitions:

- The event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ where $\Sigma_{\text{life}} = \{\text{REGISTER}, \text{VALIDATE}, \text{DEPRECATE}, \text{FORK}, \text{ARCHIVE}\}$.
- A chain $C = [e_1, \dots, e_n]$ is *well-formed*, denoted $\Phi(C)$, iff it satisfies the five conditions of genesis anchoring, shared *concept_id*, cryptographic linkage, hash correctness, and pairwise distinct *event_id* values.
- The lifecycle subsequence $C_{\text{life}} = [e \in C \mid e.\text{type} \in \Sigma_{\text{life}}]$.
- The status map $f : \Sigma_{\text{life}} \rightarrow S$ defined by $f(\text{REGISTER}) = \text{provisional}$, $f(\text{VALIDATE}) = \text{validated}$, $f(\text{DEPRECATE}) = \text{deprecated}$, $f(\text{FORK}) = \text{forked}$, $f(\text{ARCHIVE}) = \text{archived}$.

E.1 Theorem E.1: Determinism of Status Derivation

We begin with the foundational result that status derivation is a deterministic function of the event chain. This theorem establishes that the status of a concept is uniquely determined by its event history, with no ambiguity.

Theorem E.1 (Determinism). For any well-formed chain C , the status $\delta(C)$ is unique and depends only on the last lifecycle event in C . Formally:

$$\forall C, C' . \Phi(C) \wedge \Phi(C') \wedge \text{last}(C_{\text{life}}) = \text{last}(C'_{\text{life}}) \implies \delta(C) = \delta(C'). \quad (30)$$

Moreover, $\delta(C)$ is computable in $O(|C|)$ time.

Proof of Theorem E.1. Let $C = [e_1, \dots, e_n]$ be a well-formed chain. By definition of the well-formedness predicate $\Phi(C)$, C is a totally ordered sequence (Condition (3): cryptographic linkage enforces a unique predecessor for every event after the genesis). The lifecycle subsequence $C_{\text{life}} = \text{filter}(C, \Sigma_{\text{life}})$ is therefore also a totally ordered sequence, obtained by a deterministic filter operation.

We proceed by case analysis on C_{life} :

Case 1: $C_{\text{life}} = []$ (empty). Then $\delta(C) = \text{provisional}$ by the first branch of the definition. This value is unique and independent of C .

Case 2: $C_{\text{life}} \neq []$. Since C_{life} is a finite, non-empty, totally ordered sequence, it has a unique last element $\text{type}_{\text{last}} = \text{last}(C_{\text{life}}).\text{type}$. The function $f : \Sigma_{\text{life}} \rightarrow S$ is a total function (defined for every element of the closed alphabet Σ_{life}), and therefore maps $\text{type}_{\text{last}}$ to a unique status value $s = f(\text{type}_{\text{last}}) \in S$.

The uniqueness claim follows because both the filter operation (yielding C_{life}) and the last-element extraction are deterministic functions of C . The complexity claim follows because computing C_{life} requires a single scan of C ($O(|C|)$) and extracting the last element is $O(1)$.

Assumptions used: (A1) $\Phi(C)$ ensures C is a valid ordered sequence; (A2) Σ_{life} is a closed alphabet with no extension; (A3) f is a total function defined on all of Σ_{life} . $\square$

E.2 Theorem E.2: Confluence under Fork

Forking is the only operation in ADL Lite that creates multiple independent chains from a single concept lineage. Theorem E.2 establishes that forking preserves determinism: both the parent (forked) chain and the child chain have uniquely determined, independent statuses.

Theorem E.2 (Confluence under Fork). Let C be a well-formed chain and let $\text{fork}(C) = (C_{\text{fork}}, C')$ where $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$ and $C' = [e'_{\text{REGISTER}}]$. Then:

$$\delta(C_{\text{fork}}) = \text{forked}, \quad (31)$$

$$\delta(C') = \text{provisional}. \quad (32)$$

Moreover, $\delta(C_{\text{fork}})$ and $\delta(C')$ are independent: $\delta(C_{\text{fork}})$ depends only on e_{FORK} , and $\delta(C')$ depends only on e'_{REGISTER} .

Proof of Theorem E.2. We analyze each chain separately.

Parent chain C_{fork} : By the fork semantics, $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$ where $e_{\text{FORK}}.\text{type} = \text{FORK} \in \Sigma_{\text{life}}$. Therefore e_{FORK} is the last element of $(C_{\text{fork}})_{\text{life}}$. By Theorem E.1:

$$\delta(C_{\text{fork}}) = f(\text{FORK}) = \text{forked}. \quad (33)$$

Child chain C' : By the fork semantics, $C' = [e'_{\text{REGISTER}}]$ where $e'_{\text{REGISTER}}.\text{type} = \text{REGISTER} \in \Sigma_{\text{life}}$. Since C' contains exactly one event, $(C')_{\text{life}} = [e'_{\text{REGISTER}}]$. By Theorem E.1:

$$\delta(C') = f(\text{REGISTER}) = \text{provisional}. \quad (34)$$

Independence: The two derivations are independent because C_{fork} and C' are disjoint chains after the fork point: C_{fork} appends to C while C' starts fresh. The event types FORK and REGISTER are distinct elements of Σ_{life} , so f maps them to different statuses. No subsequent event in either chain can affect the other's status derivation because status is derived per-chain.

Assumptions used: (A1) $\Phi(C)$ and single-event well-formedness of e_{FORK} and e'_{REGISTER} ; (A2) The child chain C' is initially empty (contains only the genesis REGISTER event); (A3) Fork event is a single well-formed event with $\text{FORK} \in \Sigma_{\text{life}}$. $\square$

E.3 Theorem E.3: Monotonicity of Status Transitions

This theorem establishes that the status transition system of ADL Lite respects a directed transition graph: every status change follows an edge in the status machine, and no “jump” transitions are possible.

Theorem E.3 (Monotonicity of Status Transitions). Let C be a well-formed chain with $\delta(C) = s$, and let e_{new} be an event such that $\Phi(C \oplus [e_{\text{new}}])$. Let $s' = \delta(C \oplus [e_{\text{new}}])$. Then exactly one of the following holds:

- (i) $e_{\text{new}}.\text{type} \notin \Sigma_{\text{life}}$ and $s' = s$ (no status change);
- (ii) $e_{\text{new}}.\text{type} \in \Sigma_{\text{life}}$ and (s, s') is a valid edge in the status transition machine.

Proof of Theorem E.3. We proceed by structural induction on the event sequence.

Base case: $C = [e_1]$ with $e_1.\text{type} = \text{REGISTER}$. Then $\delta(C) = \text{provisional}$. For any e_{new} :

- If $e_{\text{new}}.\text{type} \notin \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = [e_1]$, so $s' = f(\text{REGISTER}) = \text{provisional} = s$.
- If $e_{\text{new}}.\text{type} \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\text{type})$, which by the ActionExecutor precondition checking is permitted only if $(\text{provisional}, f(e_{\text{new}}.\text{type}))$ is a valid transition.

Inductive step: Assume the theorem holds for all chains of length n . Let C have length $n + 1$ with $\delta(C) = s$. Consider e_{new} :

- If $e_{\text{new.type}} \notin \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = C_{\text{life}}$, so $s' = s$ by definition of δ .
- If $e_{\text{new.type}} \in \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = C_{\text{life}} \oplus [e_{\text{new}}]$, so $s' = f(e_{\text{new.type}})$. By the ActionExecutor precondition system, e_{new} can only be appended if the precondition rules are satisfied against the current snapshot derived from C . These preconditions encode exactly the valid transitions of the status machine, so (s, s') must be a valid edge.

In both cases, the theorem holds for chains of length $n + 1$. By induction, it holds for all well-formed chains.

Assumptions used: (A1) The ActionExecutor correctly implements precondition checking against the current snapshot; (A2) The precondition rules in the action registry are complete (cover all valid transitions and exclude all invalid ones); (A3) The status transition machine is a well-defined directed graph on S . $\square$

E.4 Theorem E.4: Boundedness of Confidence

Confidence values in ADL Lite are guaranteed to lie in the unit interval $[0, 1]$, ensuring that confidence comparisons are well-behaved and preventing overflow in implementations.

Theorem E.4 (Boundedness). For any well-formed chain C , the confidence $\gamma(C) \in [0, 1]$.

Proof of Theorem E.4. We proceed by case analysis and induction on the number of VALIDATE events in C .

Base case: $V = [e \in C \mid e.type = \text{VALIDATE}] = []$. By definition, $\gamma(C) = 0.0 \in [0, 1]$.

Inductive step: Assume $\gamma(C) \in [0, 1]$ for a chain C with k distinct validators. Consider $C' = C \oplus [e_{\text{new}}]$ where $e_{\text{new.type}} = \text{VALIDATE}$ and $e_{\text{new.actor}}$ is a new actor (not in $actors(V)$).

By definition:

$$\gamma(C') = \min(1.0, c'_{\text{base}} + 0.05 \times (N'_{\text{vals}} - 1)) \quad (35)$$

where $N'_{\text{vals}} = N_{\text{vals}} + 1 = k + 1$ and:

$$c'_{\text{base}} = \max\left(0.5, \frac{k \cdot \text{mean}_{a \in actors(V)} \phi(a, V) + e_{\text{new.payload.confidence}}}{k + 1}\right). \quad (36)$$

By the precondition for VALIDATE (Theorem E.6 and system invariants), $e_{\text{new.payload.confidence}} \in [0, 1]$. The weighted mean is therefore a convex combination of values in $[0, 1]$, so it lies in $[0, 1]$. The $\max(0.5, \cdot)$ ensures $c'_{\text{base}} \geq 0.5$. The bonus term $0.05 \times k \geq 0$ is non-negative. Thus:

$$c'_{\text{base}} + 0.05 \times k \geq 0.5 > 0. \quad (37)$$

The outer $\min(1.0, \cdot)$ caps the value at 1.0. Therefore $\gamma(C') \in [0, 1]$.

If $e_{\text{new.actor}} \in actors(V)$ (duplicate validator), then $N'_{\text{vals}} = N_{\text{vals}}$ and c'_{base} uses argmax (retaining the higher confidence), so $\gamma(C') \geq \gamma(C) \geq 0$ and the $\min(1.0, \cdot)$ cap still applies.

Assumptions used: (A1) Individual confidence values $e.payload.confidence \in [0, 1]$ for all VALIDATE events; (A2) The $\min(1.0, \cdot)$ construction is present in the definition of γ ; (A3) The VALIDATE precondition requires confidence ≥ 0.5 (ensuring the lower bound). $\square$

E.5 Theorem E.5: Monotonicity of Confidence

Confidence never decreases when a VALIDATE event is appended. This monotonicity property ensures that the concept governance process is cumulative: additional validation can only increase (or preserve) confidence.

Theorem E.5 (Monotonicity of Confidence). Let C be a well-formed chain and let e_{new} be a VALIDATE event from an actor a with confidence $c \geq c_{\text{base}}$, where c_{base} is the current base confidence of C . Let $C' = C \oplus [e_{\text{new}}]$. Then:

$$\gamma(C') \geq \gamma(C). \quad (38)$$

Moreover, if $\gamma(C) < 1.0$ and a is a new validator, then $\gamma(C') > \gamma(C)$ (strict monotonicity).

Proof of Theorem E.5. Let $\gamma(C)$ be computed from V with $N_{\text{vals}} = |\text{actors}(V)|$ and base confidence $c_{\text{base}} = \max(0.5, c_{\text{base}}^{\text{raw}})$ where $c_{\text{base}}^{\text{raw}} = \text{mean}_{a \in \text{actors}(V)} \phi(a, V)$. We consider two cases.

Case 1: $a \notin \text{actors}(V)$ (new validator). Then $N'_{\text{vals}} = N_{\text{vals}} + 1$ and:

$$\gamma(C') = \min(1.0, c'_{\text{base}} + 0.05 \times N_{\text{vals}}) \quad (39)$$

where c'_{base} is the mean of $N_{\text{vals}} + 1$ per-actor maxima. Because $c \geq c_{\text{base}} \geq c_{\text{base}}^{\text{raw}}$, the weighted mean satisfies:

$$c'_{\text{base}} = \frac{N_{\text{vals}} \cdot c_{\text{base}}^{\text{raw}} + c}{N_{\text{vals}} + 1} \geq \frac{N_{\text{vals}} \cdot c_{\text{base}}^{\text{raw}} + c_{\text{base}}^{\text{raw}}}{N_{\text{vals}} + 1} = c_{\text{base}}^{\text{raw}}. \quad (40)$$

Therefore $c'_{\text{base}} = \max(0.5, c'_{\text{base}}) \geq \max(0.5, c_{\text{base}}^{\text{raw}}) = c_{\text{base}}$. The bonus term increases from $0.05 \times (N_{\text{vals}} - 1)$ to $0.05 \times N_{\text{vals}}$, adding exactly 0.05. Hence:

$$\gamma(C') = \min(1.0, c'_{\text{base}} + 0.05 \times N_{\text{vals}}) \geq \min(1.0, c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1) + 0.05) \geq \gamma(C). \quad (41)$$

If $\gamma(C) < 1.0$, the $\min(1.0, \cdot)$ cap is not binding for $\gamma(C)$, and the $+0.05$ bonus yields strict inequality.

For $N_{\text{vals}} = 0$ (first validator), $\gamma(C) = 0$ and $\gamma(C') = \min(1.0, \max(0.5, c)) \geq 0.5 > 0 = \gamma(C)$.

Case 2: $a \in \text{actors}(V)$ (duplicate validator). Then $N'_{\text{vals}} = N_{\text{vals}}$ and the bonus term is unchanged. The base confidence uses argmax:

$$\varphi'(a, V') = \max(\phi(a, V), c) \geq \phi(a, V). \quad (42)$$

Therefore $c'_{\text{base}} \geq c_{\text{base}}$, and since the bonus term is unchanged, $\gamma(C') \geq \gamma(C)$.

Assumptions used: (A1) Same actor's multiple VALIDATES use argmax (highest confidence retained); (A2) The bonus term 0.05 per additional distinct validator is positive; (A3) The new actor's confidence is at least the current base confidence c_{base} . $\square$

E.6 Theorem E.6: Status–Confidence Consistency

The final theorem establishes a critical consistency property between the status and confidence derivations: a concept with status *validated* must have positive confidence. This ensures that the status system and the confidence system are mutually coherent.

Theorem E.6 (Status–Confidence Consistency). If $\delta(C) = \text{validated}$, then $\gamma(C) > 0$.

Proof of Theorem E.6. Assume $\delta(C) = \text{validated}$. By the definition of δ , this implies that the last lifecycle event in C is VALIDATE:

$$\text{last}(C_{\text{life}}).type = \text{VALIDATE}. \quad (43)$$

Therefore C_{life} is non-empty and contains at least one `VALIDATE` event. Consequently, the validation set $V = [e \in C \mid e.\text{type} = \text{VALIDATE}]$ is non-empty: $V \neq \emptyset$.

Since $V \neq \emptyset$, $\gamma(C)$ is computed by the second branch of its definition:

$$\gamma(C) = \min(1.0, c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1)). \quad (44)$$

By the system invariant that every `VALIDATE` event carries confidence $e.\text{payload.confidence} \geq 0.5$, for every actor $a \in \text{actors}(V)$:

$$\phi(a, V) = \max\{e.\text{payload.confidence} \mid e \in V \wedge e.a = a\} \geq 0.5. \quad (45)$$

The base confidence is:

$$c_{\text{base}} = \max(0.5, \text{mean}_{a \in \text{actors}(V)} \phi(a, V)) \geq \max(0.5, 0.5) = 0.5. \quad (46)$$

Since $N_{\text{vals}} = |\text{actors}(V)| \geq 1$ (at least one validator exists), the bonus term satisfies:

$$0.05 \times (N_{\text{vals}} - 1) \geq 0.05 \times 0 = 0. \quad (47)$$

Therefore:

$$\gamma(C) = \min(1.0, c_{\text{base}} + \text{bonus}) \geq \min(1.0, 0.5 + 0) = 0.5 > 0. \quad (48)$$

This completes the proof. In fact, we have established the stronger result that $\gamma(C) \geq 0.5$ whenever $\delta(C) = \text{validated}$.

Assumptions used: (A1) The system invariant requires confidence ≥ 0.5 for every `VALIDATE` event; (A2) $\delta(C) = \text{validated}$ implies at least one `VALIDATE` event in C ; (A3) The $\min(1.0, \cdot)$ cap does not bind below 0.5. $\square$

E.7 Theorem 7: CRDT Convergence of EventChain Merge

The EventChain model supports an optional LWW-Set merge semantics for reconciling concurrent branches. This theorem establishes that the merge operation satisfies CRDT convergence properties.

Theorem E.7 (CRDT Convergence). For any well-formed concurrent branches B_1, B_2 derived from the same genesis event (same `concept_id` and genesis hash), the LWW-Set merge operation $\text{Merge}(B_1, B_2) = B_1 \cup B_2$ (deduplicated by `event_id`) satisfies:

$$\text{Merge}(B_1, B_2) = \text{Merge}(B_2, B_1) \quad (\text{commutativity}) \quad (49)$$

$$\text{Merge}(\text{Merge}(B_1, B_2), B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3)) \quad (\text{associativity}) \quad (50)$$

$$\text{Merge}(B_1, B_1) = B_1 \quad (\text{idempotence}) \quad (51)$$

Furthermore, after merge, $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined.

Proof of Theorem 7. We prove each CRDT property individually.

Commutativity. $\text{Merge}(B_1, B_2) = B_1 \cup B_2 = B_2 \cup B_1 = \text{Merge}(B_2, B_1)$ by commutativity of set union. Deduplication by `event_id` preserves this property because the deduplication function is symmetric: events with the same `event_id` appearing in both branches are included exactly once, regardless of which branch is the first argument.

Associativity. $\text{Merge}(\text{Merge}(B_1, B_2), B_3) = (B_1 \cup B_2) \cup B_3 = B_1 \cup (B_2 \cup B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3))$ by associativity of set union. The deduplication step is idempotent: applying it to $(B_1 \cup B_2) \cup B_3$

yields the same result as applying it to $B_1 \cup (B_2 \cup B_3)$ because the set of distinct `event_id` values is the same in both cases.

Idempotence. $\text{Merge}(B_1, B_1) = B_1 \cup B_1 = B_1$ because each event in B_1 appears exactly once after deduplication by `event_id`, which is already satisfied within B_1 (Condition (5) of Φ ensures pairwise distinct `event_id` values).

Determinism of δ after merge. The merged set $B_1 \cup B_2$ may have a non-trivial lifecycle subsequence where events from both branches interleave. However, for equal-timestamp ties, `event_id` provides a total order (UUIDs are lexicographically comparable), making the last lifecycle event unique. By Theorem E.1, δ is therefore uniquely determined.

Assumptions used: (A1) Both B_1 and B_2 satisfy Φ (well-formedness); (A2) `event_id` values are pairwise distinct across all events in $B_1 \cup B_2$ except for shared ancestry events (which are deduplicated); (A3) UUID lexicographic ordering provides a total order for tie-breaking. $\square$

E.8 Corollary: Event-Level CRDT (G-Set)

Corollary E.1 (Event-Level G-Set CRDT). Each ADL Lite event e is a trivial CRDT: it is immutable (no update operation), so trivially convergent under any merge strategy that treats events as set elements. The EventChain C as an append-only log is a grow-only set (G-Set) CRDT Shapiro et al. [2011]: events can only be added, never removed, and the merge of two chains is the union of their event sets.

Proof of Corollary E.1. An event e is immutable by construction: the hash $e.h$ is computed over all event fields (including `event_id` and payload), and any modification would change the hash, breaking the cryptographic linkage. Therefore, events have no update operation—they are write-once. A G-Set with write-once elements is trivially a CRDT because there are no concurrent updates to resolve: all concurrent operations are disjoint additions to the set, and the merge is simply set union.

The EventChain C by design supports only the append operation ($C \oplus [e]$), not remove or update. Therefore, C satisfies the G-Set CRDT semantics: append is monotonic (the event set only grows), and Merge is union. The cryptographic hash linkage ensures that the order of events is preserved within each branch even after merge, though the interleaving of events from different branches is not deterministic without external arbitration (see Theorem 7 for tie-breaking via `event_id` ordering).

Assumptions used: (A1) Events are immutable (hash-locked); (A2) append is the only chain mutation operation; (A3) Set union with `event_id` deduplication is the merge operation. $\square$

E.9 Summary of Proof Dependencies

The seven theorems and one corollary form a structured dependency graph:

- Theorem E.1 (Determinism) is foundational; no other theorem is used in its proof.
- Theorem E.2 (Confluence) depends on Theorem E.1.
- Theorem E.3 (Status Monotonicity) depends on Theorem E.1 and the ActionExecutor precondition system.
- Theorem E.4 (Boundedness) is independent of the others but shares assumptions with Theorem E.5.
- Theorem E.5 (Confidence Monotonicity) depends on Theorem E.4 for the cap argument and on the argmax semantics for duplicate validators.

- Theorem E.6 (Consistency) depends on Theorem E.4 (for the cap not interfering) and on the VALIDATE precondition system.
- Theorem E.7 (CRDT Convergence) depends on Theorem E.1 (for determinism of δ after merge) and on the well-formedness predicate Φ (for pairwise distinct event_id values).
- Corollary E.1 (G-Set CRDT) follows from the immutability of events and the append-only semantics of EventChains.

All seven proofs together establish that ADL Lite’s derivation semantics is deterministic, confluent under fork, transition-monotonic, bounded, confidence-monotonic, internally consistent, and CRDT-convergent under LWW-Set merge. These properties provide the formal foundation for the system’s operational correctness.

E.10 Sensitivity Analysis: Confidence Aggregation Parameters

Table 19 reports $\gamma(C)$ under three $(\beta, c_{\min})$ parameter combinations for four validator-confidence scenarios (three validators each). The base confidence $c_{\text{base}} = \max(c_{\min}, \text{mean}(\phi))$ and $\gamma(C) = \min(1.0, c_{\text{base}} + \beta \times (N_{\text{vals}} - 1))$.

Table 19: Sensitivity of $\gamma(C)$ to bonus β and floor $c_{\min}$ (three validators).

Validator confidences	$\beta = 0.03, c_{\min} = 0.3$	$\beta = 0.05, c_{\min} = 0.5$	$\beta = 0.08, c_{\min} = 0.7$
[0.6, 0.6, 0.6] (low)	0.66	0.70	0.86
[0.7, 0.7, 0.7] (medium)	0.76	0.80	0.86
[0.9, 0.9, 0.9] (high)	0.96	1.00	1.00
[0.5, 0.7, 0.9] (mixed)	0.76	0.80	0.86

Theorems E.4–E.6 hold for all entries because: (i) $\gamma(C) \in [0, 1]$ by the $\min(1.0, \cdot)$ cap; (ii) confidence monotonicity holds because $\beta > 0$; (iii) status-confidence consistency holds because $c_{\min} \geq 0.3$ and any validated concept has $c_{\text{base}} \geq c_{\min}$. With the precondition $c \geq c_{\text{base}}$, strict monotonicity is preserved for all $\beta > 0$; the $\beta \leq c_{\min}/10$ bound required by the earlier formulation (which used $c \geq 0.5$) is no longer necessary.

F RDF-star Interoperability Demonstration

G RDF-star Interoperability Demonstration

This appendix demonstrates the concrete interoperability path from ADL Lite EventChains to RDF-star, including export, query, and reversible mapping.

G.1 EventChain to PROV-O Export

Consider the EventChain for concept `disc-capital-trap` with two events: REGISTER (by discoverer) and VALIDATE (by reviewer, confidence 0.85). The PROV-O export is:

```

1 @prefix prov: <http://www.w3.org/ns/prov#> .
2 @prefix adl: <https://adl-lite.org/ns/> .
3
4 adl:evt-001 a prov:Activity ;

```

```

5     prov:wasAssociatedWith adl:actor-discoverer ;
6     adl:eventType "REGISTER" ;
7     adl:previousEvent "genesis" ;
8     adl:hash "sha256:abc123..." .
9
10    adl:evt-002 a prov:Activity ;
11        prov:wasAssociatedWith adl:actor-reviewer ;
12        prov:wasInformedBy adl:evt-001 ;
13        adl:eventType "VALIDATE" ;
14        adl:payloadConfidence 0.85 ;
15        adl:hash "sha256:def456..." .

```

G.2 PROV-O to RDF-star Conversion

RDF-star enables statement-level annotation, which is natural for ADL Lite events (each event annotates a statement about the concept’s lifecycle). The conversion:

```

1  # The concept’s validation as an RDF-star triple
2  << adl:disc-capital-trap adl:status "validated" >>
3      adl:validator adl:actor-reviewer ;
4      adl:confidence 0.85 ;
5      adl:eventHash "sha256:def456..." ;
6      adl:timestamp "2024-01-15T14:30:00Z" .
7
8  # A relation as RDF-star
9  << adl:capital-trap adl:isomorphic-to adl:gradient-explosion >>
10      adl:confidence 0.92 ;
11      adl:mappingType "topological" ;
12      adl:validator adl:actor-reviewer .

```

G.3 SPARQL-star Query

```

1  SELECT ?concept ?status ?validator ?confidence
2  WHERE {
3      << ?concept adl:status ?status >>
4          adl:validator ?validator ;
5          adl:confidence ?confidence .
6      FILTER (?confidence > 0.8)
7  }

```

This query retrieves all concepts with validated status and confidence > 0.8, including the validator identity and confidence value—information that requires reification in standard RDF but is first-class in RDF-star.

G.4 Reversible Mapping

The mapping ADL Lite → RDF-star is *lossy* in one direction: RDF-star discards the cryptographic hash chain (it captures the *semantics* of events but not their *integrity evidence*). However, a *reversible* mapping is possible by extending the RDF-star representation with provenance metadata:

```

1  << adl:disc-capital-trap adl:status "validated" >>
2      adl:provenance adl:evt-002 .

```

```

3
4 adl:evt-002 a adl:Event ;
5   adl:hash "sha256:def456..." ;
6   adl:previousEventHash "sha256:abc123..." .

```

With this extension, an ADL Lite system can: (1) export to RDF-star for SPARQL querying and Semantic Web interoperability, (2) import from RDF-star by reconstructing the EventChain from `adl:provenance` links, and (3) verify integrity by checking that the reconstructed chain’s hashes match the stored `adl:hash` values.

References

- Sabah Al-Fedaghi. Occurrence-Only Modeling: Events as Fundamental Ontological Primitives. In *Applied Ontology*, 2024.
- Paulo Sérgio Almeida and Ehud Shapiro. The Blocklace: A Universal, Byzantine Fault-Tolerant, Conflict-free Replicated Data Type. *arXiv preprint arXiv:2402.08068*, 2024. Submitted to Distributed Computing.
- Robert Arp, Barry Smith, and Andrew D. Spear. *Building Ontologies with Basic Formal Ontology*. MIT Press, 2015.
- Authors TBD. AgentSafe: A Unified Governance Framework for LLM Agents. *arXiv preprint arXiv:2512.03180*, 2025a. Unified governance: design-time + runtime + audit.
- Authors TBD. From Agent Traces to Trust: A Unified Provenance Framework for LLM Agents. *arXiv preprint arXiv:2606.04990*, 2025b. Unified provenance framework; explicitly states unified trace schema is still needed.
- Authors TBD. KYA: A Framework-Agnostic Trust Layer for LLM Agents. *arXiv preprint arXiv:2605.25376*, 2025c. Framework-agnostic trust layer with HMAC chain, 1,800 ops/sec, 15+ adapters, veldt-ky on PyPI.
- Authors TBD. SafeAgent: A Safety Protocol for LLM Agents. *arXiv preprint arXiv:2604.17562*, 2025d. Protocol-level safety for LLM agents.
- Authors TBD. Multi-Agent LLM Ontology Engineering. *arXiv preprint arXiv:2604.23090*, 2025e. Multi-agent LLM OE: Domain Expert→Manager→Coder→QA workflow.
- Davide Francesco Barbieri, Daniele Braga, Stefano Ceri, Emanuele Della Valle, and Michael Grossniklaus. C-SPARQL: A Continuous Query Language for RDF Data Streams. In *ISWC*, 2009.
- Tim Berners-Lee, James Hendler, and Ora Lassila. The Semantic Web. *Scientific American*, 284(5): 34–43, 2001.
- Alexander Boldachev. Reactive Subject-Event Ontology: Causal Guards over Event Graphs. In *arXiv preprint arXiv:2503.XXXXX*, 2025.
- Diego Calvanese, Giuseppe De Giacomo, Domenico Lembo, Maurizio Lenzerini, and Riccardo Rosati. Tractable Reasoning and Efficient Query Answering in Description Logics: The DL-Lite Family. *Journal of Automated Reasoning*, 39(3):385–429, 2007.

- Miguel Castro and Barbara Liskov. Practical Byzantine Fault Tolerance. In *OSDI*, 1999.
- Scott A. Crosby and Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. *USENIX Security*, 2009.
- Rasmus Dahlberg, Tobias Pulls, Roel Peeters, Dirk Westhoff, and Nathalie Mitton. Efficient Sparse Merkle Trees: Reducing Proof Size for Transparent and Verifiable Data Structures. In *NordSec*, pages 59–74. Springer, 2020. doi: 10.1007/978-3-030-69906-9_4.
- Dimitrios Doumanas, Georgios Bouchouras, Andreas Soularidis, Konstantinos Kotis, and George A. Vouros. From Human- to LLM-Centered Collaborative Ontology Engineering. *Applied Ontology*, 2025. doi: 10.3233/AO-240023.
- Kit Fine. Ontological Dependence. *Proceedings of the Aristotelian Society*, 95:269–290, 1995.
- Foam Contributors. Foam: A Personal Knowledge Management and Sharing System for VSCode. <https://github.com/foambubble/foam>, 2020. Open-source VSCode extension for personal knowledge graphs. Accessed: 2025-01-15.
- Aldo Gangemi, Nicola Guarino, Claudio Masolo, Alessandro Oltramari, and Luc Schneider. Sweetening Ontologies with DOLCE. *Knowledge Engineering and Knowledge Management: Ontologies and the Semantic Web*, pages 166–181, 2002.
- Birte Glimm, Ian Horrocks, Boris Motik, Giorgos Stoilos, and Zhe Wang. Hermit: An OWL 2 Reasoner. *Journal of Automated Reasoning*, 53(3):245–269, 2014. ISSN 0168-7433. doi: 10.1007/s10817-014-9305-1.
- Jonathan Gruss, Andrei Ciortea, Guido Salvaneschi, and Simon Mayer. Real-Time Collaboration in Linked Data Systems. In *Proceedings of the ISWC 2023 Posters, Demos and Industry Tracks*, volume 3632 of *CEUR Workshop Proceedings*, 2023. Applies CRDTs to RDF resource editing in Solid Pods.
- Ramanathan V. Guha, Dan Brickley, and Steve Macbeth. Schema.org: Evolution of Structured Data on the Web. *Communications of the ACM*, 59(2):44–51, 2016. ISSN 0001-0782. doi: 10.1145/2844544.
- Giancarlo Guizzardi. *Ontological Foundations for Structural Conceptual Models*. PhD thesis, University of Twente, 2005.
- Giancarlo Guizzardi et al. UFO-B: Executable Event-Driven Foundational Ontology Specifications. In *ER 2024*, 2024.
- Yaroslav O. Halchenko, Kyle Meyer, Benjamin Poldrack, Debanjum S. Solanky, Alexander S. Wagner, Jason Gors, David MacFarlane, Doreen Pustina, Vanessa Sochat, Satrajit S. Ghosh, Christian Mönch, Christopher J. Markiewicz, Laura Waite, Ilya Shlyakhter, Alejandro de la Vega, Soichi Hayashi, Christian O. Häusler, Jean-Baptiste Poline, Tobias Kadelka, et al. DataLad: Distributed System for Joint Management of Code, Data, and Their Relationship. *Journal of Open Source Software*, 6(63):3262, 2021. doi: 10.21105/joss.03262.
- Ahmad Hemid, Waleed Shabbir, Abderrahmane Khiat, Christoph Lange-Bever, Christoph Quix, and Stefan Decker. OntoEditor: Real-Time Collaboration via Distributed Version Control for Ontology Development. In *The Semantic Web: ESWC 2024 Satellite Events*, volume 14664 of *Lecture Notes*

- in *Computer Science*, pages 326–341. Springer, 2024. doi: 10.1007/978-3-031-60626-7_18. Uses Operational Transformation (OT), not CRDTs.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. *Knowledge Graphs*. Number 22 in Synthesis Lectures on Data, Semantics, and Knowledge. Springer, 2021. ISBN 9783031007903. doi: 10.2200/S01125ED1V01Y202109DSK022.
- IBM. IBM AML Simulated Dataset. <https://www.kaggle.com/datasets/berkanozdemir/aml-simulated>, 2024. Accessed: 2025-01-01.
- ICOM-CIDOC. CIDOC Conceptual Reference Model. ISO 21127:2014, 2014.
- Robert Kowalski and Marek Sergot. A Logic-Based Calculus of Events. *New Generation Computing*, 4(1):67–95, 1986.
- Tobias Kuhn and Michel Dumontier. Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data. *Journal of Biomedical Semantics*, 6, 2015.
- Tobias Kuhn, Christine Chichester, Michael Krauthammer, and Michel Dumontier. Nanopublications: A Growing Resource of Provenance-Centric Scientific Linked Data. In *IEEE eScience*, 2015.
- Danh Le-Phuoc, Minh Dao-Tran, Josiane Xavier Parreira, and Manfred Hauswirth. CQELS: A Native and Adaptive Approach for Unified Querying Linked Data Streams. In *ISWC*, pages 694–709, 2011.
- Kevin Lin. Dendron: A Hierarchical Tool for Thought. <https://wiki.dendron.so/>, 2020. Open-source knowledge management tool for VSCode. Accessed: 2025-01-15.
- Martin Kleppmann Martin et al. Automerge: A JSON CRDT for Real-Time Collaborative Editing. *ACM SIGOPS Operating Systems Review*, 2020.
- John McCarthy and Patrick J. Hayes. Some Philosophical Problems from the Standpoint of Artificial Intelligence. In *Machine Intelligence 4*, pages 463–502. Edinburgh University Press, 1969.
- Kevin Nicolai and Peter Dadam. Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types. *Proceedings of the ACM on Human-Computer Interaction*, 2021.
- Abel Nieto et al. Formally Verified CRDTs in Iris Separation Logic. In *POPL 2024*, 2024.
- Obsidian Team. Obsidian: A Knowledge Base on Top of a Local Folder of Plain Text Markdown Files. <https://obsidian.md/>, 2024. Accessed: 2025-01-15.
- Ontotext. RDF-star Support in GraphDB. In *GraphDB Documentation*, 2024.
- Palantir Technologies. Palantir Foundry Data Engine. <https://www.palantir.com/platforms/foundry/>, 2024.
- John R. Searle. *The Construction of Social Reality*. Free Press, New York, 1995.
- Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-Free Replicated Data Types. *Lecture Notes in Computer Science*, 6976, 2011.

- Ryan Shaw, Raphael Troncy, and Lynda Hardman. LODÉ: Linking Open Descriptions of Events. In *The Semantic Web: Research and Applications*, pages 153–167, 2011.
- Barry Smith, Michael Ashburner, Cornelius Rosse, Jonathan Bard, William Bug, Werner Ceusters, Louis J. Goldberg, Karen Eilbeck, Amelia Ireland, Christopher J. Mungall, et al. The OBO Foundry: Coordinated Evolution of Ontologies to Support Biomedical Data Integration. *Nature Biotechnology*, 25(11):1251–1255, 2007. doi: 10.1038/nbt1346.
- Barry Smith, Werner Ceusters, Bert Klagges, Jacob Koehler, Anand Kumar, Jane Lomax, Chris Mungall, Fabian Neuhaus, Alan L. Rector, and Cornelius Rosse. The Information Artifact Ontology (IAO). *BMC Bioinformatics*, 11(1):83, 2010. doi: 10.1186/1471-2105-11-83.
- Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, Björn Grüning, Marco La Rosa, Simone Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, RO-Crate Community, Paul Groth, and Carole Goble. Packaging Research Artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. doi: 10.3233/DS-210053.
- Andreas Soularidis, Dimitrios Doumanas, Konstantinos Kotis, and George A. Vouros. Automating Agentic Collaborative Ontology Engineering with Role-Playing Simulation of LLM-Powered Agents and RAG Technology. *The Knowledge Engineering Review*, 40, 2025. doi: 10.1017/S026988892510009X.
- Manu Sporny, Gregg Kellogg, and Markus Lanthaler. JSON-LD 1.1: A JSON-based Serialization for Linked Data. W3C, 2020.
- TerminusDB/DFRNT. TerminusDB: A Git-like Knowledge Graph Database. <https://terminusdb.org/>, 2024. Open-source graph database with immutable, version-controlled RDF storage. Accessed: 2025-01-15.
- Thanh Luong Tuan and Abhijit Sanyal. Ontology-Constrained Neural Reasoning in Enterprise Agentic Systems: A Neurosymbolic Architecture for Domain-Grounded AI Agents. *arXiv preprint arXiv:2604.00555*, 2026. Foundation AgenticOS (FAOS) platform; three-layer ontology (Role, Domain, Interaction) for grounding enterprise LLM agents.
- Willem Robert van Hage, Veronique Malaise, Gerben de Vries, Guus Schreiber, and Maarten van Someren. The Simple Event Model (SEM). *The Semantic Web: Research and Applications*, pages 381–395, 2011.
- W3C. PROV-O: The PROV Ontology. <https://www.w3.org/TR/prov-o/>, 2013.
- W3C. SHACL Shapes Constraint Language. <https://www.w3.org/TR/shacl/>, 2017.
- W3C. RDF Dataset Canonicalization (URDNA2015). <https://www.w3.org/TR/rdf-canon/>, 2023.
- W3C DID Working Group. Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, 2022. URL <https://www.w3.org/TR/did-core/>.
- W3C OWL Working Group. OWL 2 Web Ontology Language: Document Overview. In *W3C Recommendation*, 2012.
- W3C RDF-star Working Group. RDF 1.2: RDF-star and SPARQL-star. W3C Working Draft, 2024.
- W3C RDF Stream Processing Community Group. RDF Stream Processing: Requirements and Design Rationale. W3C, 2013.

W3C Verifiable Credentials Working Group. Verifiable Credential Data Integrity 1.0. W3C Candidate Recommendation, 2024.

Ludwig Wittgenstein. *Tractatus Logico-Philosophicus*. Routledge & Kegan Paul, 1922.

Greg Young. Event Sourcing: State from Events. *CodeBetter*, 2010.

Greg Young. Command and Query Responsibility Segregation (CQRS) Pattern. *Microsoft Patterns & Practices*, 2012.